

RAPPORT DE STAGE DE FIN D'ÉTUDES

Présenté en vue de l'obtention du

Diplôme National De Mastère Professionnel

Spécialité : Sécurité des Systèmes D'Information et des Infrastructures

Par

Belhsan HAJOUNI et Yasmine MANSOURI

Automatisation de la sécurité de l'infrastructure avec Terraform et Ansible

Encadrant professionnel : Madame Syrine MANSOURI

Encadrant académique : Docteur Aymen AYARI

Réalisé au sein de Cloud Temple Tunisia



RAPPORT DE STAGE DE FIN D'ÉTUDES

Présenté en vue de l'obtention du

Diplôme National De Mastère Professionnel

Spécialité : Sécurité des Systèmes D'Information et des Infrastructures

Par

Belhsan HAJOUNI et Yasmine MANSOURI

Automatisation de la sécurité de l'infrastructure avec Terraform et Ansible

Encadrant professionnel : Madame Syrine MANSOURI

Encadrant académique : Docteur Aymen AYARI

Réalisé au sein de Cloud Temple Tunisia



J'autorise les étudiants **Yasmine MANSOURI** et **Belhsan HAJOUNI** à déposer leur rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Madame Syrine MANSOURI**

Signature et cachet

J'autorise les étudiants **Yasmine MANSOURI** et **Belhsan HAJOUNI** à déposer leur rapport de stage en vue d'une soutenance.

Encadrant académique, **Docteur Aymen AYARI**

Signature

Remerciements

Nous adressons nos remerciements les plus sincères à toute l'équipe de Cloud Temple Tunisia pour nous avoir assistés dans notre projet.

*Nous remercions aussi **Madame Syrine MANSOURI** notre encadrante à l'entreprise, pour ses directives pertinentes et sa bonne collaboration.*

*Nous sommes honorées d'avoir été encadrées par **Docteur Aymen AYARI** à l'ISI et le remercions vivement pour ses remarques et ses conseils avertis.*

Nous tenons à remercier les enseignants tout au long de notre cursus universitaire et les féliciter pour les grands efforts qu'ils fournissent pour nous offrir une formation consistante et polyvalente.

Enfin, nous tenons à adresser nos très sincères remerciements aux membres du jury, tout en espérant qu'ils trouveront dans ce rapport la clarté et la motivation escomptées.

Table des matières

Introduction générale	1
1 Etude Préalable du Sujet	3
Introduction	4
1 Présentation de l'organisme d'accueil	4
1.1 Introduction	4
1.2 Secteur d'activité	4
1.3 Services	5
1.4 Organigramme	5
2 Présentation du projet	6
2.1 Etude de l'existant : Analyse de l'infrastructure existante	7
2.2 Critique de l'existant	7
2.3 Problématique	7
2.4 Solution proposées	7
3 Approche méthodologique	8
3.1 Généralités	8
3.2 Méthode agile	9
3.3 Déroulement du projet	10
Conclusion	10
2 État de l'art	11
Introduction	12
1 La Virtualisation	12
1.1 La virtualisation des serveurs	12
1.2 La virtualisation des réseaux	12
1.3 La virtualisation du stockage	13
1.3.1 DAS	13
1.3.2 NAS	14
1.3.3 SAN	14

1.4	Types d'hyperviseurs	15
1.4.1	Hyperviseur de type 1	15
1.4.2	Hyperviseur de type 2	15
1.4.3	Comparaison entre les solutions d'Hypervision	16
2	Centre de données	17
2.1	Classification des Datacenters	18
3	Le Cloud Computing	18
3.1	Les caractéristiques du Cloud Computing	19
3.2	Les modèles de déploiement du Cloud Computing	19
3.2.1	Le Cloud public	20
3.2.2	Le Cloud privé	20
3.2.3	Le cCloud hybride	21
3.2.4	Le Cloud communautaire	21
3.3	Les modèles du service Cloud Computing	21
3.3.1	Infrastructure as a Service (IaaS)	22
3.3.2	Platform as a Service (PaaS)	22
3.3.3	Software as a Service (SaaS)	22
3.4	La sécurité dans le Cloud Computing	22
3.4.1	La confidentialité	23
3.4.2	L'intégrité	23
3.4.3	La disponibilité	23
4	DevOps	23
4.1	Infrastructures as Code (IaC)	24
4.2	Approche CI/CD	25
4.2.1	Définition	25
4.2.2	Intégration continue (CI)	25
4.2.3	Distribution continue et déploiement continu (CD)	26
5	DevSecOps	26
6	Choix Technologiques	27
6.1	Choix de solutions Cloud	28
6.2	Système de gestion de version	29

6.3	CI/CD	30
6.4	Choix d'outil pour le provisionnement de l'infrastructure	31
6.5	Choix d'outil pour la gestion des secrets	32
6.6	Choix d'outil pour l'analyse de conformité	33
6.7	Résultats des comparaisons	34
7	Analyse et définition des besoins	35
7.1	Besoins Fonctionnels	35
7.2	Besoins Non-Fonctionnels	36
	Conclusion	37
3	Étude conceptuelle	38
	Introduction	39
1	Présentation de l'architecture logicielle	39
2	Description de la conception du logiciel	40
2.1	Couche gestion des secrets	41
2.1.1	Architecture logique	41
2.1.2	Diagramme d'interaction	41
2.2	Couche provisionnement	42
2.2.1	Architecture logique	42
2.2.2	Diagramme d'interaction	43
2.3	Couche configuration	44
2.3.1	Architecture logique	44
2.3.2	Diagramme d'interaction	44
2.4	Couche analyse de conformité	45
2.4.1	Architecture logique	45
2.4.2	Diagramme d'interaction	45
2.5	Couche automatisation	46
2.5.1	Architecture logique	46
2.5.2	Diagramme d'interaction	46
3	Présentation de l'architecture de la solution envisagée	47
3.1	Architecture de la détection et réponse aux endpoints	47
	Conclusion	49

4	Réalisation	50
	Introduction	51
1	Environnement de travail	51
1.1	Environnement matériel	51
1.2	Environnement logiciel	51
2	Implémentation de solution	52
2.1	La phase de gestion des secrets	52
2.2	La phase de provisionnement avec Terraform	53
2.3	La phase de configuration automatisé anc Ansible	55
2.4	La phase de l'analyse de conformité et EDR	59
2.5	La phase de l'automatisation	64
3	Résultat	66
3.1	Résultat de la configuration automatisée	67
3.2	Résultat de l'analyse de conformité	68
3.3	Simulation des attaques	70
3.3.1	Détection et blocage des attaques par force brute SSH	70
3.3.2	Détection et suppression des logiciels malveillants à l'aide de l'intégration avec VirusTotal	72
	Conclusion	73
	Conclusion générale et Perspectives	74

Table des figures

1.1	Naissance de Cloud Temple Tunisia	4
1.2	Organigramme de Cloud Temple Tunisia	6
1.3	Diagramme de GANTT	10
2.1	Les différentes méthodes d'accès du stockage [4]	13
2.2	Comparaison entre les types d'hyperviseurs	16
2.3	Modèles de déploiement du CCloud	20
2.4	Les niveaux de services du Cloud Computing [15]	21
2.5	Cycle de vie DevOps [19]	23
2.6	L'approche CI/CD	25
2.7	L'intégration continue	26
2.8	L'approche DevSecOps	27
3.1	Architecture logicielle	39
3.2	Architecture logique (Gestion des secrets)	41
3.3	Diagramme d'interaction (Gestion des secrets)	42
3.4	Architecture logique (Provisionnement)	43
3.5	Diagramme d'interaction (Provisionnement)	43
3.6	Architecture logique (Configuration)	44
3.7	Diagramme d'interaction (Configuration)	44
3.8	Architecture logique (Analyse de conformité)	45
3.9	Diagramme d'interaction (Analyse de conformité)	45
3.10	Architecture logique (Automatisation)	46
3.11	Diagramme d'interaction (Automatisation)	46
3.12	Architecture de la solution envisagée	47
3.13	Architecture de EDR	48
4.1	Authentification et autorisation	52
4.2	Configuration de vault pour stocker un secret	52
4.3	Integration de vault dans Terraform	53

4.4	Configuration des Variables dans Terraform	53
4.5	Script Principal de Provisionnement	54
4.6	Résultat du Déploiement sur vSphere	55
4.7	Configuration automatique des adresses IP	55
4.8	Le role d'installation de Docker	56
4.9	Playbook d'installation de Docker	56
4.10	Le role d'installation de JDK	57
4.11	Playbook d'installation de JDK	57
4.12	Playbook d'installation des agents Wazuh	58
4.13	Le role des agents Wazuh	58
4.14	Fichier de configuration de Wazuh Indexer	59
4.15	Script de l'installation de Wazuh Indexer	60
4.16	Script de l'installation de Wazuh Server	60
4.17	Script de l'installation de Wazuh Dashboard	61
4.18	Dashboard Wazuh	61
4.19	Configuration de la réponse active	62
4.20	Commande de blocage du pare-feu	62
4.21	Définition des règles locales	62
4.22	Script de suppression des menaces	63
4.23	Configuration des répertoires à surveiller	63
4.24	Fichier Jenkins pour la provisionnement automatique avec Terraform	64
4.25	Fichier Jenkins pour la configuration automatique des IPs	65
4.26	Jenkinksfile de l'installation automatique de JDK et Docker	65
4.27	Jenkinksfile de l'installation automatique des agnets Wazuh	66
4.28	Lancement du pipeline Terraform avec succès	66
4.29	Résultat du playbook Ansible sur l'écran du console Jenkins	67
4.30	Résultat du cryptage par Vault	68
4.31	Installation automatisée des agents Wazuh avec succès	68
4.32	Analyse de conformité	69
4.33	SCA détaillé pour la vérification des permissions de fichiers système	69
4.34	Attaque force brute SSH	70

4.35 Les alertes affichées sur Wazuh dashboard	71
4.36 Fichier JSON détaillé de la réponse active firewall-drop	71
4.37 Téléchargement du fichier malveillant dans le répertoire /root	72
4.38 Résultat de l'intégration	72

Liste des tableaux

2.1	Tableau comparatif entre les solutions d’hypervision	16
2.2	Tableau comparatif entre VMware vSphere et OpenStack	28
2.3	Tableau comparatif entre GitHub, GitLab CE	29
2.4	Etude comparative des outils CI/CD	30
2.5	Comparaison des outils de gestion d’infrastructure	31
2.6	Tableau comparatif entre Vault et StrongDM	32
2.7	Comparaison entre Wazuh et OpenSCAP	33
2.8	Tableau de synthèse des choix technologiques	34
4.1	Environnement locale	51
4.2	Environnement Cloud	51
4.3	Environnement logiciel	51

Liste des abréviations

—	CTT	=	C loud T emple T unisia
—	DAS	=	D irect A ttached S torage
—	EDR	=	E ndpoint D etection and R esponse
—	FC	=	F iber C hannel
—	HBA	=	H ost B us of L evel and A daptators
—	HCL	=	H ashiCorp C onfiguration L anguage
—	IP	=	I nternet P rotocol
—	IaC	=	I nfrastructure a s C ode
—	IaaS	=	I nfrastructure a s a S ervice
—	MAC	=	M edia A ccess C ontrol
—	NAS	=	N etwork A ttached S torage
—	NIST	=	N ational I nstitute of S tandards and T echnology
—	PaaS	=	P latform a s a S ervice
—	SAN	=	S torage A rea N etwork
—	SCA	=	S ecurity C onfiguration A ssessment
—	SLA	=	N ational S ervice of L evel and A greement
—	SSH	=	S ecure S hell
—	SaaS	=	S oftware a s a S ervice
—	VDC	=	V irtuel D atacenter
—	VM	=	V irtual M achine
—	WBS	=	W ork B reakdown S tructure
—	WWN	=	W orld W ide N ame

Introduction générale

De nos jours, le Cloud Computing représente un challenge majeur dans le monde informatique, notamment dans les domaines des systèmes d'information et de communication. Le concept de Cloud est largement répandu au sein des entreprises, quelles que soient leurs activités, car il permet d'accélérer leur croissance économique tout en se déchargeant des infrastructures physiques (serveurs, etc.) et des coûts qui y sont associés (refroidissement, sécurité, etc.) grâce à une mutualisation efficace des ressources. Cependant, l'évolution des architectures physiques et logiques des systèmes d'information représente un défi majeur pour les entreprises en termes de fiabilité, disponibilité, performance et coût. Cela nécessite une solution sophistiquée et évoluée pour rester compétitif sur le marché.

Dans ce contexte, l'automatisation sécurisée du déploiement et de configuration de l'infrastructure représente une approche révolutionnaire. En mettant en œuvre cette solution, les entreprises peuvent déployer leur propre cloud privé dans un centre de données automatisé et sécurisé, répondant ainsi de manière efficace et efficiente à l'évolution du marché. Grâce aux performances élevées que les centres de données offrent, cette automatisation assure des ressources de calcul et de stockage à la demande, le tout avec une connexion à très haut débit, rendant l'entreprise plus agile et compétitive.

Ce projet vise à implémenter les meilleures pratiques de sécurité de manière automatisée, en utilisant des technologies de pointe. L'objectif est d'assurer une infrastructure sécurisée et conforme, capable de résister aux menaces actuelles et futures, tout en minimisant les interventions manuelles et les erreurs humaines. Pour cela, nous allons concevoir et mettre en place une architecture cible optimale, en choisissant les meilleures solutions pour assurer la flexibilité, la reproductibilité et la scalabilité du processus.

Pour répondre aux différentes exigences posées par le sujet, une bonne compréhension de toutes les perspectives s'avère indispensable. C'est la raison pour laquelle nous avons adopté le plan suivant : Le premier chapitre est un chapitre introductif qui a pour but de présenter l'entreprise d'accueil et le cadre de notre projet, c'est-à-dire le contexte général du projet, la problématique ainsi que la solution proposée. Le deuxième chapitre abordera les aspects fondamentaux d'une

infrastructure virtuelle, où nous expliquerons d'une façon détaillée les technologies utilisées tout au long de ce projet ainsi que la spécification des besoins. Le troisième chapitre décrira la conception. Le troisième chapitre présentera l'architecture logicielle de notre projet, détaillant les différents composants et leur intégration. Il décrira également la conception du logiciel pour répondre aux exigences du projet. Le quatrième chapitre décrira l'environnement matériel nécessaire pour le déploiement de notre solution et l'architecture système qui en découle. Il détaillera ensuite le travail réalisé, notamment la configuration des serveurs, l'intégration des outils de sécurité et l'automatisation des processus de déploiement et de surveillance. Nous finirons ce rapport par une conclusion générale, dans laquelle nous allons récapituler le travail réalisé et exposer les perspectives éventuelles du projet.

ETUDE PRÉALABLE DU SUJET

Plan

Introduction	4
1 Présentation de l'organisme d'accueil	4
2 Présentation du projet	6
3 Approche méthodologique	8
Conclusion	10

Introduction

Dans ce chapitre introductif, nous allons définir le cadre global de notre projet en introduisant l'organisme d'accueil « Cloud Temple Tunisia » dans lequel nous avons effectué ce stage. Nous allons ensuite présenter le contexte du projet afin de définir les besoins et les contraintes de l'entreprise cliente.

1 Présentation de l'organisme d'accueil

1.1 Introduction

Cloud Temple Tunisia est une entreprise de services numériques résultant d'un partenariat entre le groupe tunisien Poulina Group Holding, actif dans divers secteurs, et Dragonfly (ex-Intrinsec), une filiale du groupe Neurones, une entreprise française spécialisée dans la sécurité informatique, l'hébergement et l'infogérance Cloud. Cette alliance a été créée le 14 février 2014, avec un capital initial de 2,5 millions de dinars, dans le but d'établir CTT en tant que prestataire de services cloud.



FIGURE 1.1 : Naissance de Cloud Temple Tunisia

1.2 Secteur d'activité

Cloud Temple Tunisia a développé son activité sur les trois piliers suivants :

[label=●,font=,leftmargin=2.5cm]

- Cloud Computing
- Sécurité informatique
- Infogérance 24/7

CTT se distingue par son expertise dans la migration des systèmes d'information essentiels vers le Cloud, ainsi que dans la sécurisation des environnements informatiques d'entreprise. Cette expertise a été enrichie progressivement par des services d'infogérance 24/7. En effet, Cloud Temple propose des infrastructures flexibles à la demande, offrant les niveaux de performance et de sécurité les plus élevés disponibles sur le marché.

1.3 Services

CTT propose une solution « IaaS » à travers son offre de VDC. En combinant les atouts du Cloud privé et public, cette offre permet aux clients de se focaliser sur leurs activités principales tout en bénéficiant d'une plus grande souplesse, disponibilité et compétitivité pour répondre aux exigences du marché.

Voici les offres qu'ils proposent à leurs clients :

[label=●,font=,leftmargin=2.5cm]

- Offre de serveurs gérés
- Offre de VDC
- Offre infogérance
- Offre de Plan de Reprise d'Activité et Plan de Continuité de'Activité (PRA/PCA)

1.4 Organigramme

L'organigramme de la figure 1.2 montre comment la structure a été divisée par secteurs de responsabilité et par domaines de compétences au sein de l'entreprise Cloud Temple qui est constituée de trois parties : L'administration, l'équipe technique et le marketing. Notre projet de stage s'est déroulé au sein de l'équipe technique.

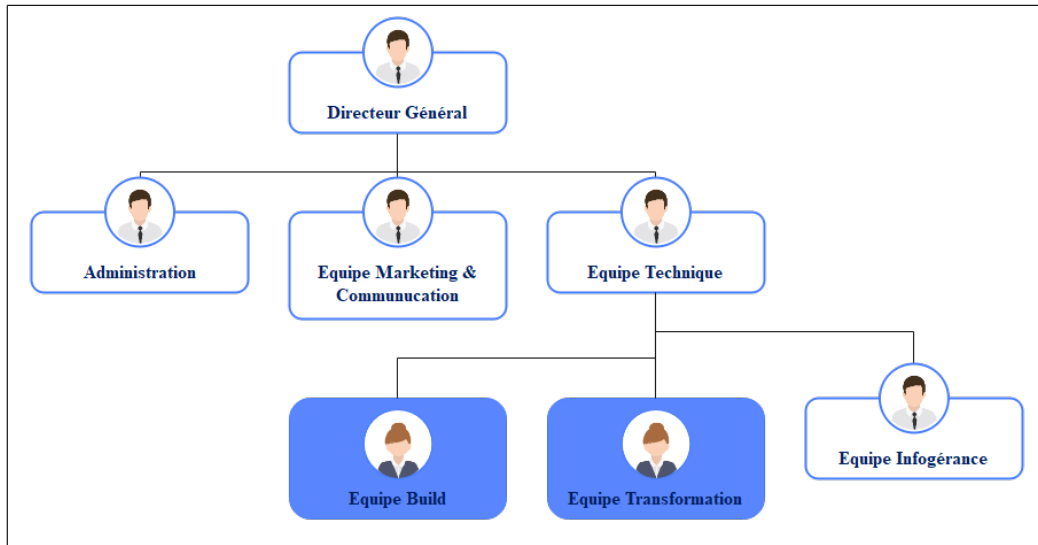


FIGURE 1.2 : Organigramme de Cloud Temple Tunisia

Le projet a été réalisé en collaboration avec l'équipe technique, où chaque sous équipe est responsable d'un certain nombre de tâches :

[font=,leftmargin=2.5cm]

- L'équipe transformation : cette équipe est responsable de l'étude du projet, y compris la taille et la planification, tout en travaillant avec le client.
- L'équipe build : cette équipe est impliquée dans la phase de préparation de l'infrastructure.
- L'équipe Infogérance : elle est responsable de la maintenance et de la gestion de l'infrastructure technique de l'entreprise.

2 Présentation du projet

Dans le cadre de notre ambitieux projet d'automatisation de la sécurité de l'infrastructure, nous avons l'intention de réviser complètement notre approche en intégrant des pratiques avancées de sécurité informatique. En utilisant des outils de pointe, nous automatisons non seulement le déploiement et la configuration automatique, mais également la mise en place de mesures de sécurité robustes. Grâce à cette approche holistique nous pouvons renforcer notre posture de sécurité, améliorer notre efficacité opérationnelle et relever avec agilité les défis du Cloud Computing.

2.1 Etude de l'existant : Analyse de l'infrastructure existante

CTT propose une solution d'IaaS à travers son offre de VDC. En combinant les atouts du Cloud privé et public, cette offre permet aux clients de se focaliser sur leurs activités principales tout en bénéficiant d'une plus grande souplesse, disponibilité et compétitivité pour répondre aux exigences du marché.

2.2 Critique de l'existant

Lors de notre analyse approfondie, plusieurs défis cruciaux ont été mis en lumière, exigeant une approche intégrée pour automatiser la sécurité de notre infrastructure. Voici les points clés que nous avons identifiés :

[label=●,font=,leftmargin=1.5cm]

- Les délais prolongés pour le déploiement manuel sur vSphere constituent à la fois une source d'inefficacité opérationnelle et une vulnérabilité potentielle, exposant nos systèmes à des risques pendant des périodes critiques.
- La configuration manuelle de l'infrastructure présente des incohérences et des vulnérabilités de sécurité, notamment en raison des possibilités d'omission ou d'erreur humaine.
- La gestion manuelle expose notre organisation à des risques accrus, pouvant entraîner des configurations incorrectes ou des lacunes en matière de sécurité.

2.3 Problématique

Comment automatiser la mise en place et le provisionnement pour une infrastructure cloud tout en garantissant la gestion des règles de sécurité, afin d'obtenir une posture de sécurité robuste et conforme aux exigences réglementaires ?

2.4 Solution proposées

Face à la complexité croissante de la sécurité des infrastructures cloud, nous proposons une solution complète pour automatiser la mise en place et le provisionnement d'une infrastructure cloud tout en garantissant une gestion rigoureuse des règles de sécurité.

Nous adopterons une approche qui nous permet de définir et provisionner l'infrastructure de manière déclarative, facilitant la création d'environnements sécurisés dès le départ. La configuration des serveurs sera effectuée de manière cohérente et répétable, alignée sur des politiques de sécurité

prédéfinies. Nous vérifierons en continu la conformité de nos configurations par rapport aux standards de sécurité et aux exigences réglementaires, en détectant et corrigeant rapidement les vulnérabilités potentielles. La gestion des secrets et des données sensibles sera sécurisée, garantissant un accès contrôlé. Pour la surveillance de la sécurité, nous mettrons en place des mécanismes permettant de détecter et de réagir rapidement aux activités suspectes. Enfin, nous instaurerons des processus d'amélioration continue et d'automatisation des correctifs de sécurité. Cette stratégie nous permettra de garantir une infrastructure cloud robuste et conforme, tout en minimisant les efforts manuels et en maximisant l'efficacité opérationnelle.

3 Approche méthodologique

L'adaptation d'une méthodologie appropriée est cruciale pour gérer efficacement un projet. Elle permet de mettre en place une approche méthodique et structurée pour planifier, exécuter, contrôler et clôturer le projet dans les délais impartis et en accord avec les attentes du client. Le recours à une méthodologie spécifique garantit la qualité du produit final, diminue les risques et les coûts associés au projet, tout en assurant la satisfaction du client. Une méthodologie bien établie permet aussi de définir de manière précise les procédures, les fonctions et les obligations de chaque membre de l'équipe, ainsi que les livrables attendus à chaque étape du projet. De plus, elle établit des normes et des standards pour assurer la cohérence des résultats obtenus. Dans notre cas, nous avons opté pour la méthodologie Agile Scrum.

3.1 Généralités

Le cycle de vie d'un projet désigne toutes les étapes du développement, partant de sa conception tout en assurant sa pérennisation.

D'une façon générale, le cycle de vie d'un projet est composé des étapes suivantes :

[label=●,font=,leftmargin=2cm]

- L'analyse de l'existant et la spécification des besoins
- La conception du système
- La réalisation
- Les tests : test des fonctionnalités
- Exploitation
- Maintenance

Pour assurer le bon déroulement de chaque projet, il faut avoir une démarche structurante pour garantir sa bonne gestion. Plusieurs méthodes connues pour la gestion d'un projet :

[label=●,font=,leftmargin=2cm]

- Découpage en phases (cycle en V)
- Découpage en activités WBS
- **Nouveau** : les méthodes Agiles

Chacune des deux premières méthodes classiques susmentionnées présente des risques de mauvaise gestion du projet à noter :

[label=●,font=,leftmargin=2cm]

- Mauvaise interprétation des souhaits du client
- Changement des besoins fonctionnels
- Dépassement des délais
- Bugs
- Abandon du projet

Une petite recherche a montré qu'au final, seuls un quart des projets sont considérés comme réussis avec les anciennes méthodes.

3.2 Méthode agile

La méthode Agile se distingue par son approche itérative, progressive et centrée sur la collaboration. Elle intègre activement le client dans le processus afin de répondre efficacement à ses besoins. Cette approche repose sur un cycle de développement itératif, comprenant des phases de développement, de tests unitaires, d'intégration et de correction du code, le tout étant alimenté par les retours continus des clients. Cette interaction permanente garantit la création d'un produit de haute qualité, car elle permet d'ajuster constamment les fonctionnalités en fonction des besoins réels. Ainsi, la méthode Agile se démarque grâce à sa capacité à garantir une qualité de production, une gestion dynamique du projet et une implication directe du client dans le processus de développement.

3.3 Déroutement du projet

Afin d'avoir une vue globale sur le déroulement de notre projet, une planification détaillée des différentes tâches est nécessaire. Le diagramme de Gantt de la figure 1.3 illustre la répartition et l'évolution du projet tout au long de la période du stage

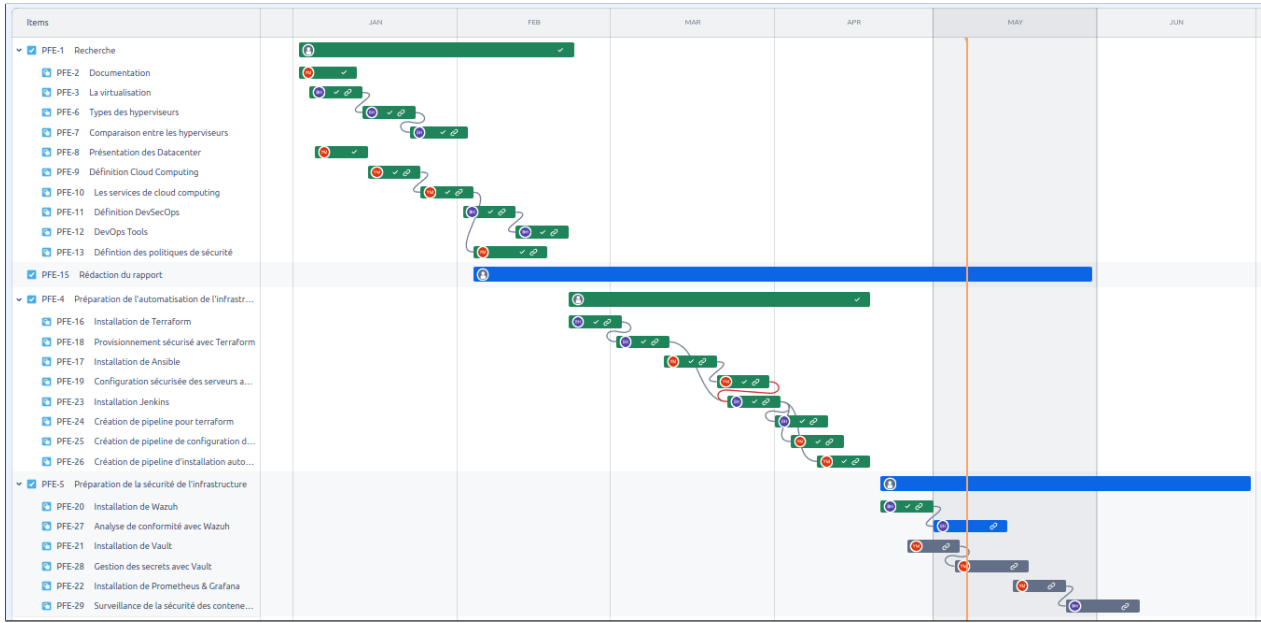


FIGURE 1.3 : Diagramme de GANTT

Conclusion

Dans ce premier chapitre, nous avons présenté le contexte général du projet, puis nous avons étudié en profondeur les solutions actuelles et critiques. Cela nous a permis de comprendre les besoins et d'examiner la solution qui convient le mieux aux attentes de l'entreprise.

Le chapitre suivant porte sur la présentation des différentes technologies avec laquelle on va réaliser notre solution et une étude comparative entre les différentes technologies qui peuvent résoudre la même problématique.

ÉTAT DE L'ART

Plan

Introduction	12
1 La Virtualisation	12
2 Centre de données	17
3 Le Cloud Computing	18
4 DevOps	23
5 DevSecOps	26
6 Choix Technologiques	27
7 Analyse et définition des besoins	35
Conclusion	37

Introduction

Dans ce chapitre, nous allons explorer les concepts fondamentaux nécessaires à la compréhension de notre projet. Nous aborderons divers sujets tels que la virtualisation, le Cloud computing, les datacenters, DevOps et la sécurité, afin de vous donner une base solide pour la suite de notre travail. Enfin, nous allons expliquer les besoins fonctionnels et non fonctionnelles.

1 La Virtualisation

La virtualisation est un concept de base lorsque nous parlons du Cloud Computing. C'est un mécanisme informatique qui consiste à créer une version virtuelle à partir d'un ou plusieurs éléments physiques, tel qu'un système d'exploitation, un serveur, un dispositif de stockage ou des ressources réseau [1].

La virtualisation est applicable dans plusieurs domaines de systèmes d'information. Nous pouvons distinguer trois principaux domaines que sont la virtualisation des serveurs, du stockage et des réseaux.

1.1 La virtualisation des serveurs

La virtualisation des serveurs est un principe permettant de faire fonctionner et d'exécuter plusieurs serveurs virtuels sur un seul serveur physique comme s'ils fonctionnent sur des serveurs physiques distincts. Nous pouvons citer quelques avantages de ce type de virtualisation, en effet la mutualisation des capacités de chaque serveur nous permet de réduire les investissements en infrastructure physiques et l'utilisation optimale des ressources des machines virtuelles sur les machines physiques nous permet de réduire les coûts [2]

1.2 La virtualisation des réseaux

La virtualisation réseau regroupe l'ensemble des outils employés pour créer une représentation artificielle de l'environnement réseau d'une entreprise. Elle consiste à recréer virtuellement un réseau physique ainsi que ses éléments tels que les ports, les commutateurs, les pare-feux, les équilibres de charge, les routeurs, etc. Cette virtualisation permet d'exécuter des applications sur un réseau virtuel de la même manière que sur un réseau physique. Cette approche permet de réduire la nécessité de matériel tout en améliorant la flexibilité et la portabilité des charges de travail.

1.3 La virtualisation du stockage

La virtualisation du stockage est un concept qui fournit une séparation de la présentation logique par rapport à la réalité physique des données de stockage [3]. C'est le fait d'isoler les données de leur localisation physique. La couche de virtualisation présente un espace de stockage logique et s'occupe de faire le lien avec la localisation physique des données. L'hyperviseur est le responsable de la liaison vers ces espaces de stockage appelés volumes physiques. Ainsi, les requêtes d'entrées/sorties sont redirigées en fonction de celle-ci et donc les serveurs n'ont plus besoin de savoir où sont leurs données. L'accès aux volumes physiques peut se faire selon différentes méthodes, présenté dans la figure 2.1 suivante :

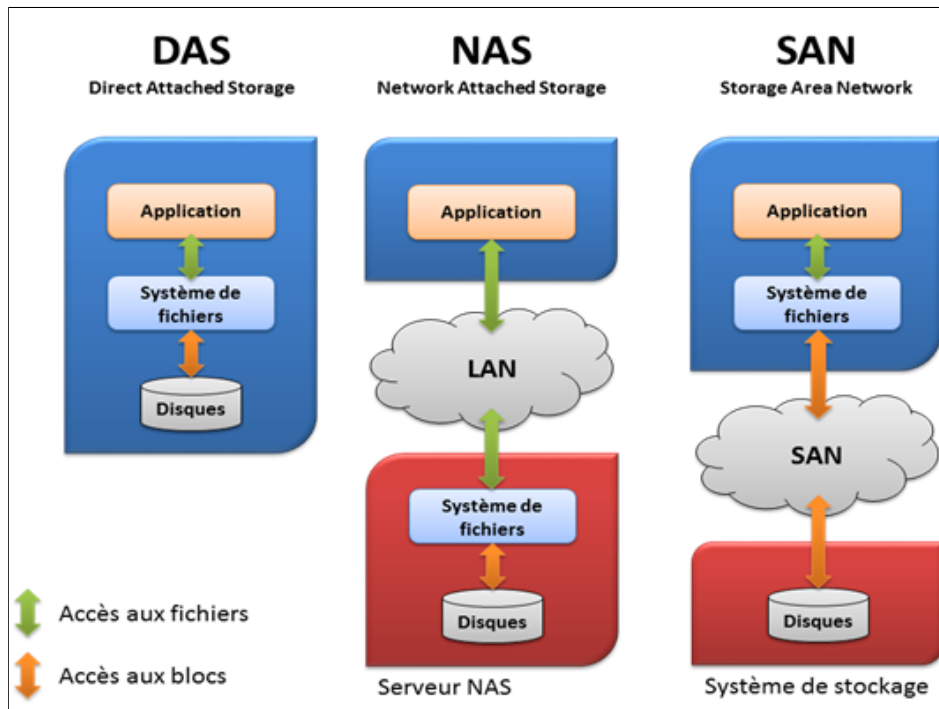


FIGURE 2.1 : Les différentes méthodes d'accès du stockage [4]

1.3.1 DAS

Comme son nom indique DAS est un accès direct sur les baies de disque qui se trouve physiquement dans le serveur lui-même. Les informations échangées entre le serveur et les disques sont transmises en bloc.. Il n'est pas possible de relier à un disque plusieurs machines simultanément et donc l'espace libre sur les disques de chaque serveur ne peuvent pas être redistribuée entre les machines, ce qui fait que l'allocation de l'espace disque est très peu optimisé.

1.3.2 NAS

Network Attached Storage est la connexion d'une baie de stockage directement sur le réseau. Il s'agit d'une baie de stockage qui dispose à la fois de son propre système d'exploitation dédiée à la gestion des données, de son propre système de fichier ainsi qu'un ensemble de disques indépendantes. Tous les périphériques connectés au même réseau peuvent donc l'accéder sans passer par un hôte intermédiaire et elle fonctionne comme un disque dur partagé d'un ordinateur régulier.

1.3.3 SAN

Le SAN se présente sous la forme d'armoire contenant plusieurs baies de disques, il est relié au système d'information par un réseau dédié généralement en fibre optique. Pour un SAN en fibre optique, les matériels sont reliés entre eux via un switch FC qui s'appelle FABRIC. Les réseaux SAN sont d'ailleurs configurés en double FABRIC afin d'obtenir une haute disponibilité par la redondance des chemins d'accès.

Le système de fichier est géré par les serveurs et les données transmises à travers SAN sont accessibles en mode bloc par le système de fichier de serveurs. Chaque serveur voit l'espace de disque d'une baie SAN auquel il a l'accès comme son propre disque. Le SAN permet de mutualiser les ressources de stockage. Il est ainsi possible de relier un disque à plusieurs machines simultanément, l'espace disque n'est pas limité par les caractéristiques du serveur et devient évolutif par l'ajout de disque sur le SAN. L'ensemble des éléments du réseau possède un WWN qui est l'équivalent de l'adresse MAC sur un réseau IP .

Principalement, nous avons comme élément :

[leftmargin=1.5cm]

- **Des commutateurs :** Nexus
- **Des câbles :** Les câbles sont de type fibre optique.
- **Des baies de stockage :** Élément principal du SAN, la baie de stockage contient l'ensemble des disques. Par défaut, toutes les baies sont équipées d'un système de RAID au minimum. Les plus grosses baies, peuvent proposer des solutions de virtualisation, de tiering du stockage et autres qui permettent d'avoir de plus grandes performances et disponibilité.

- **Des HBA** : Comparable à une carte réseau, elle permet à un serveur de se connecter au SAN. L'ensemble des chemins et disques que nous voulons connecter au serveur sont directement reconnus et utilisables. C'est pourquoi nous avons besoin de faire un zonage entre la baie et le serveur pour ne pas avoir accès à tous les disques disponibles. Afin d'avoir une haute disponibilité du SAN, chaque élément est redondé ainsi que les HBA. Il y en a au moins deux par serveur pour pouvoir accéder à deux fabrications.

1.4 Types d'hyperviseurs

L'un des principaux composants de la virtualisation est l'hyperviseur. Il permet à plusieurs systèmes d'exploitation de s'exécuter en même temps sur une même machine physique, il permet aussi d'allouer alternativement à chaque système d'exploitation les ressources nécessaires en termes de RAM, CPU et Stockage [5]. Nous présentons deux types d'hyperviseurs :

1.4.1 Hyperviseur de type 1

Le premier type d'hyperviseur est un système qui s'installe directement sur la couche matérielle du serveur. Il est en charge du matériel ainsi que la gestion des systèmes d'exploitation invités qui sont installés dans les machines virtuelles. Ceci permet de libérer le plus de ressources possible pour ces machines. Toutefois, il est possible d'exécuter uniquement un hyperviseur à la fois sur un serveur. Parmi ces hyperviseurs, nous pouvons citer comme deux exemples VMware ESXi et Microsoft Hyper-V [6] .

1.4.2 Hyperviseur de type 2

Un hyperviseur de type 2 est un logiciel qui s'installe et s'exécute à l'intérieur d'un autre système d'exploitation déjà présent sur une machine physique. Plus de ressources sont alors utilisées puisque nous faisons tourner l'hyperviseur et le système d'exploitation qui le supporte, ce qui engendre une réduction des performances.

Ce type d'hyperviseur présente un gros avantage grâce à la possibilité d'installer et d'exécuter plusieurs hyperviseurs sur un seul système hôte. Parmi ces hyperviseurs, nous trouvons VMware Workstation, Oracle VM VirtualBox et Microsoft Virtual PC [6].

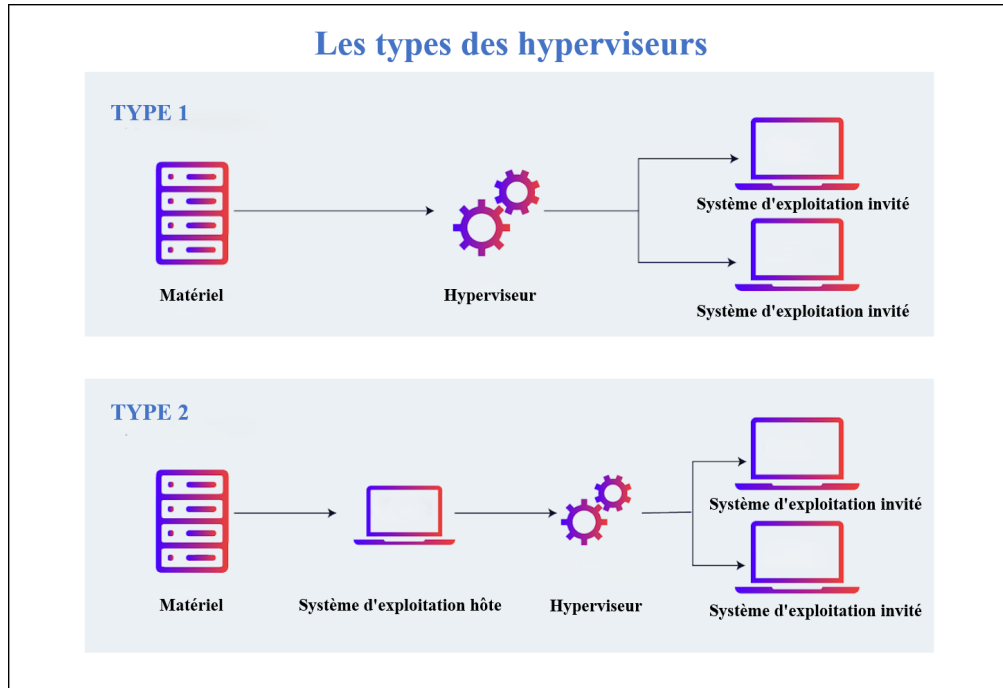


FIGURE 2.2 : Comparaison entre les types d'hyperviseurs

La figure 2.2 ci-dessus résume la différence entre les deux types d'hyperviseurs.

1.4.3 Comparaison entre les solutions d'Hypervision

L'hyperviseur de type 1 est la méthode de virtualisation d'infrastructure la plus adaptée dans le cas de la virtualisation d'un centre de traitement de données, puisque ce type d'hyperviseur est directement lié à la couche matérielle nous pouvons allouer un maximum de ressources aux machines virtuelles [7]. Alors nous avons fait une étude comparative entre les solutions d'hypervision les plus fréquentes dans le marché comme le montre le tableau 2.1 afin de choisir le bon hyperviseur .

Solution d'hypervision	VMware ESXi	Microsoft Hyper-V	Citrix XEN	KVM
Open source	Non	Non	Oui	Oui
Mémoire RAM maximale par hôte	4 To	2 To	1 To	1 To
Mémoire RAM maximale par VM	1 To	1 To	64 Go	64 Go
CPU virtuels maximum par hôte	4 096	2 048	512	512
CPU virtuels maximum par VM	64	64	4	4

TABLEAU 2.1 : Tableau comparatif entre les solutions d'hypervision

Après l'évaluation comparative effectuée en tenant compte d'un ensemble de critères de performances, nous remarquons que :

[leftmargin=1.5cm]

- Les hyperviseurs ESXi et Hyper-V sont les plus performants. Ils présentent des critères égaux tels que la taille maximale de la mémoire et le nombre maximum des CPUs virtuels par VM.
- L'hyperviseur ESXi nous offre plus de légèreté et de flexibilité au niveau des ressources utilisées par un hôte.

Donc nous avons choisi pour notre projet de travailler avec l'hyperviseur ESXi, l'hyperviseur le plus fiable en production et disposant des principales fonctions nécessaires pour offrir les niveaux de performance, de fiabilité et d'évolutivité dont les clients ont besoin.

2 Centre de données

Un centre de données, également appelé "Datacenter", est un centre de traitement de données dans lequel sont logés divers des équipements informatiques, tels que des serveurs, des dispositifs de stockage, des composants réseau, et d'autres éléments essentiels pour le bon fonctionnement des infrastructures informatiques.

Il offre un environnement contrôlé, un haut niveau de sécurité physique en déployant un grand nombre de mesures antivol et anti-incendie, une alimentation électrique et une climatisation garanties par un contrat SLA et un support technique 24/7 [8]. Leur conception vise à garantir la disponibilité, la fiabilité et les performances des systèmes et services qui y sont hébergés.

Les datacenters jouent un rôle crucial dans la gestion et la conservation des données, ainsi que dans l'exécution des applications et services informatiques. Ils sont utilisés par de nombreuses organisations, qu'il s'agisse d'entreprises, d'institutions gouvernementales ou d'autres entités, pour abriter leurs infrastructures informatiques et assurer la continuité de leurs opérations. Les datacenters peuvent être exploités en interne par une organisation ou loués auprès de prestataires de services spécialisés.

2.1 Classification des Datacenters

La classification des Datacenters permet une vision claire et précise des niveaux de sécurité et de fiabilité que peuvent offrir ces derniers. L'organisation "Uptime Institute" classe les Datacenters en quatre niveaux qui correspondent à un certain nombre de garanties sur le type de matériel déployé dans le Datacenter afin d'assurer sa redondance [9].

[leftmargin=1.5cm]

- Un Datacenter Tier I est un site basique sans redondance, avec une seule alimentation électrique et une climatisation non redondante. Il offre un taux de disponibilité annoncé de 99.67%. Cette valeur correspond à 28.8 heures d'interruption par an.
- Un Datacenter Tier II est caractérisé par une redondance partielle, en effet les éléments de production de froid ou/et d'électricité sont redondants, mais leur distribution n'est pas redondante. Il offre un taux de disponibilité de 99.75% qui correspond à 22 heures de coupure par an.
- Un Datacenter Tier III assure une disponibilité de 99,98% qui correspond à 1.6 heures d'interruption par an. Il est caractérisé par une redondance passive des voies d'alimentation et de refroidissement.
- Un Datacenter Tier IV représente le plus haut niveau de garantie avec un taux de disponibilité de 99,99% qui correspond à 0.8 heures d'interruption par an. La redondance active des circuits électriques, des dispositifs de refroidissement et du matériel réseau est assurée par ce dispositif, ce qui assure une disponibilité totale des services hébergés.

Notre plateforme principale est hébergée dans DATAXION, le premier Datacenter Tier III+ en Tunisie à cause de l'existence d'une seule source d'électricité

3 Le Cloud Computing

Le Cloud Computing, selon le NIST, il s'agit d'un modèle qui offre un accès réseau à la demande à un ensemble de ressources informatiques configurables, comme les réseaux, les serveurs, le stockage, les applications et les services qui peuvent être provisionnées rapidement et distribuées avec un minimum de gestion ou d'interaction avec le fournisseur de services [10]. Physiquement situés dans des datacenters hautement sécurisés, ces nuages hébergent des centaines de serveurs.

Le Cloud Computing permet de dématérialiser l'informatique en stockant les données sur une

multitude de serveurs distants, interconnectés et accessibles à tout moment et en tout lieu via des réseaux, notamment sans fil. Les utilisateurs n'ont plus besoin de posséder leurs propres serveurs mais peuvent accéder à de nombreux services de manière évolutive, sans avoir à gérer l'infrastructure sous-jacente, souvent complexe, avec un minimum de gestion ou d'interaction avec le fournisseur de services.

3.1 Les caractéristiques du Cloud Computing

Les principales caractéristiques du Cloud Computing sont :

[label=•,leftmargin=2cm]

- **Accès aux services par l'utilisateur à la demande :** La mise en œuvre des systèmes est entièrement automatisée et l'utilisateur peut mettre en place et gérer la configuration à distance via une console de commande.
- **Accès Internet large bande utilisé :** Les centres de données et les clients sont généralement reliés par le backbone Internet afin de bénéficier d'une excellente connectivité.
- **Accès à un pool de ressources :** L'utilisateur peut gérer les ressources (RAM, CPU et Stockage) sous forme des pools et sans la nécessité de connaître leurs emplacements et une fois les ressources libérées, d'autres utilisateurs peuvent les utiliser.
- **Élasticité :** La quantité des ressources est ajustée dynamiquement selon les besoins des utilisateurs.
- **La disponibilité :** Le Cloud Computing permet d'assurer le bon fonctionnement des services sans interruption.
- **La facturation à l'utilisation :** La facturation est calculée en fonction de la durée et de la quantité de ressources utilisées [11].

3.2 Les modèles de déploiement du Cloud Computing

Nous distinguons quatre modèles de déploiement dans le Cloud Computing comme présente la figure 2.3 : Le Cloud public, également le premier apparu, le Cloud privé, le Cloud hybride qui est la combinaison des deux premiers et le Cloud communautaire.

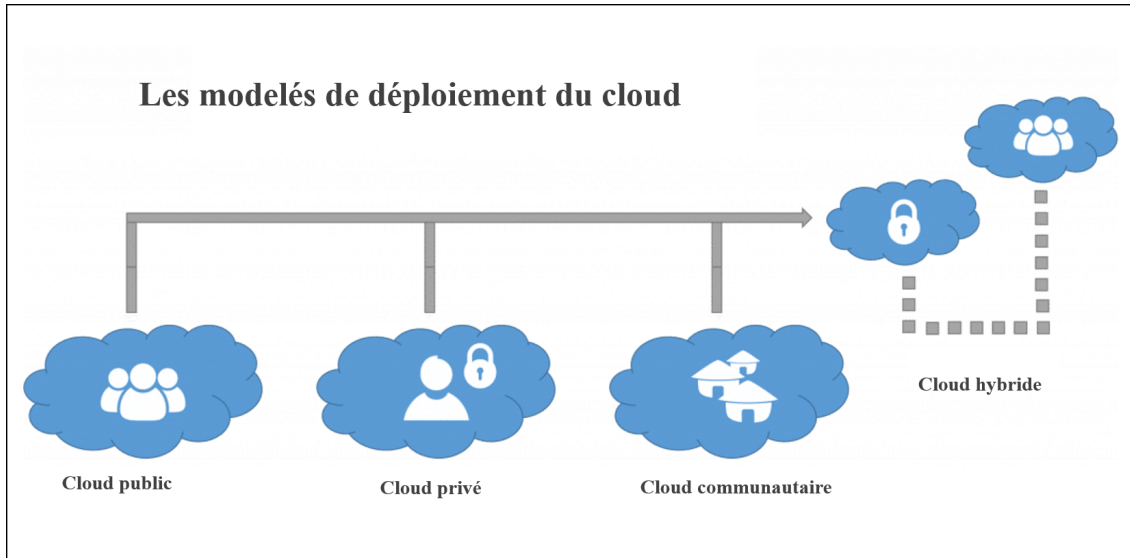


FIGURE 2.3 : Modèles de déploiement du CCloud

3.2.1 Le Cloud public

Les utilisateurs peuvent utiliser des services cloud via l'Internet public sans être conscients de l'emplacement exact de leurs données ni de l'exécution de leurs traitements. Les ressources informatiques et les bases de données de l'utilisateur peuvent être hébergées dans n'importe quel Data center du prestataire et peuvent passer d'un Data center à l'autre pour optimiser les capacités du prestataire. Ces services présentent un coût raisonnable et parfois nous trouvons des services gratuits. Exemples de Cloud publics : Google AppEngine, Ibm4S Blue Cloud, Amazon Elastic Compute Cloud.

3.2.2 Le Cloud privé

L'ensemble des ressources est exclusivement mis à disposition d'un seul client. C'est la typologie de Cloud la plus répandue, car il peut être déployé sous deux formes distinctes :

[label=●,leftmargin=1.5cm]

- Cloud privé interne : Les données sont hébergées par l'entreprise elle-même, parfois partagées ou mutualisées en mode privatif avec les filiales.
- Cloud privé externe : Les données sont hébergées chez un tiers, il est entièrement dédié à l'entreprise et accessible via des réseaux sécurisés de type VPN.

L'avantage de ce type de déploiement réside dans l'aspect de la sécurité et la protection des données [12]

3.2.3 Le cCloud hybride

C'est la mise en commun des ressources du Cloud privé et public, qui sont interconnectés pour travailler ensemble au sein d'une même entreprise. En effet, le Cloud privé offre un certain niveau d'efficacité alors que les services du cloud public sont susceptibles d'être plus avantageux en termes de coûts et de flexibilité par rapport aux Clouds privés. C'est la raison pour laquelle les entreprises optent pour cette solution en s'appuyant sur les services de Cloud publics pour les applications et des données non critiques, tout en laissant les applications et les données critiques sur un Cloud privé [13].

3.2.4 Le Cloud communautaire

Comme son nom l'indique, le cloud communautaire est utilisé par plusieurs entreprises qui ont des besoins communs. Il peut héberger une application métier très spécialisée, mais commune à plusieurs entités, qui décident de fédérer leurs efforts en construisant un Cloud pour l'héberger et la gérer [14].

3.3 Les modèles du service Cloud Computing

Le Cloud Computing offre trois modèles de service principaux, ils s'organisent du niveau le moins visible pour les utilisateurs finaux au plus visible comme présentés par la figure 2.4. L'infrastructure en tant que service (IaaS) est gérée par les architectes réseaux, la plate-forme en tant que service (PaaS) est destinée aux développeurs d'applications et finalement le logiciel en tant que service (SaaS) est le produit final pour les utilisateurs.

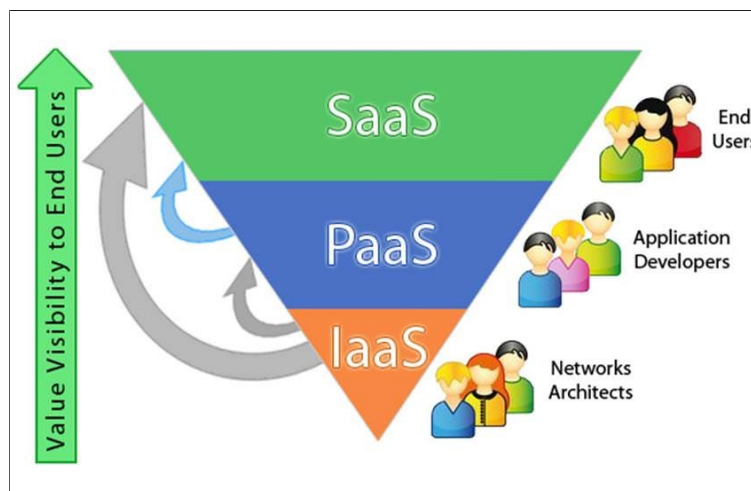


FIGURE 2.4 : Les niveaux de services du Cloud Computing [15]

3.3.1 Infrastructure as a Service (IaaS)

Comme son nom indique le client se bénéficie d'une infrastructure en tant que service, offrant à l'utilisateur l'accès à une infrastructure matérielle sur demande. Celui-ci peut créer son architecture et déployer ses propres applications (système d'exploitation, base de données, etc.). Donc l'utilisateur contrôle les systèmes d'exploitations, les applications et non pas l'infrastructure. Ce service libère l'entreprise de la nécessité de posséder (gérer, maintenir et contrôler) ses propres serveurs et autres infrastructures de traitement de données [16]. Parmi les prestataires d'IaaS, nous pouvons citer : Cloud Temple, Amazon Web Services, OVH Cloud.

3.3.2 Platform as a Service (PaaS)

Dans ce modèle de service les fournisseurs de services cloud mettent à la disposition des utilisateurs une plateforme de développement complète, comprenant des outils de développement, des langages de programmation, des bibliothèques et des services de déploiement. Cela permet aux utilisateurs de se concentrer sur le développement d'applications sans avoir à se préoccuper de l'infrastructure sous-jacente. Les fournisseurs se chargent de la gestion de l'infrastructure, du système d'exploitation et des mises à jour de la plateforme [17]. Les principaux acteurs qui proposent ce modèle d'infrastructure sont : Google App Engine et Microsoft Azure .

3.3.3 Software as a Service (SaaS)

SaaS est un service qui permet aux entreprises de s'abonner à un logiciel à distance au lieu de les acquérir et de devoir les installer sur leur propre matériel informatique. Ce service offre au client la possibilité de payer suite à une consommation d'une application sur la base du temps d'utilisation et non plus par l'achat de licences. L'utilisateur n'a plus besoin d'installer et de maintenir l'application sur sa machine mais juste d'un terminal pour y accéder à travers le Cloud [18]. Des exemples courants de SaaS incluent : Gmail, Facebook, Dropbox.

3.4 La sécurité dans le Cloud Computing

Le nombre et la diversité des utilisateurs des infrastructures Cloud ainsi que la quantité importante de composants matériels et logiciels impliquent la présence de menaces de sécurité.

3.4.1 La confidentialité

La confidentialité garantit que les données du client ne seront accessibles que par les entités autorisées. L'infrastructure Cloud comporte des mécanismes de gestion des identités et des accès, l'isolation des machines virtuelles avec l'hyperviseur qui permet de concevoir des architectures isolées entre les machines virtuelles .

3.4.2 L'intégrité

Les systèmes dans le Cloud fournissent un certain nombre de mécanismes de protection de l'intégrité des données afin de les protéger contre les modifications non autorisées. En effet, l'infrastructure Cloud comporte des mécanismes de contrôle d'accès par l'affectation des droits spécifiques à chaque utilisateur, la mise à jour et l'installation des patchs de sécurité .

3.4.3 La disponibilité

Le Cloud Computing assure la disponibilité des données grâce à la redondance et au basculement automatique réalisés avec les technologies de virtualisation.

4 DevOps

DevOps est la combinaison de philosophies culturelles, de pratiques et d'outils qui augmentent la capacité d'une organisation à livrer des applications et des services à grande vitesse : en faisant évoluer et en améliorant les produits à un rythme plus rapide que les organisations employant des méthodes classiques de développement de logiciels et de gestion d'infrastructure Cette vitesse permet aux organisations de mieux servir leurs

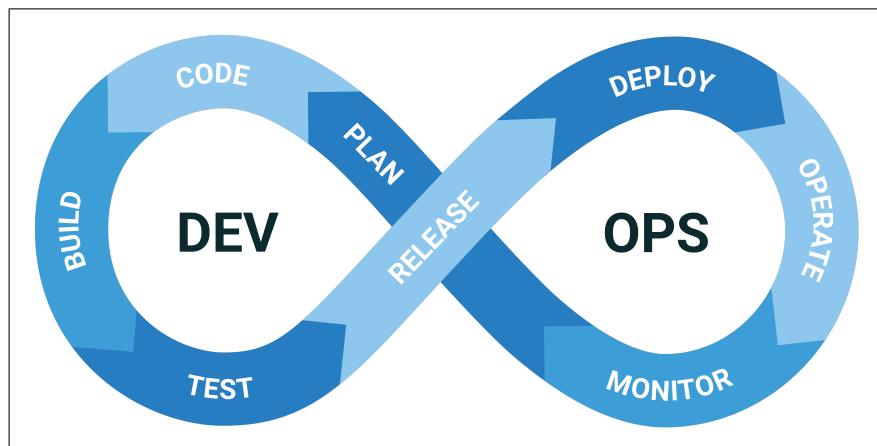


FIGURE 2.5 : Cycle de vie DevOps [19]

La méthodologie DevOps repose sur les principes suivants :

[label=●,leftmargin=2cm]

- **Collaboration** : Les équipes de développement (DEV) et d'opérations (OPS) travaillent de manière étroite et collaborative. Elles partagent les responsabilités et les objectifs communs, favorisant ainsi la communication et la coopération tout au long du cycle de vie du développement logiciel.
- **Automatisation** : L'automatisation est essentielle dans la méthodologie DevOps. Les tâches répétitives et manuelles sont automatisées autant que possible, ce qui permet de réduire les erreurs, d'améliorer l'efficacité et de gagner du temps. Cela inclut l'automatisation des tests, des déploiements, des configurations et des processus de surveillance.
- **Monitoring et rétroaction** : Les équipes DevOps accordent une grande importance à la surveillance continue des performances des applications et des infrastructures. Cela permet de détecter rapidement les problèmes, d'effectuer des ajustements et d'obtenir des informations pour améliorer les futures versions des logiciels.

L'objectif principal de la méthodologie DevOps est de créer un cycle de développement logiciel rapide, itératif et fiable, en réduisant les frictions entre les équipes et en automatisant les processus. Cela permet aux organisations de fournir des logiciels de haute qualité plus rapidement, tout en favorisant l'innovation continue et l'adaptabilité aux changements.

4.1 Infrastructures as Code (IaC)

L'infrastructure en tant que code est une configuration informatique qui permet aux développeurs et aux techniciens d'exploitation de gérer et de configurer automatiquement l'infrastructure informatique via du code sans processus manuels. Par conséquent, on parle souvent d'infrastructure «définie par logiciel» ou d'infrastructure programmable.

L'infrastructure en tant que code peut être utilisée dans tout l'environnement informatique. Mais pour le cloud computing, l'infrastructure en tant que service (IaaS), c'est particulièrement important. De même, DevOps nécessite des flux de travail agiles et des flux de travail automatisés, et ces tâches et flux de travail automatisés ne peuvent être réalisés que si l'infrastructure informatique est disponible en permanence. Il est crucial d'avoir cette infrastructure afin d'exécuter et de tester le code développé.

4.2 Approche CI/CD

4.2.1 Définition

En automatisant les étapes du développement, l'approche CI/CD permet d'accroître la fréquence de distribution des applications. L'intégration continue, la distribution continue et le déploiement continu sont les principaux concepts clés de cette approche. Elle résout les problèmes posés par l'intégration de nouveaux segments de code pour les équipes de développement et d'exploitation. Plus précisément, le CI/CD garantit une automatisation et une surveillance tout au long du cycle de vie des applications, de l'intégration et des tests jusqu'à la distribution et le déploiement [20].

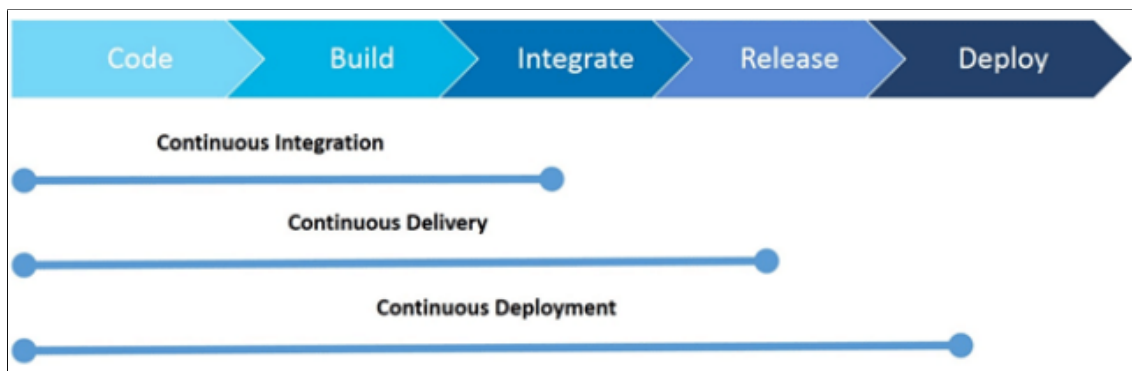


FIGURE 2.6 : L'approche CI/CD

L'acronyme « CI/CD » a plusieurs significations. Le « CI » de CI/CD désigne toujours l'« intégration continue », à savoir un processus d'automatisation pour les développeurs. Le « CD » de CI/CD désigne la « distribution continue » et/ou le « déploiement continu », qui sont des concepts très proches, parfois utilisés de façon interchangeable.

4.2.2 Intégration continue (CI)

Pour les développeurs, l'intégration continue implique de modifier régulièrement le code de leur application, de les tester, puis de les fusionner dans un référentiel commun. Grâce à cette solution, il est possible de ne pas travailler simultanément sur un grand nombre d'éléments d'une application, qui pourraient entrer en conflit les uns avec les autres.

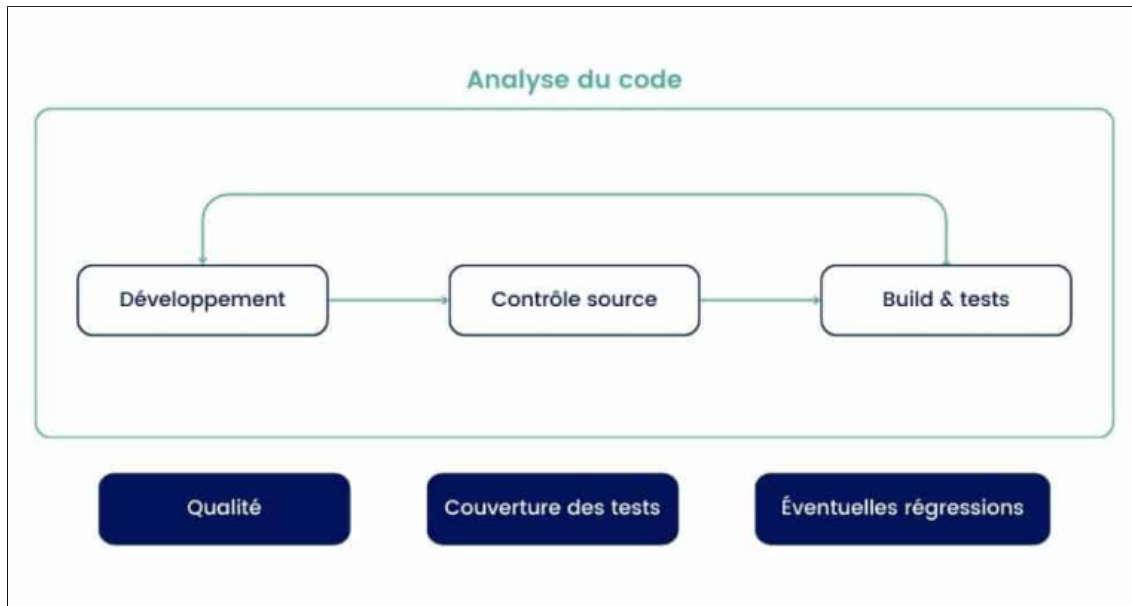


FIGURE 2.7 : L'intégration continue

4.2.3 Distribution continue et déploiement continu (CD)

La distribution continue et le déploiement continu sont deux concepts concernant l'automatisation d'étapes plus avancées du pipeline, mais ils sont parfois dissociés pour illustrer le haut degré d'automatisation.

[label=●,leftmargin=1.5cm]

- **La distribution continue** : Le processus de distribution continue permet de résoudre les problèmes de visibilité et de communication entre l'équipe de développement et l'équipe métier. Ainsi, son objectif consiste à simplifier au maximum le déploiement de nouveau code.
- **Le déploiement continu** : Ce processus permet de soulager les équipes d'exploitation surchargées par les tâches manuelles qui ralentissent la distribution des applications. Il repose sur la distribution continue et automatise l'étape suivante du pipeline.

5 DevSecOps

DevSecOps, abréviation de développement, sécurité et opérations, consiste à automatiser l'intégration de la sécurité à chaque étape du cycle de vie du développement logiciel. Cela inclut la conception initiale, l'intégration, les tests, le déploiement et la distribution des logiciels [21].

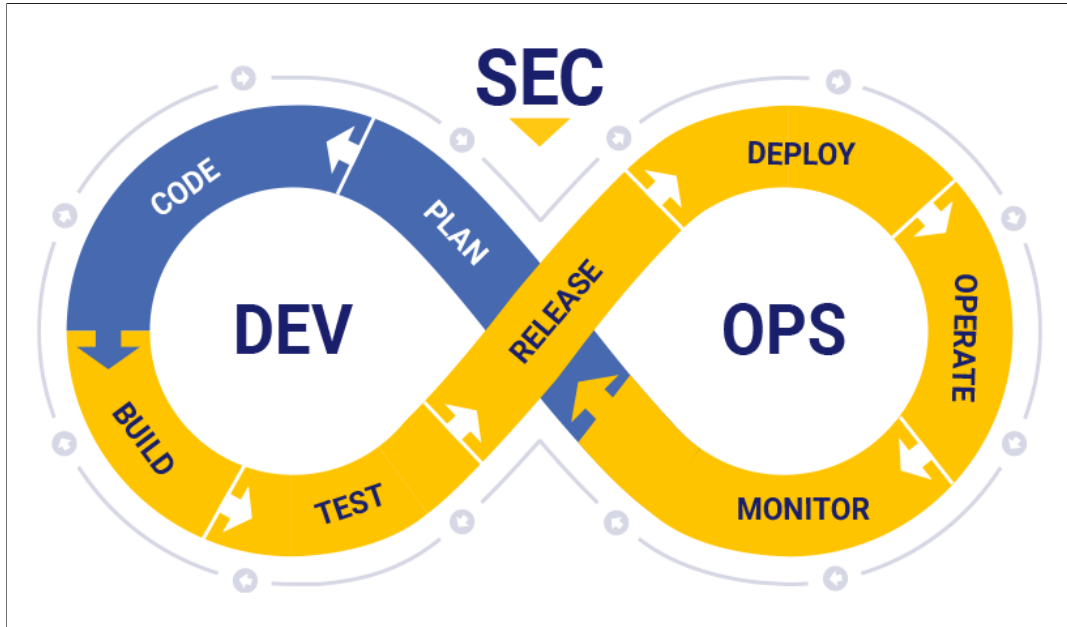


FIGURE 2.8 : L'approche DevSecOps

La sécurité des applications et des infrastructures est intégrée de manière fluide dans les processus et les outils Agile et DevOps grâce à l'approche DevSecOps. En abordant les problèmes de sécurité dès leur apparition, DevSecOps facilite leur résolution de manière plus simple, rapide et économique, avant qu'ils ne soient mis en production. De plus, DevSecOps transforme la sécurité des applications et de l'infrastructure en une responsabilité partagée par les équipes de développement, de sécurité et des opérations informatiques, plutôt que de la confiner à un silo de sécurité distinct. Cette approche permet de produire des logiciels plus sûrs, plus rapidement, en automatisant la distribution de logiciels sécurisés sans ralentir le cycle de développement.

6 Choix Technologiques

Dans cette partie on va présenter et expliquer notre choix technologique pour chaque partie du projet, car pour chaque besoin il y a plusieurs solutions, donc il est important de bien choisir les produits qu'on va adoptés selon le type du projet et notre environnement hardware et logiciel.

6.1 Choix de solutions Cloud

Critères	VMware vSphere	OpenStack
Type	Plateforme de virtualisation intégrée pour le cloud privé.	Plateforme de cloud computing open source pour le cloud privé (et public).
Coût	Coûts de licence pour VMware, plus les coûts de support et de maintenance.	Gratuit en termes de licences, mais peut entraîner des coûts liés au support, à la formation, et à la personnalisation.
Facilité d'installation et de configuration	Relativement simple avec une suite intégrée, mais nécessite une expertise VMware.	Plus complexe, nécessite une bonne compréhension des composants OpenStack et de leur intégration.
Facilité d'utilisation	Interface utilisateur graphique intuitive et outils de gestion consolidés.	Interface utilisateur et dashboard disponibles, mais avec une courbe d'apprentissage pour la gestion via les outils CLI et API
Scalabilité	Scalabilité verticale et horizontale gérée via vCenter et VMware Cloud Foundation.	Naturellement conçu pour être scalable, supporte la scalabilité horizontale à grande échelle.
Sécurité	Fortes mesures de sécurité intégrées et bonnes pratiques recommandées par VMware.	Bon niveau de sécurité, mais dépend fortement de la configuration et des politiques mises en œuvre par l'administrateur.

TABLEAU 2.2 : Tableau comparatif entre VMware vSphere et OpenStack

Le choix de VMware vSphere par notre client, au lieu d'OpenStack, peut s'expliquer par la recherche de simplicité d'utilisation, de fiabilité et de support technique complet. VMware offre une plateforme de virtualisation mature et intégrée, réputée pour sa stabilité et ses performances élevées, accompagnée d'un support professionnel.

Bien qu'OpenStack présente l'avantage d'être une solution open source, la complexité de sa mise en

place et de sa gestion, ainsi que les coûts associés à l'expertise technique nécessaire, peuvent rendre VMware plus attrayant pour les entreprises cherchant une solution clé en main et moins exigeante en termes de ressources internes.

6.2 Système de gestion de version

Critères	GitHub	GitLab CE
Tarification	Gratuit pour les dépôts publics, avec des plans payants pour les dépôts privés. Options de tarification flexibles, y compris des options gratuites pour les dépôts publics et privés.	Gratuit et open source, mais nécessite un hébergement auto-géré.
Fonctionnalités de gestion des projets	Interface utilisateur conviviale avec des fonctionnalités avancées telles que les projets GitHub, les problèmes, les pulls, les pages GitHub, etc.	Fonctionnalités similaires à GitHub avec des outils de gestion de projet, mais parfois moins intuitives. Disponibles pour GitLab CE et GitLab SaaS.
Intégration CI/CD	Intégration native avec Actions GitHub pour CI/CD, avec une vaste gamme d'intégrations tierces disponibles.	Intégration CI/CD native disponible avec GitLab CI pour GitLab CE et GitLab SaaS. Possibilité d'auto-héberger des runners CI/CD.
Communauté et Support	Grande communauté d'utilisateurs et de contributeurs, avec une documentation complète et un support de qualité.	Communauté active, mais peut être moins étendue que GitHub. Support disponible pour GitLab CE et GitLab SaaS, mais peut être limité pour la version auto-hébergée.

TABLEAU 2.3 : Tableau comparatif entre GitHub, GitLab CE

Nous avons choisi GitHub sans hésitation. C'est la plateforme que nous connaissons tous et que nous utilisons pour partager et gérer notre code. C'est simple, clair et pratique. Avec GitHub,

nous pouvons facilement voir qui a contribué à quoi, suivre les problèmes et automatiser certaines tâches fastidieuses. Et le meilleur dans tout ça, c'est que nous pouvons commencer gratuitement. En résumé, pour notre projet de fin d'études, GitHub est l'outil parfait pour collaborer avec nos camarades et rendre notre travail plus efficace.

6.3 CI/CD

Après avoir trouvé où héberger notre code, maintenant c'est aux tours de l'outil d'intégration et déploiement continu :

Critères	Jenkins	GitLab CI
Fréquence des Intégrations	Rapide et Fréquente	Intégré avec Git
Automatisation des Tests	Oui	Oui
Facilité de Configuration	Requiert une configuration détaillée	Configuration simple et intuitive
Extensibilité	Très extensible grâce à un large écosystème de plugins	Intégré nativement avec d'autres outils GitLab
Support Communautaire	Grande communauté d'utilisateurs et de contributeurs	Communauté active et documentations riches
Intégration CI/CD	Offre des fonctionnalités CI/CD complètes	Intègre nativement CI/CD dans un même outil
Personnalisation des Workflows	Personnalisable avec une grande flexibilité	Personnalisable avec des fichiers de configuration YAML

TABLEAU 2.4 : Etude comparative des outils CI/CD

Nous avons choisi Jenkins pour notre projet en raison de sa flexibilité et de son extensibilité inégalées grâce à un vaste écosystème de plugins. Cette capacité à personnaliser nos pipelines CI/CD pour répondre à des besoins spécifiques est cruciale. De plus, étant un outil open source, Jenkins est une solution économique, idéale pour notre projet.

6.4 Choix d'outil pour le provisionnement de l'infrastructure

Il est également possible de faire appel à des outils ou des infrastructures logicielles d'IaC spécifiques qui proposent leurs propres langages de configuration pour pouvoir gérer les ressources quel que soit le fournisseur.

Le tableau 2.5 montre la différence entre ces outils :

Critères	 Terraform	 CHEF	 puppet	 ANSIBLE
Facilité de prise en main	Syntaxe simple et déclarative. Configuration basée sur des fichiers.	Courbe d'apprentissage plus raide. Configuration basée sur des états.	Syntaxe flexible mais complexe. Configuration basée sur des manifestes.	Syntaxe simple et lisible. Configuration basée sur des playbooks YAML.
Scalabilité	Conçu pour le provisionnement à grande échelle. Prise en charge des infrastructures complexes.	Adapté aux environnements scalables. Capacités de gestion distribuée.	Gestion des grands environnements possible. Nécessite une architecture maître/esclave.	Convient aux petites et grandes infrastructures. Pas de serveur maître requis.
Automatisation des tâches	Automatisation robuste et fiable. Intégration transparente avec les principaux fournisseurs de cloud.	Automatisation puissante avec des recettes et des rôles. Capable de gérer diverses tâches.	Automatisation flexible mais complexe. Gestion des configurations à long terme.	Automatisation simple et efficace. Bonne gestion des configurations à court terme.

TABLEAU 2.5 : Comparaison des outils de gestion d'infrastructure

Après avoir examiné les différents outils de gestion d'infrastructure, nous avons choisi Terraform

pour le provisionnement et Ansible pour la configuration. Terraform nous offre une syntaxe simple et une capacité à gérer des infrastructures à grande échelle de manière fiable, tandis qu'Ansible nous permet de configurer nos systèmes de manière efficace avec ses playbooks YAML clairs et lisibles. Ensemble, ces deux outils offrent une solution complète pour la gestion de notre infrastructure, répondant à nos besoins en termes de facilité d'utilisation, de scalabilité et d'automatisation des tâches

6.5 Choix d'outil pour la gestion des secrets

Caractéristiques	Vault	StrongDM
Sécurité	Offre un haut niveau de sécurité avec un chiffrement avancé et des contrôles d'accès stricts.	Fournit une sécurité solide avec des audits et une surveillance, mais principalement axée sur les accès privilégiés.
Gestion des secrets	Gestion centralisée des secrets, rotation automatique, et intégration avec divers outils DevOps.	Gère les accès privilégiés et les connexions aux bases de données, mais la gestion des secrets est moins complète.
Intégrations	Large gamme d'intégrations avec des outils DevOps, des services cloud, et des systèmes de gestion des identités.	Intégré avec les outils de gestion des accès et des bases de données, mais moins d'options pour les intégrations DevOps.
Tarification	Propose des plans compétitifs et souvent plus économiques pour les entreprises de toutes tailles.	Peut être coûteux en fonction de la taille de l'infrastructure et des besoins en accès privilégiés
Flexibilité d'utilisation	Très flexible, adapté à divers cas d'utilisation, notamment la gestion des secrets pour les développeurs et les administrateurs systèmes.	Principalement axé sur la gestion des accès aux bases de données et aux serveurs, moins flexible pour la gestion des secrets divers.

TABEAU 2.6 : Tableau comparatif entre Vault et StrongDM

Nous avons opté pour Vault en raison de sa gestion exhaustive des secrets, sa flexibilité

exceptionnelle et sa capacité d'intégration étendue avec les outils DevOps. En comparaison avec StrongDM, Vault offre des contrôles d'accès granulaires et une gestion centralisée des secrets, répondant ainsi parfaitement à nos exigences de sécurité complexes. En outre, Vault présente des options de tarification plus avantageuses, adaptées aux entreprises de toutes tailles.

6.6 Choix d'outil pour l'analyse de conformité

Critères	Wazuh	OpenSCAP
Facilité de déploiement	Installation simple avec des agents pré-compilés pour différentes plateformes. Configurations centralisées via une interface web conviviale.	Configuration plus complexe nécessitant souvent une expertise Linux. Peut nécessiter l'installation et la configuration manuelles des scanners.
Fonctionnalités de sécurité	Détection d'intrusion en temps réel avec analyse de journaux et corrélation d'événements. Surveillance de la conformité avec des règles prédéfinies et des rapports détaillés.	Analyse de la conformité et des vulnérabilités selon les profils prédéfinis. Peut nécessiter des scripts personnalisés pour l'intégration avec d'autres outils de sécurité.
Support et communauté	Communauté active et documentation complète. Support professionnel disponible avec des options de formation.	Communauté plus restreinte avec moins de ressources disponibles. Support limité et souvent basé sur la documentation en ligne.

TABLEAU 2.7 : Comparaison entre Wazuh et OpenSCAP

Après avoir effectué une recherche approfondie sur les différentes solutions disponibles et à la suite de la comparaison entre ces dernières, nous avons décidé de choisir Wazuh pour les raisons suivantes :

[label=•,leftmargin=2cm]

- Wazuh offre une solution de sécurité intégrée, comprenant la surveillance des journaux, la détection d'intrusion et la conformité réglementaire.

- Wazuh est facile à installer et à configurer, fournissant une documentation approfondie pour accompagner l'utilisateur tout au long de l'installation et de la configuration.
- Wazuh dispose d'un tableau de bord intuitif qui permet de visualiser les alertes de sécurité et les événements de manière claire et concise.
- Wazuh est open-source et offre une grande flexibilité pour la personnalisation et l'adaptation aux besoins spécifiques de l'entreprise. Nous avons également considéré le coût de la solution, qui est avantageux par rapport aux autres solutions commerciales disponibles sur le marché.

6.7 Résultats des comparaisons

Après une évaluation minutieuse, nous avons sélectionné une suite d'outils et de plateformes pour optimiser notre infrastructure et nos processus de développement. VMware vSphere a été choisi comme notre solution de cloud privé pour sa fiabilité et ses performances. Github forme le cœur de notre système de gestion de versions et d'intégration continue, promouvant l'efficacité et l'automatisation du workflow. Terraform et Ansible nous aident à automatiser l'infrastructure avec un code déclaratif, tandis que Wazuh assure la vérification de la conformité des configurations et la détection des vulnérabilités potentielles. De plus, Vault est intégré pour la gestion sécurisée des secrets et des clés, garantissant que les informations sensibles sont protégées de manière optimale. Ensemble, ces choix représentent une stratégie cohérente pour une infrastructure moderne et agile.

Critères	Technologie
Solution de Cloud Privé	VMware vSphere
Système de gestion de version	Github
Outil CI/CD	Jenkins
Outils d'automatisation de l'infrastructure	Terraform et Ansible
Outil d'analyse de conformité	Wazuh
Outil de gestion des secrets	Vault

TABLEAU 2.8 : Tableau de synthèse des choix technologiques

7 Analyse et définition des besoins

7.1 Besoins Fonctionnels

Les besoins fonctionnels définissent les fonctionnalités à implémenter afin que notre nouveau système fonctionne comme prévu. Les fonctionnalités attendues de notre solution sont présentées ci-dessous :

[label=●,leftmargin=1.5cm]

— **Provisionnement sécurisé :**

L'utilisation d'un outil pour provisionner l'infrastructure en appliquant des paramètres de sécurité prédéfinis est essentielle. Cet outil doit permettre la création d'environnements sécurisés dès le départ, assurant ainsi une base solide pour l'ensemble de l'infrastructure.

— **Configuration sécurisée des serveurs :**

Il est nécessaire d'utiliser un outil pour appliquer des configurations sécurisées sur les serveurs provisionnés. Les paramètres de sécurité doivent être conformes aux normes établies, garantissant que les serveurs sont configurés de manière optimale dès leur mise en service.

— **Analyse de conformité :**

L'intégration d'un outil dédié pour vérifier la conformité des configurations est cruciale. Cet outil doit détecter les vulnérabilités potentielles et générer des rapports détaillés sur la sécurité, permettant ainsi de maintenir un haut niveau de conformité et de sécurité au sein de l'infrastructure.

— **Gestion des secrets :**

Pour garantir la protection des informations critiques, il est impératif d'utiliser un outil pour stocker et gérer de manière sécurisée les secrets et les données sensibles nécessaires à l'infrastructure. Cela assure une gestion efficace et sécurisée des informations sensibles.

— **Automatisation et amélioration continue :**

L'automatisation du processus de mise à jour des configurations de sécurité et des correctifs est essentielle pour garantir que l'infrastructure reste sécurisée et conforme aux politiques définies. De plus, la mise en place de mécanismes d'alerte pour notifier les violations de sécurité ou les écarts par rapport aux politiques définies permettra de réagir rapidement et efficacement.

— **La poursuite des activités et le plan de reprise après un sinistre :**

Un plan de continuité des opérations et un plan de reprise après sinistre doivent être développés et testés régulièrement pour garantir la résilience de l'infrastructure en cas de cyberattaque ou de défaillance majeure, en assurant que les sauvegardes sont effectuées régulièrement et stockées de manière sécurisée, avec des tests de restauration fréquents.

7.2 Besoins Non-Fonctionnels

Certains besoins, bien que non critiques pour la fonctionnalité de l'application, revêtent une importance significative pour la qualité de ses services. Ces besoins, souvent désignés sous le terme de "besoins non fonctionnels", ont un impact indirect sur l'expérience utilisateur et le résultat final. Les principaux besoins non fonctionnels de notre travail se résument dans les points suivants :

[label=●,leftmargin=1.5cm]

- Performance : Il est important que notre solution puisse facilement s'adapter à l'augmentation du nombre de ressources, tout en minimisant le temps nécessaire pour configurer ces ressources de manière efficace.
- Sécurité : Nous devons nous assurer que seules les personnes autorisées ont accès aux ressources pour éviter les intrusions, et nous devons également crypter les données sensibles pour les protéger lorsqu'elles sont en transit ou au repos. Il est crucial que toutes nos mesures de sécurité respectent les réglementations en vigueur.
- Fiabilité : Notre système doit être capable de se rétablir rapidement des erreurs et des pannes pour garantir que nos services restent disponibles, répondant ainsi aux besoins opérationnels de manière continue.
- Efficacité opérationnelle : Automatiser les tâches répétitives nous permettra d'optimiser nos processus et de réduire les erreurs humaines. En facilitant l'intégration continue et le déploiement continu, nous assurerons des mises à jour rapides et fiables du système.
- Évolutivité : Le système doit être évolutif pour s'adapter à la croissance de l'infrastructure
- Temps de Réponse : Le temps de réponse pour les opérations d'automatisation doit être rapide et prévisible.
- Gestion des Ressources : Le système doit optimiser l'utilisation des ressources matérielles, en allouant efficacement la capacité nécessaire.
- Confidentialité : Étant donné que le système fonctionnera sur le réseau privé de l'entreprise,

l'aspect de la confidentialité est dûment pris en compte.

Conclusion

Tout au long de ce chapitre, nous avons introduit des notions et des concepts en relation avec le Cloud Computing. De plus, nous avons identifié les besoins fonctionnels et non fonctionnels et avons procédé au choix des technologies. Dans le chapitre suivant, nous entamerons la tâche de la conception de notre projet.

ÉTUDE CONCEPTUELLE

Plan

Introduction	39
1 Présentation de l'architecture logicielle	39
2 Description de la conception du logiciel	40
3 Présentation de l'architecture de la solution envisagée	47
Conclusion	49

Introduction

Dans ce chapitre, nous nous concentrerons sur l'étude conceptuelle de notre projet. Nous analyserons les interactions entre les différentes étapes, en utilisant des diagrammes d'interactions pour illustrer les processus clés tels que la définition des politiques de sécurité, le provisionnement sécurisé, et la configuration des serveurs. Cette analyse nous fournira une base théorique solide pour la mise en œuvre pratique des meilleures pratiques de sécurité.

1 Présentation de l'architecture logicielle

La figure 3.1 présente l'architecture logicielle, illustrant les différentes couches et interactions essentielles du notre projet.

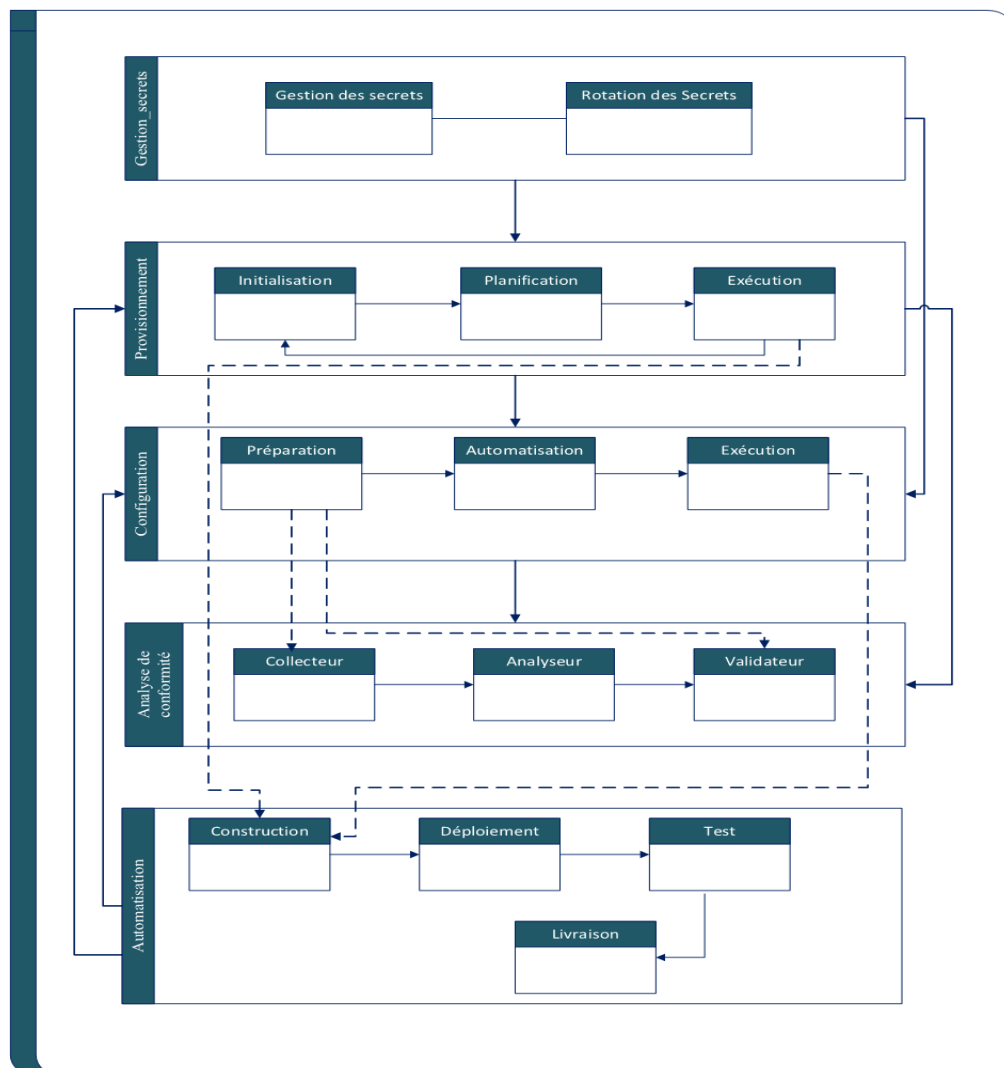


FIGURE 3.1 : Architecture logicielle

2 Description de la conception du logiciel

Couche gestion des secrets

[label=●,leftmargin=2cm]

- Initialisation
- Gestion et stockage de secrets
- Accès et utilisation des secrets

Couche provisionnement

[label=●,leftmargin=2cm]

- Initialisation
- Planification
- Exécution

Couche configuration

[label=●,leftmargin=2cm]

- Préparation
- Automatisation
- Exécution

Couche analyse de conformité

[label=●,leftmargin=2cm]

- Collecteur
- Analyseur
- Validateur

Couche automatisation

[label=●,leftmargin=2cm]

- Construction
- Déploiement
- Test
- Livraison

2.1 Couche gestion des secrets

2.1.1 Architecture logique

Dans la figure 3.2, nous illustrons les principales composantes du système de gestion des secrets, comprenant l'authentification, la gestion et le stockage des secrets, l'accès et l'utilisation des secrets, ainsi que l'audit et la surveillance. Chaque composante est interconnectée, formant un flux logique pour assurer la sécurité et la traçabilité des informations sensibles.

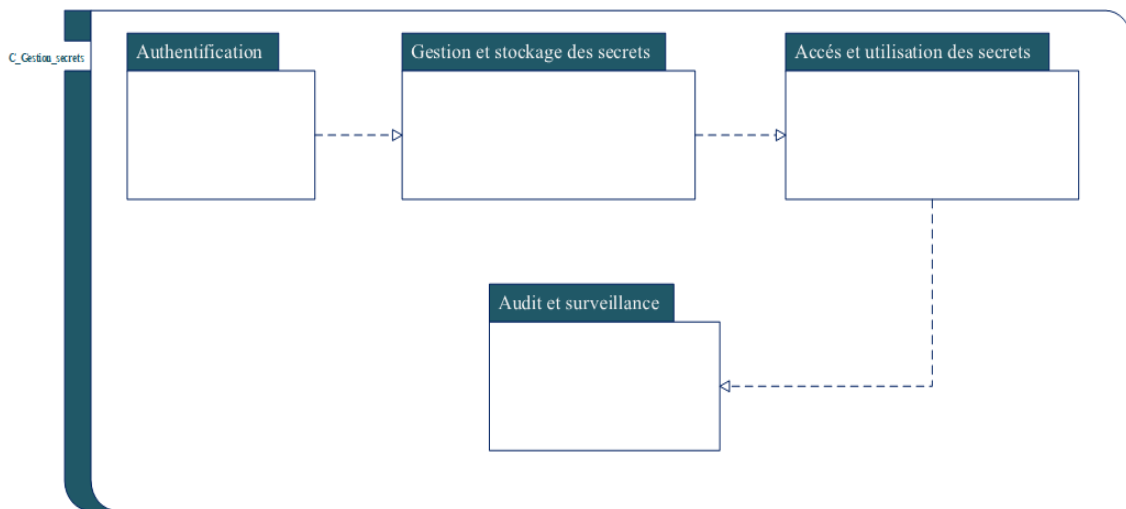


FIGURE 3.2 : Architecture logique (Gestion des secrets)

2.1.2 Diagramme d'interaction

La figure 3.3 présente le processus d'interaction entre un utilisateur et le serveur de vault. Les étapes incluent l'authentification avec un jeton, la demande de secrets, l'utilisation des secrets avec un accès réussi, et l'audit et la surveillance pour garantir la sécurité et la conformité tout au long de l'interaction.

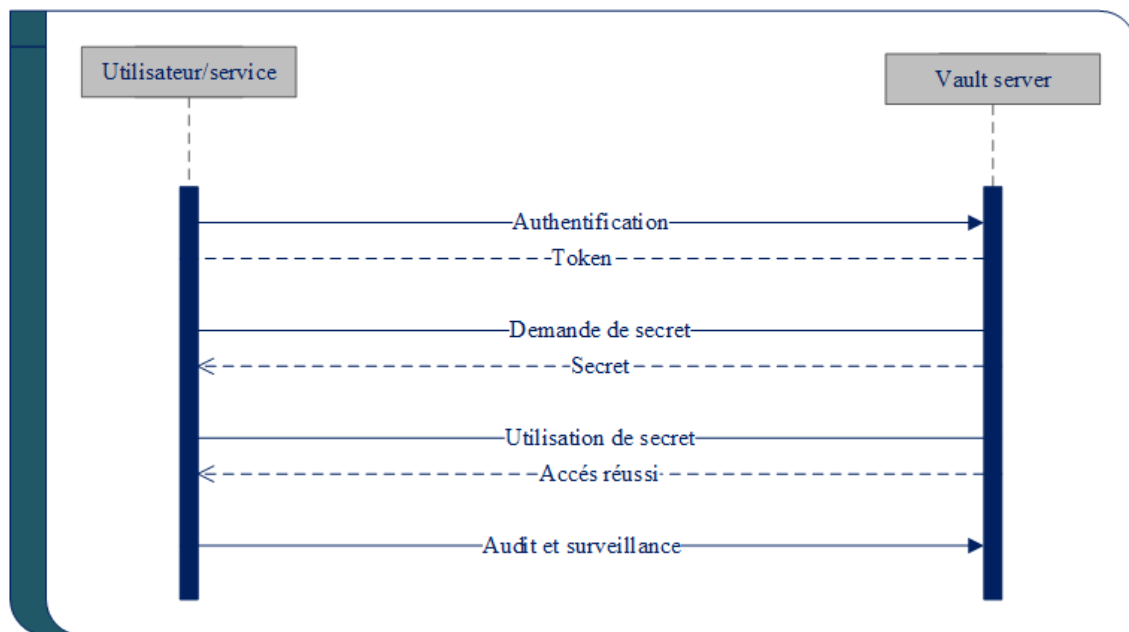


FIGURE 3.3 : Diagramme d'interaction (Gestion des secrets)

2.2 Couche provisionnement

Avant de plonger dans les détails spécifiques de la couche de provisionnement, il est crucial de comprendre son rôle fondamental dans l'architecture globale. La couche de provisionnement est responsable de l'initialisation, de la planification et de l'exécution des ressources nécessaires pour soutenir l'infrastructure. Elle établit les bases sur lesquelles les autres composants de l'architecture s'appuient, garantissant que les environnements sont correctement configurés et prêts pour les étapes ultérieures du déploiement et de la gestion de la sécurité.

2.2.1 Architecture logique

La figure 3.4 suivante illustre ces processus et leurs interactions clés.

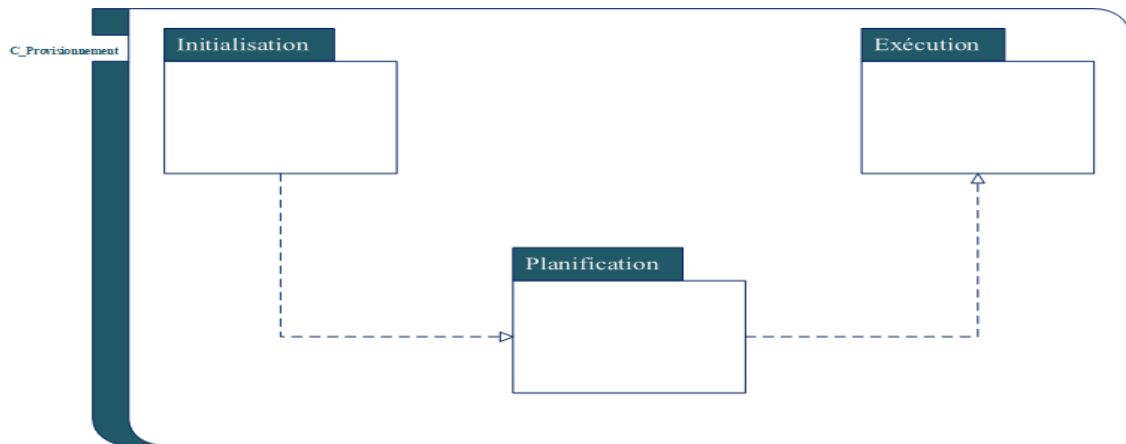


FIGURE 3.4 : Architecture logique (Provisionnement)

2.2.2 Diagramme d'interaction

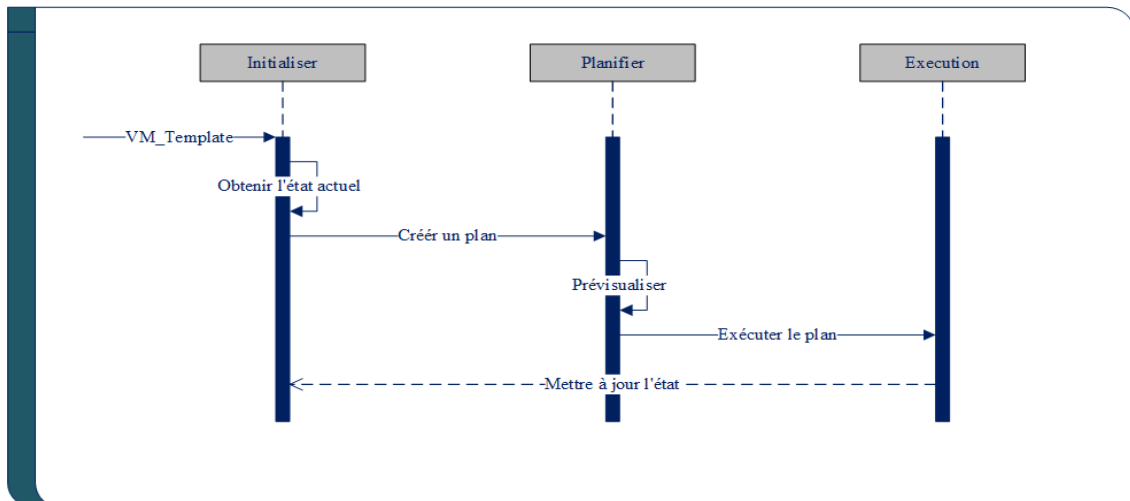


FIGURE 3.5 : Diagramme d'interaction (Provisionnement)

2.3 Couche configuration

2.3.1 Architecture logique

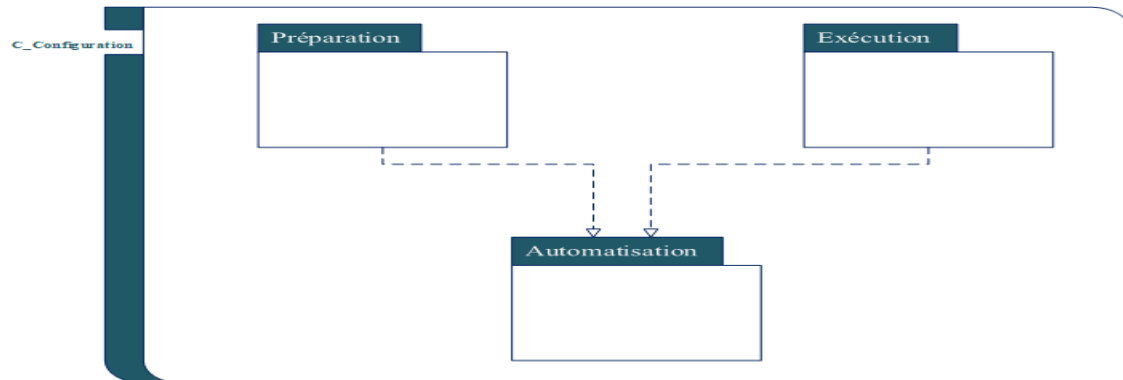


FIGURE 3.6 : Architecture logique (Configuration)

Dans la figure 3.6, nous présentons le processus de configuration, qui se divise en trois phases principales : Préparation, Exécution et Automatisation. La phase de préparation consiste à rassembler toutes les informations nécessaires pour configurer le système. Ensuite, l'exécution met en œuvre les étapes définies pour configurer le système conformément aux spécifications. Enfin, l'automatisation joue un rôle crucial en intégrant des scripts et des outils pour automatiser le processus de configuration, assurant ainsi une répétabilité et une efficacité accrues.

2.3.2 Diagramme d'interaction

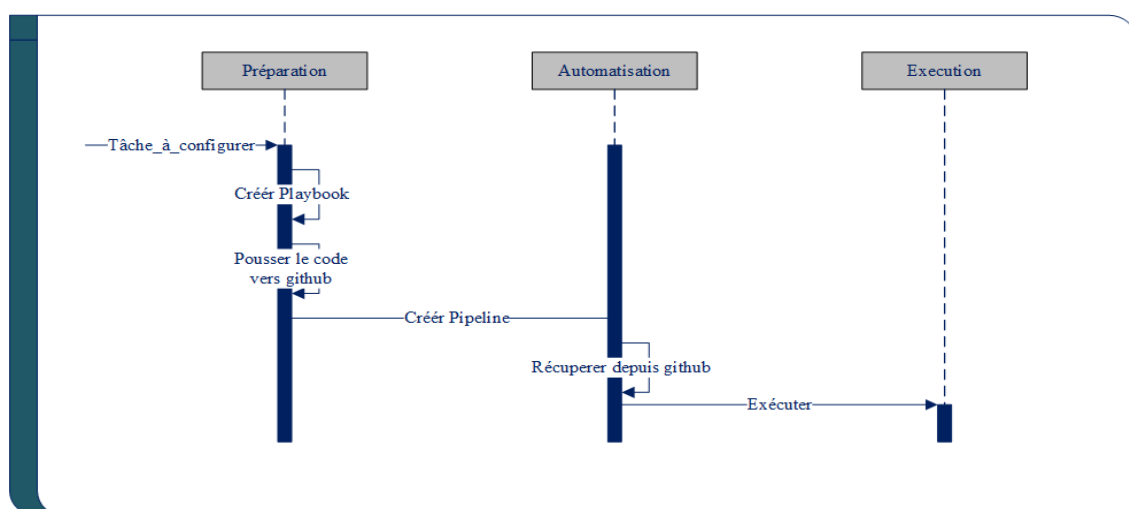


FIGURE 3.7 : Diagramme d'interaction (Configuration)

2.4 Couche analyse de conformité

2.4.1 Architecture logique

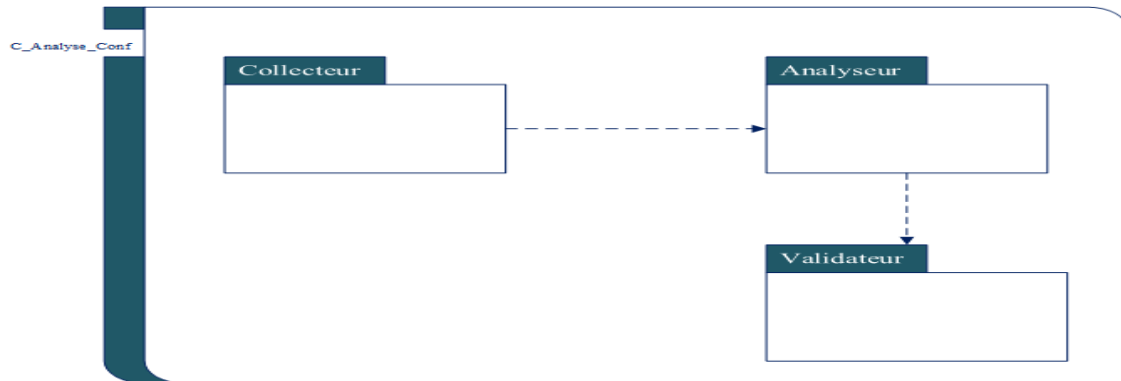


FIGURE 3.8 : Architecture logique (Analyse de conformité)

Dans cette illustration, nous montrons le processus d'analyse de conformité, qui est subdivisé en trois étapes : Collecteur, Analyseur et Valideur. La phase de Collecteur recueille les données de conformité à partir de différentes sources. L'Analyseur traite ces données pour identifier des anomalies, des incohérences ou des points de non-conformité potentiels. Enfin, le Valideur vérifie les résultats de l'analyse pour s'assurer qu'ils sont corrects et conformes aux normes de sécurité définies. Ce processus est essentiel pour maintenir la qualité et la conformité des configurations de sécurité au sein du projet.

2.4.2 Diagramme d'interaction

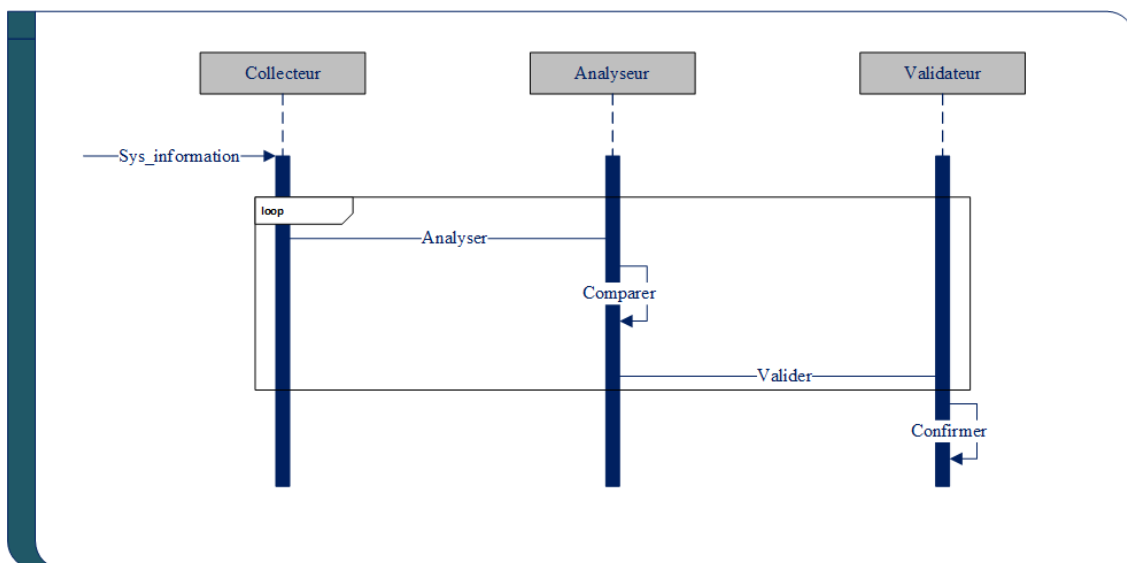


FIGURE 3.9 : Diagramme d'interaction (Analyse de conformité)

2.5 Couche automatisations

2.5.1 Architecture logique

La figure 3.10 illustre le cycle de vie de l'automatisation dans notre projet, comprenant les étapes de Construction, Déploiement, Test et Livraison. La phase de Construction implique le développement et l'intégration des composants logiciels. Une fois construits, ces composants sont Déployés dans l'environnement cible. Les Tests sont effectués pour garantir que le système fonctionne comme prévu, et finalement, la phase de Livraison assure la mise en production du logiciel testé. Ce cycle est itératif, avec des retours en boucle pour une amélioration continue.

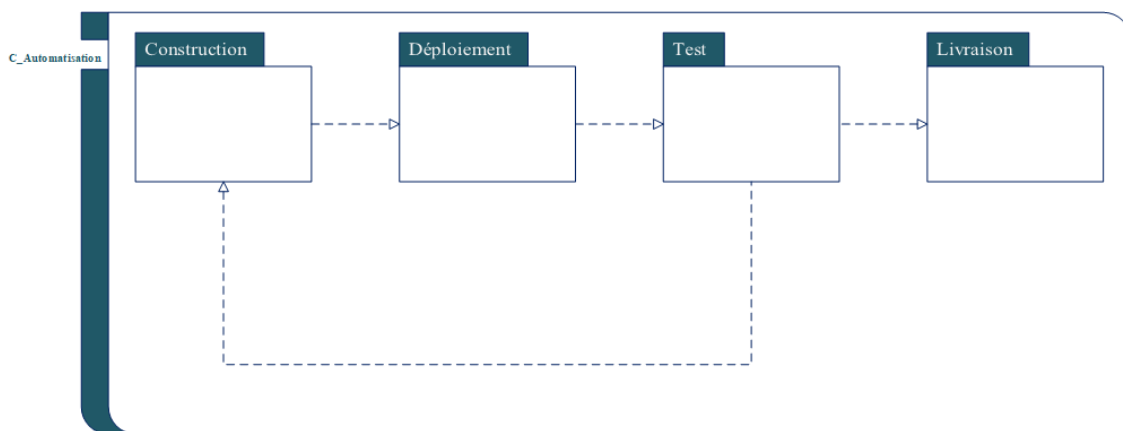


FIGURE 3.10 : Architecture logique (Automatisation)

2.5.2 Diagramme d'interaction

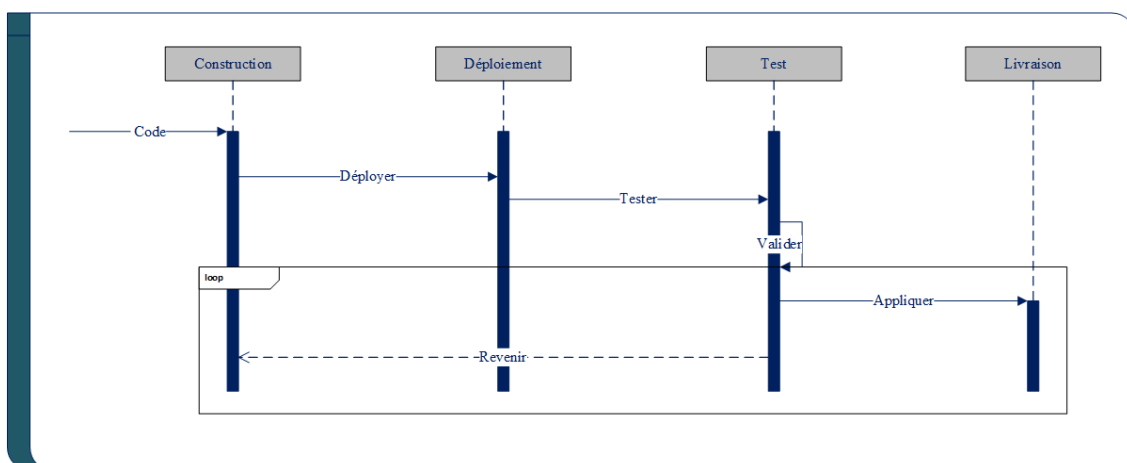


FIGURE 3.11 : Diagramme d'interaction (Automatisation)

3 Présentation de l'architecture de la solution envisagée

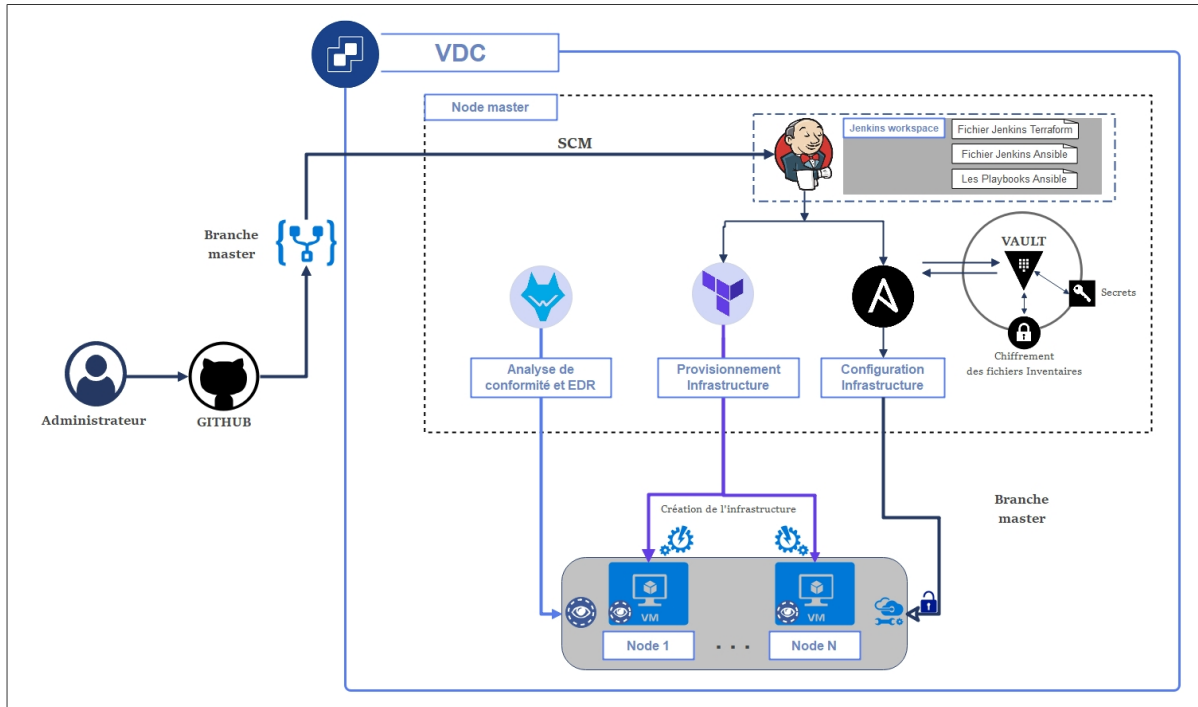


FIGURE 3.12 : Architecture de la solution envisagée

L'architecture illustrée dans la figure 3.12 détaille le flux de travail automatisé pour la sécurité de l'infrastructure. L'administrateur pousse les modifications vers la branche master sur GitHub, ce qui déclenche un processus via SCM dans l'environnement VDC. Jenkins, en tant que serveur d'intégration et déploiement continu, exécute des scripts stockés dans son espace de travail pour le provisionnement de l'infrastructure avec Terraform et la configuration des ressources avec Ansible. Wazuh est utilisé pour l'analyse de conformité ainsi que pour les capacités de détection et réponse aux incidents (EDR), assurant que l'infrastructure respecte les politiques de sécurité et de conformité et peut réagir aux menaces en temps réel. Les secrets nécessaires sont gérés et chiffrés par Vault, garantissant la sécurité des données sensibles. Une fois l'infrastructure créée, les nœuds sont provisionnés et configurés de manière sécurisée, assurant une gestion conforme et sécurisée de l'infrastructure déployée.

3.1 Architecture de la détection et réponse aux endpoints

L'architecture présentée dans la figure 3.13 illustre le flux de travail du système de détection et réponse aux endpoints (EDR) utilisant Wazuh. Les agents Wazuh détectent les attaques et

transmettent les informations au serveur Wazuh pour la collecte et l'analyse. Les données sont ensuite indexées par Wazuh Indexer pour un stockage efficace. Les résultats de l'analyse et les alertes sont visualisés via le tableau de bord Wazuh, accessible par l'administrateur. Ce dernier peut ainsi surveiller les alertes et les réponses actives, assurant une réaction rapide aux incidents. Enfin, les agents reçoivent les réponses appropriées pour répondre aux attaques identifiées.

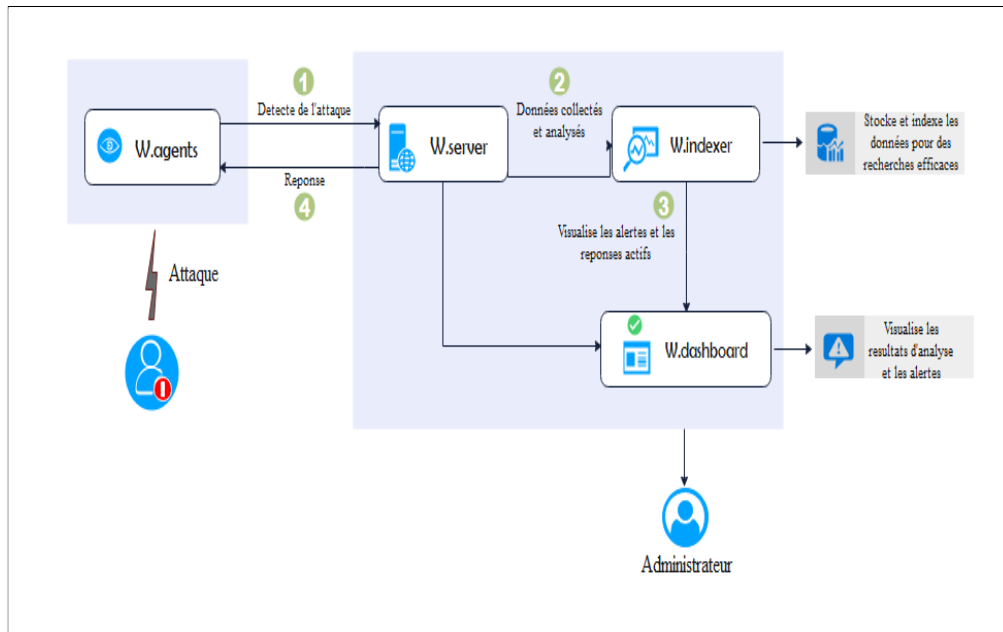


FIGURE 3.13 : Architecture de EDR

[label=●,leftmargin=1.25cm]

- **L'indexeur WAZUH** : Il s'agit d'un moteur de recherche et d'analyse en texte intégral hautement évolutif. Ce composant central indexe et stocke les alertes générées par le serveur Wazuh.
- **Le serveur WAZUH** : Il analyse les données reçues des agents et les traite à l'aide de décodeurs et de règles, en utilisant des informations sur les menaces pour détecter des indicateurs de compromission connus. Un serveur unique permet d'analyser les données de centaines ou de milliers d'agents et peut évoluer horizontalement lorsqu'il est configuré en cluster.
Ce composant central sert également à gérer les agents, les configurer et les mettre à jour à distance.
- **Le tableau de bord WAZUH** : C'est l'interface utilisateur web permettant de visualiser et d'analyser les données. Elle inclut des tableaux de bord prêts à l'emploi pour les

événements de sécurité, la conformité réglementaire, les applications vulnérables détectées, les données de surveillance de l'intégrité des fichiers, les résultats de l'évaluation de la configuration, la surveillance des événements de l'infrastructure cloud, entre autres.

- **Les agents WAZUH** : Ils sont installés sur des terminaux les machines virtuelles dans notre cas. Ils offrent des capacités de prévention, de détection et de réponse aux menaces.

Conclusion

Dans ce chapitre, nous avons détaillé la conception de notre projet en élaborant les modèles UML pour présenter la vue formelle des données qui seront utilisées. Nous avons également détaillé les différents composants de notre plateforme et leurs déroulements à l'aide des diagrammes de classe et les diagrammes de séquence. Cette phase est primordiale avant d'entamer la phase d'implémentation qui sera détaillé dans le chapitre suivant.

RÉALISATION

Plan

Introduction	51
1 Environnement de travail	51
2 Implémentation de solution	52
3 Résultat	66
Conclusion	73

Introduction

Dans ce chapitre, nous présenterons l'environnement matériel et logiciel adopté pour obtenir un produit fini et nous présenterons également les principales interfaces graphiques. En effet, ce chapitre sera composé de deux parties. La première partie détaillera l'environnement de développement et la seconde partie concernera la mise en œuvre de la solution proposée.

1 Environnement de travail

1.1 Environnement matériel

Marque	RAM	CPU	Stockage	Système exploitation
Dell	16 GB	Intel(R) Core(TM) i5-1135G7 @ 2.40GHz 2.42 GHz	250 GB	Windows 11 Pro
MSI	16 GB	Intel(R) Core (TM) i5-9600KF CPU @ 3.70GHz	1.2 TB	Windows 10 Pro

TABLEAU 4.1 : Environnement locale

	RAM	CPU	Stockage
VM master(VDC)	16 Go	4	100 Go
VM worker1(VDC)	16 Go	4	100 Go
VM worker 2(VDC)	16 Go	4	100 Go

TABLEAU 4.2 : Environnement Cloud

1.2 Environnement logiciel

Outils	Version
Serveur Ubuntu	20.04
2 clients Ubuntu	20.04
Terraform	1.8.2
Ansible	Core 2.12.10
Wazuh	4.7.4
Jenkins	2.455
Vault	1.17

TABLEAU 4.3 : Environnement logiciel

2 Implémentation de solution

Dans cette partie, nous allons détailler le travail réalisé pour mettre en œuvre la solution proposée. Nous allons décrire les étapes suivies ainsi que les défis rencontrés et les solutions apportées. Ce travail inclut la configuration de l'environnement de développement, l'intégration des divers composants de l'architecture système, et la validation du bon fonctionnement de l'ensemble.

2.1 La phase de gestion des secrets

Nous avons effectué l'authentification avec Vault via la commande "vault login", et avons ensuite activé le moteur de secrets KV avec "vault secrets enable -path=secret kv" comme indiqué par le succès des opérations affichées dans la figure 4.1.

```
root@master-vm:~# vault login
WARN[0000] log.go:249 gosamin take.(*defaultlogger).Warn DBUS_SESSION_BUS_ADDRESS envvar looks to be not set, this can lead to runaway dbus-daemon processes. To avoid this, set envvar DBUS_SESSION_BUS_ADD
ESS=$XDG_RUNTIME_DIR/bus (if it exists) or DBUS_SESSION_BUS_ADDRESS=/dev/null.
Token (will be hidden):
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key      Value
---      -
token    |_&6z|n||/v|e8VtM7Fj|k|K|N|
token_accessor     7mWUdYV777H U W Y J E L C P S Q
token_duration      =
token_renewable     false
token_policies       ["root"]
identity_policies    []
policies             ["root"]

root@master-vm:~# vault secrets enable -path=secret kv
WARN[0000] log.go:249 gosamin take.(*defaultlogger).Warn DBUS_SESSION_BUS_ADDRESS envvar looks to be not set, this can lead to runaway dbus-daemon processes. To avoid this, set envvar DBUS_SESSION_BUS_ADD
ESS=$XDG_RUNTIME_DIR/bus (if it exists) or DBUS_SESSION_BUS_ADDRESS=/dev/null.
Success! Enabled the kv secrets engine at: secret/

root@master-vm:~#
```

FIGURE 4.1 : Authentification et autorisation

La figure 4.2 montre que nous avons utilisé la commande "vault kv put secret/vsphere" pour stocker les secrets vSphere, incluant les informations de connexion et les détails du réseau, avec succès dans Vault.

```

root@master-vm:~# vault kv put secret/vsphere login="j" password="j" vsphere_server="vc-vstack-01-lab.virtualstack.in" datastore_name="ds002-lab-stx3" data1-dtx1="datacenter_name="DTX1"
"cluster name"="Clu001-UCS02-PRD" "network name"="VLAN LAB"
WARN[0000] log.go:244 gosnowflake.(*defaultLogger).Warn DBUS_SESSION_BUS_ADDRESS envvar looks to be not set, this can lead to runaway dbus-daemon processes. To avoid this, set envvar DBUS_SESSION_BUS_ADDRESS=$XDG_RUNTIME_DIR/bus (if it exists) or DBUS_SESSION_BUS_ADDRESS=/dev/null.
Success! Data written to: secret/vsphere
root@master-vm:~#

```

FIGURE 4.2 : Configuration de vault pour stocker un secret

La capture suivante montre que nous avons intégré Vault dans notre code Terraform pour récupérer les secrets depuis Vault et les utiliser dans notre infrastructure vSphere.

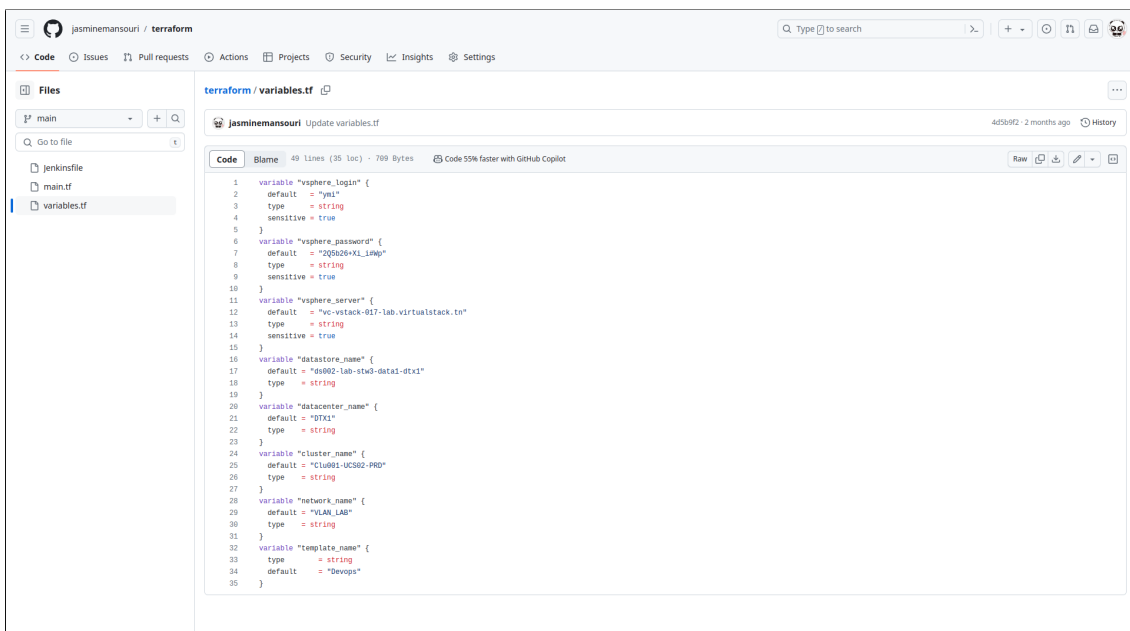


```
1 terraform {
2   required_version = ">= 1.7.0" // Assurez-vous que cela correspond à une version supportée par votre configuration.
3   required_providers {
4     vsphere = {
5       source = "hashicorp/vsphere"
6       version = "= 1.15.0"
7     }
8   }
9 }
10 provider "vault" {
11   address = "http://127.0.0.1:8200"
12 }
13 data "vault_kv_secret" "vsphere" {
14   path = "secret/vsphere"
15 }
16 provider "vsphere" {
17   user          = data.vault_kv_secret.vsphere.data["login"]
18   password      = data.vault_kv_secret.vsphere.data["password"]
19   vsphere_server = data.vault_kv_secret.vsphere.data["vsphere_server"]
20   allow_unverified_ssl = true
21 }
```

FIGURE 4.3 : Integration de vault dans Terraform

2.2 La phase de provisionnement avec Terraform

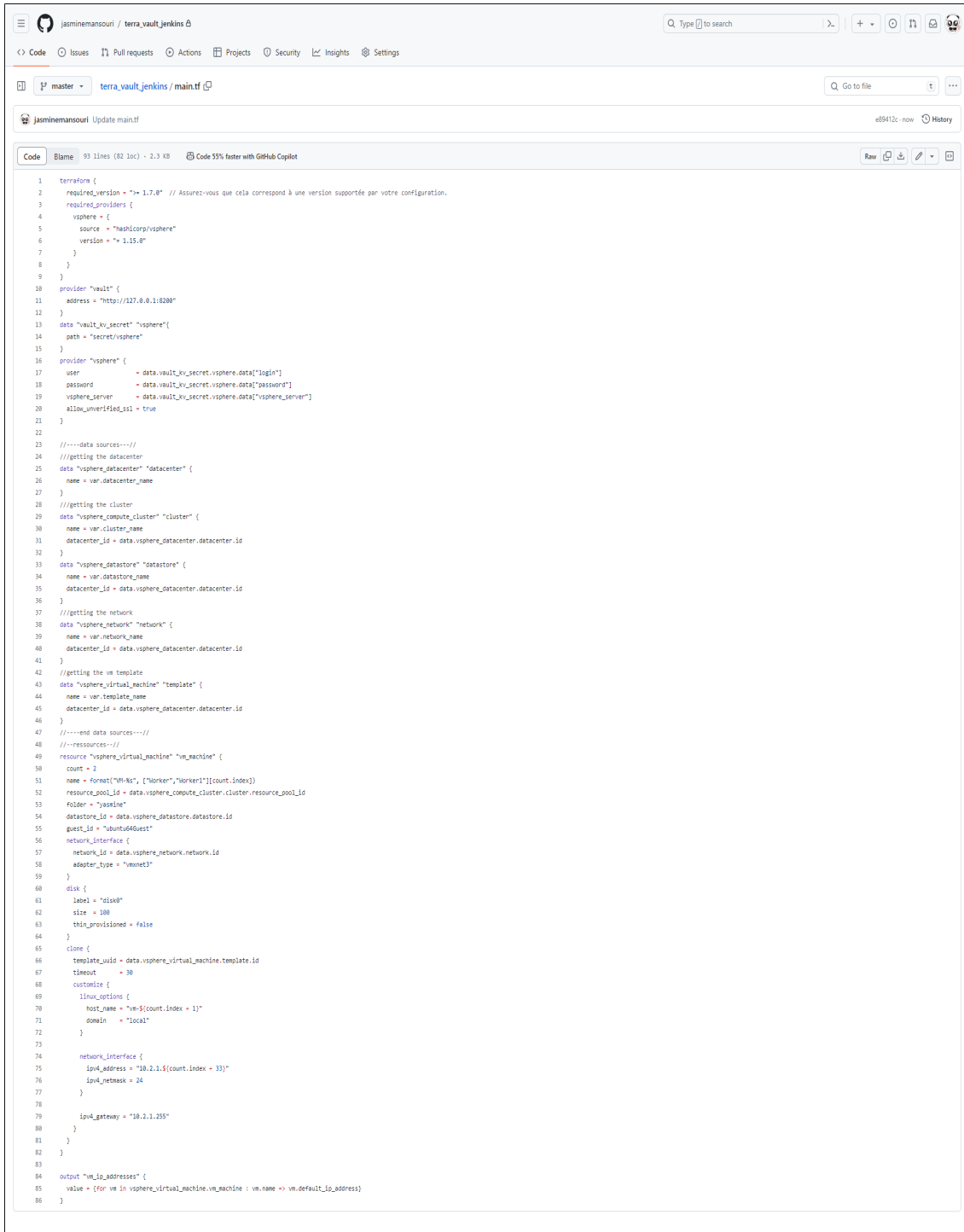
La figure 4.4 présente le fichier variables.tf où sont définies les variables nécessaires pour la connexion et la configuration de l'infrastructure vSphere.



```
1 variable "vsphere_login" {
2   default = "yml"
3   type    = string
4   sensitive = true
5 }
6 variable "vsphere_password" {
7   default = "2020b0x1_lamp"
8   type    = string
9   sensitive = true
10 }
11 variable "vsphere_server" {
12   default = "vc-vstack-017-lab.virtualstack.to"
13   type    = string
14   sensitive = true
15 }
16 variable "datastore_name" {
17   default = "ds002-lab-stw3-data1-dtx1"
18   type    = string
19 }
20 variable "datacenter_name" {
21   default = "DTX1"
22   type    = string
23 }
24 variable "cluster_name" {
25   default = "Clu001-UCS02-PR0"
26   type    = string
27 }
28 variable "network_name" {
29   default = "VLAN_LAB"
30   type    = string
31 }
32 variable "template_name" {
33   type    = string
34   default = "Devops"
35 }
```

FIGURE 4.4 : Configuration des Variables dans Terraform

La figure 4.5 affiche la configuration principale utilisée pour provisionner les machines virtuelles sur la plateforme vSphere à l'aide de Terraform en utilisant le langage HCL. Ce script définit les fournisseurs, les sources de données et les ressources nécessaires pour créer et configurer les machines virtuelles, en utilisant les variables définies précédemment.



```
1 terraform {
2   required_version = ">= 1.7.0" // Assurez-vous que cela correspond à une version supportée par votre configuration.
3 }
4 required_providers {
5   vsphere = {
6     source = "hashicorp/vsphere"
7     version = "~> 1.15.0"
8   }
9 }
10 provider "vault" {
11   address = "http://127.0.0.1:8200"
12 }
13 data "vault_kv_secret" "vsphere" {
14   path = "secret/vsphere"
15 }
16 provider "vsphere" {
17   user       = data.vault_kv_secret.vsphere.data["login"]
18   password   = data.vault_kv_secret.vsphere.data["password"]
19   vsphere_server = data.vault_kv_secret.vsphere.data["vsphere_server"]
20   allow_unverified_ssl = true
21 }
22
23 //----data sources----
24 //getting the datacenter
25 data "vsphere_datacenter" "datacenter" {
26   name = var.datacenter_name
27 }
28 //getting the cluster
29 data "vsphere_compute_cluster" "cluster" {
30   name = var.cluster_name
31   datacenter_id = data.vsphere_datacenter.datacenter.id
32 }
33 data "vsphere_datastore" "datastore" {
34   name = var.datastore_name
35   datacenter_id = data.vsphere_datacenter.datacenter.id
36 }
37 //getting the network
38 data "vsphere_network" "network" {
39   name = var.network_name
40   datacenter_id = data.vsphere_datacenter.datacenter.id
41 }
42 //getting the vm template
43 data "vsphere_virtual_machine" "template" {
44   name = var.template_name
45   datacenter_id = data.vsphere_datacenter.datacenter.id
46 }
47 //----end data sources----
48 //---resources---
49 resource "vsphere_virtual_machine" "vm_machine" {
50   count = 1
51   name = Format("VM-%s", ["Worker", "Worker1"][count.index])
52   resource_pool_id = data.vsphere_compute_cluster.cluster.resource_pool_id
53   folder = "yaxline"
54   datastore_id = data.vsphere_datastore.datastore.id
55   guest_id = "Ubuntu64Guest"
56   network_interface {
57     network_id = data.vsphere_network.network.id
58     adapter_type = "vmxnet3"
59   }
60   disk {
61     label = "disk0"
62     size = 100
63     thin_provisioned = false
64   }
65   clone {
66     template_uuid = data.vsphere_virtual_machine.template.id
67     timeout = 30
68     customize {
69       linux_options {
70         host_name = "VM-${count.index + 1}"
71         domain = "local"
72       }
73     }
74     network_interface {
75       ip4_address = "10.2.1.5${count.index + 33}"
76       ip4_netmask = 24
77     }
78     ip4_gateway = "10.2.1.255"
79   }
80 }
81
82
83
84 output "vm_ip_addresses" {
85   value = [for vm in vsphere_virtual_machine.vm_machine : vm.name => vm.default_ip_address]
86 }
```

FIGURE 4.5 : Script Principal de Provisionnement

L'interface vSphere dans la figure 4.6 montre le succès du déploiement des machines virtuelles, illustrant les machines VM-Worker1 et VM-Worker2 qui ont été créées et configurées selon les spécifications fournies dans les fichiers Terraform. Chaque machine virtuelle est alimentée et prête à l'emploi, avec des ressources allouées et des systèmes d'exploitation installés.

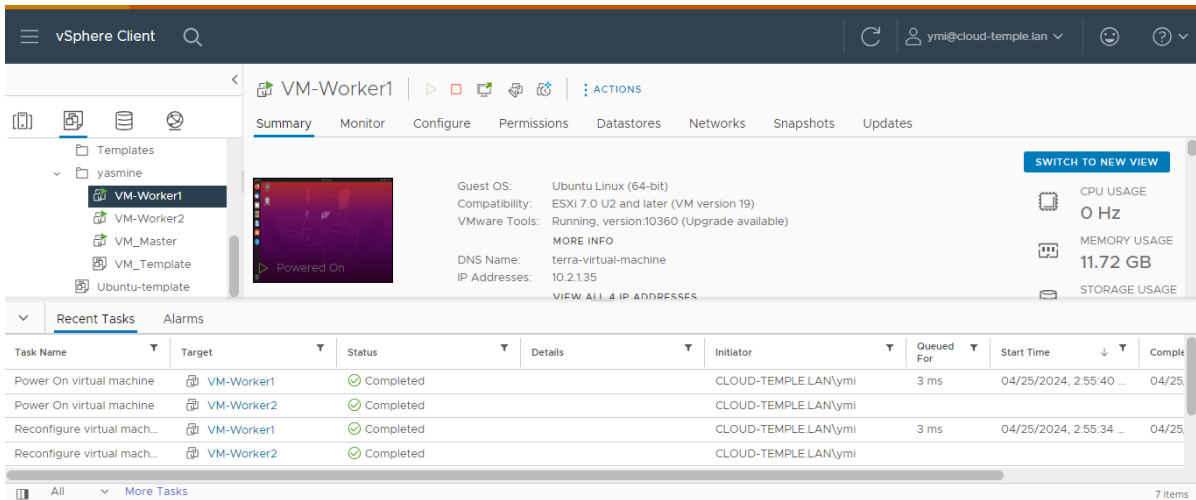


FIGURE 4.6 : Résultat du Déploiement sur vSphere

2.3 La phase de configuration automatisé anc Ansible

[label=●,leftmargin=1.5cm]

- **Configuration automatique des adresses IP** : La figure 4.7 illustre le playbook contenant des tâches Ansible pour configurer les adresses IP sur les VMs créées précédemment. Les tâches incluent la modification des fichiers de configuration réseau, la définition des adresses IP statiques en fonction de l'infrastructure.

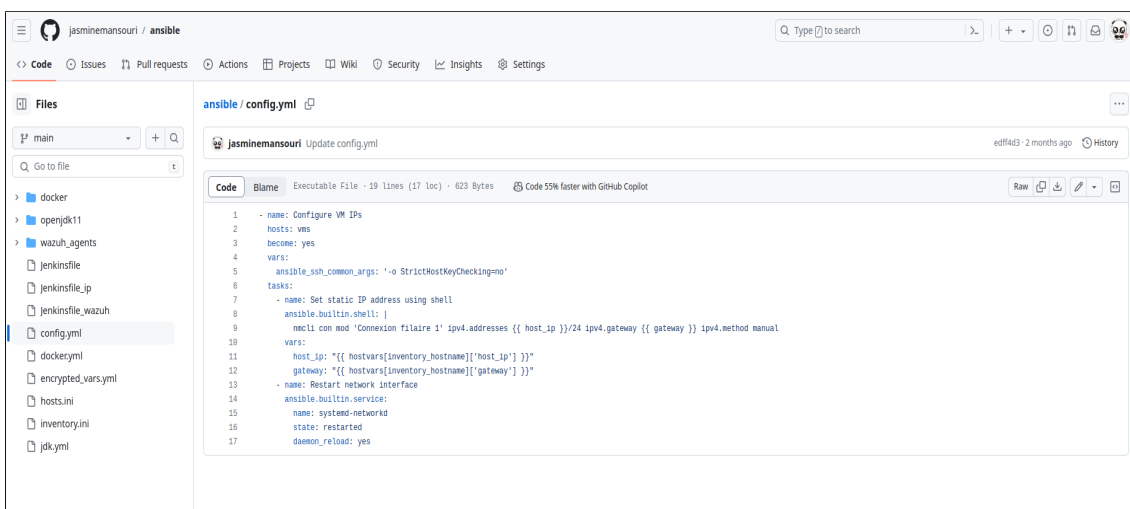


FIGURE 4.7 : Configuration automatique des adresses IP

- **Installation automatique de Docker** : La figure 4.8 illustre le role contenant une séquence détaillée de tâches pour installer Docker, incluant l'installation des paquets requis, l'ajout de la clé GPG Docker, la configuration du dépôt Docker, et le démarrage du service Docker

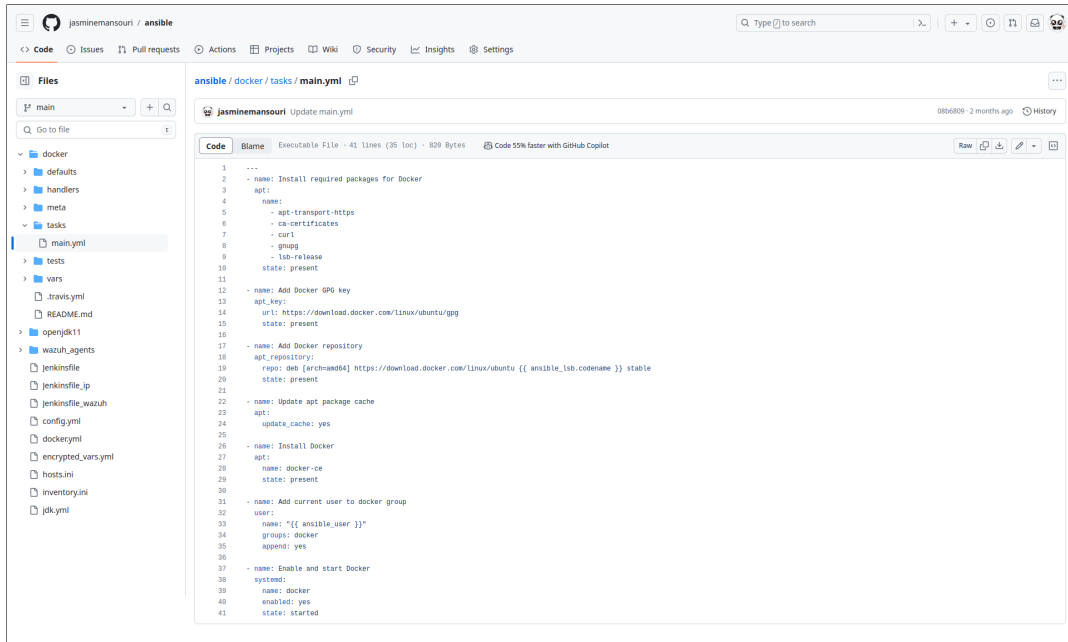


FIGURE 4.8 : Le role d'installation de Docker

La capture suivante représente le playbook qui sert à appeler le role définie précédemment pour entammer la tache de l'installation de docker

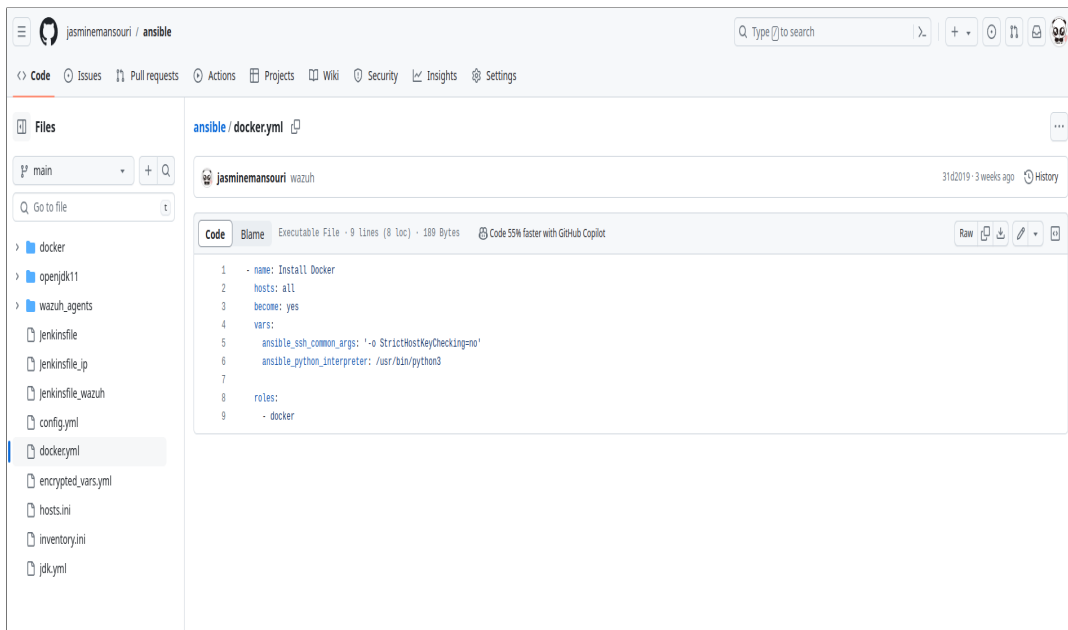


FIGURE 4.9 : Playbook d'installation de Docker

- **Installation automatique de JDK** : La figure 4.10 illustre le role contenant les etapes de l'installation de JDK

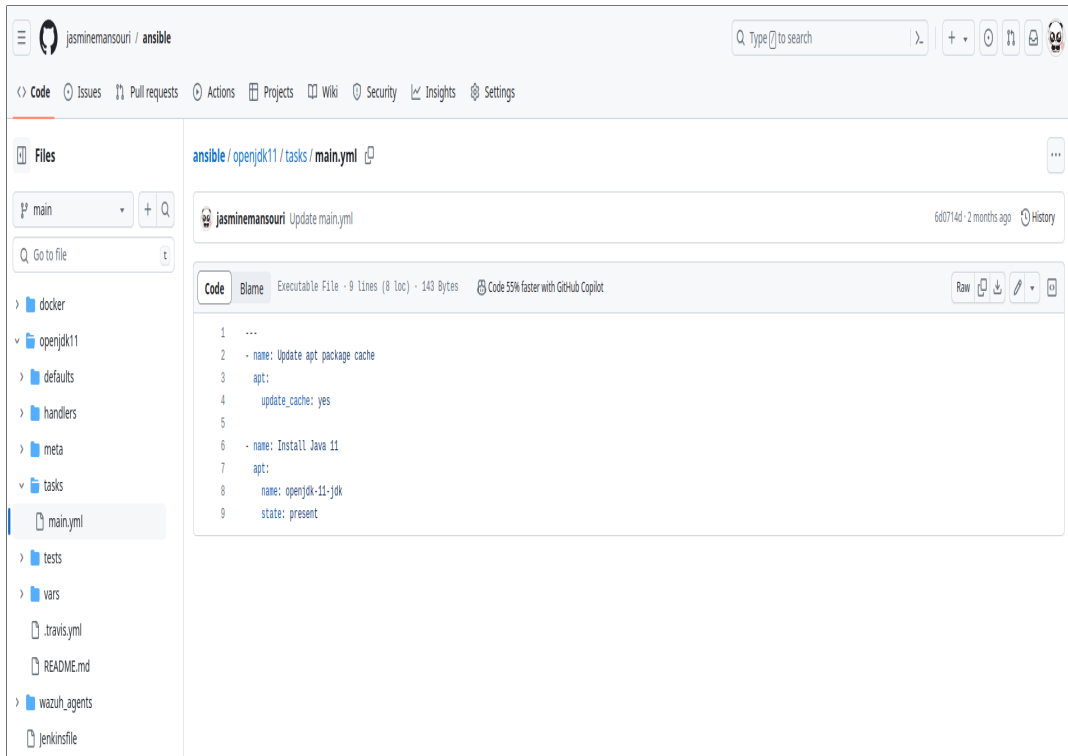


FIGURE 4.10 : Le role d'installation de JDK

La figure suivante illustre le playbook qui sert à appeler le role définie précédemment pour entammer la tache de l'installation de JDK

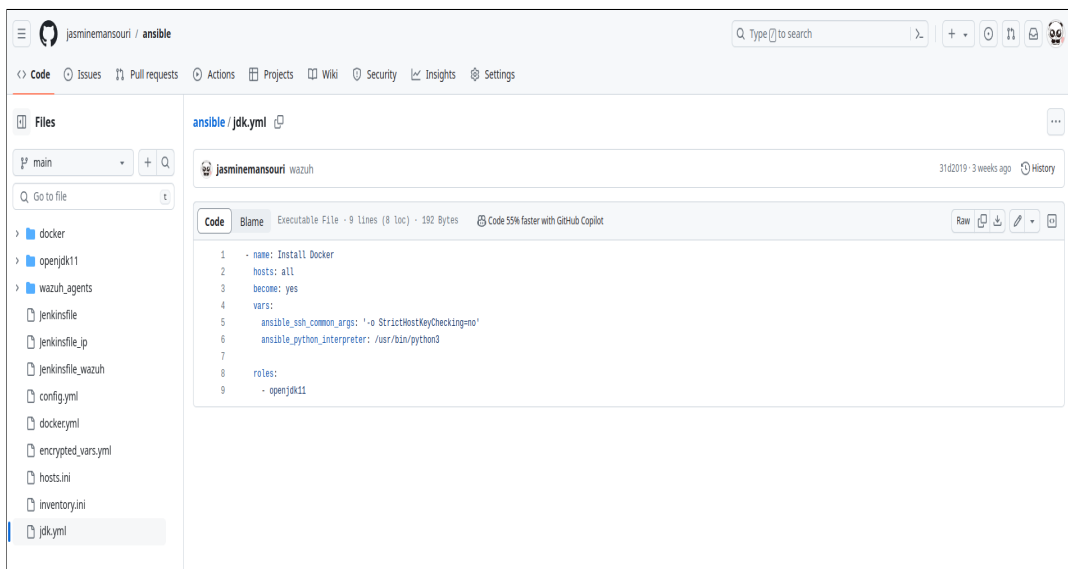


FIGURE 4.11 : Playbook d'installation de JDK

- **Installation automatique des agents Wazuh** :

La figure 4.12 montre notre playbook Ansible pour déployer un agent Wazuh sur des hôtes spécifiés. Les paramètres de connexion et de gestion du manager Wazuh, tels que l'adresse, le port, et l'utilisateur API, sont configurés dans ce fichier.

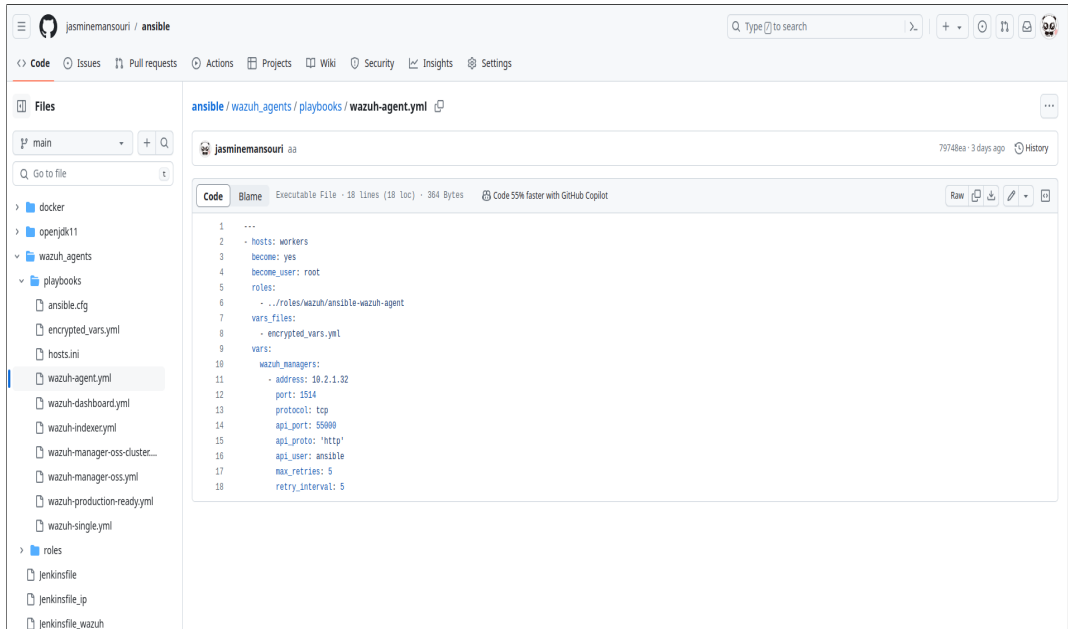


FIGURE 4.12 : Playbook d'installation des agents Wazuh

La capture ci-dessous présente notre script de rôle Ansible. Ce script contient une série de tâches structurées pour installer et configurer l'agent Wazuh sur des systèmes cibles, incluant le téléchargement de fichiers, l'exécution de commandes, et la gestion de services.

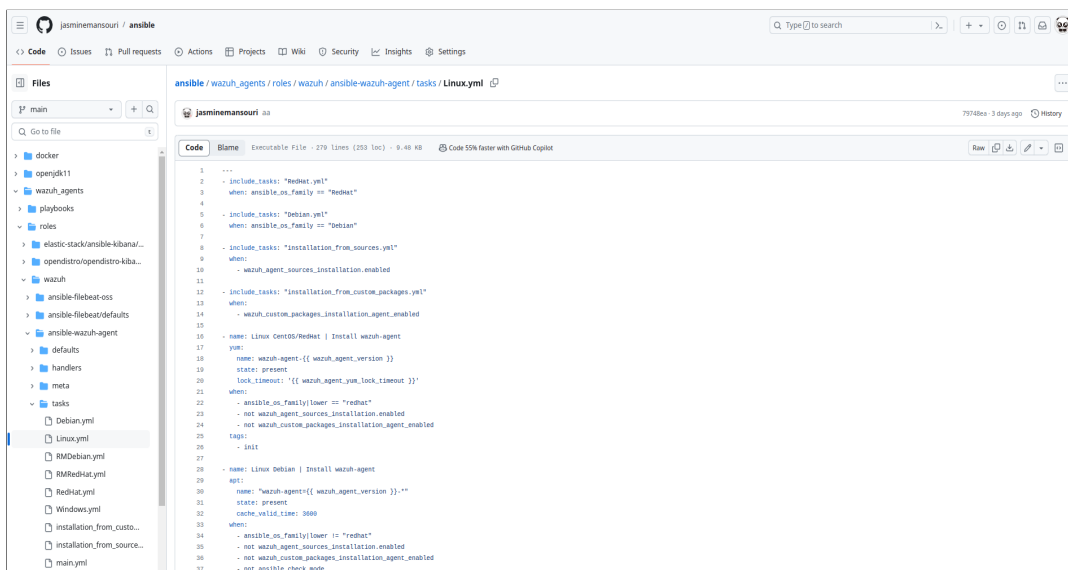


FIGURE 4.13 : Le role des agents Wazuh

2.4 La phase de l'analyse de conformité et EDR

[label=●,leftmargin=1cm]

— Installation de Wazuh Indexer :

Nous avons défini les adresses IP des différents composants de Wazuh comme il est montré dans la figure ci-dessous.



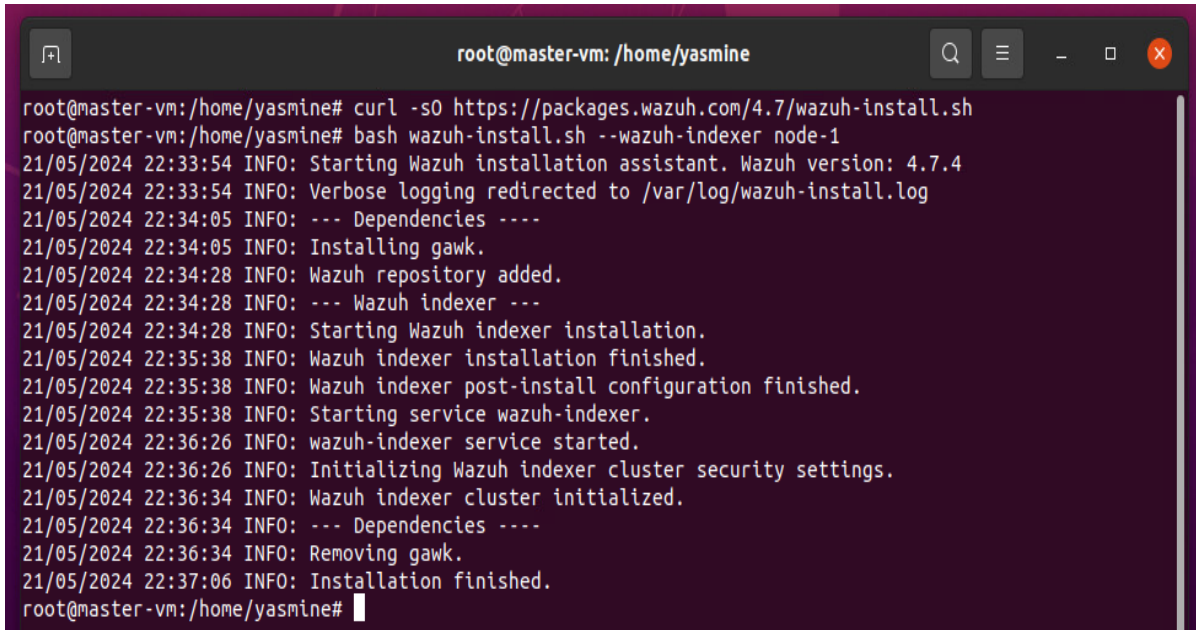
```
root@master-vm: /home/yasmine
GNU nano 4.8                               ./config.yml                               Modified
nodes:
# Wazuh indexer nodes
indexer:
- name: node-1
  ip: "10.2.1.32"
#- name: node-2
#  ip: "<indexer-node-ip>"
#- name: node-3
#  ip: "<indexer-node-ip>"

# Wazuh server nodes
# If there is more than one Wazuh server
# node, each one must have a node_type
server:
- name: wazuh-1
  ip: "10.2.1.32"
#  node_type: master
#- name: wazuh-2
#  ip: "<wazuh-manager-ip>"
#  node_type: worker
#- name: wazuh-3
#  ip: "<wazuh-manager-ip>"
#  node_type: worker

# Wazuh dashboard nodes
dashboard:
- name: dashboard
  ip: "10.2.1.32"
```

FIGURE 4.14 : Fichier de configuration de Wazuh Indexer

La figure 4.15 définit l'installation de l'indexeur Wazuh, avec des étapes similaires d'ajout de dépôts, d'installation des dépendances et de démarrage du service d'indexation.

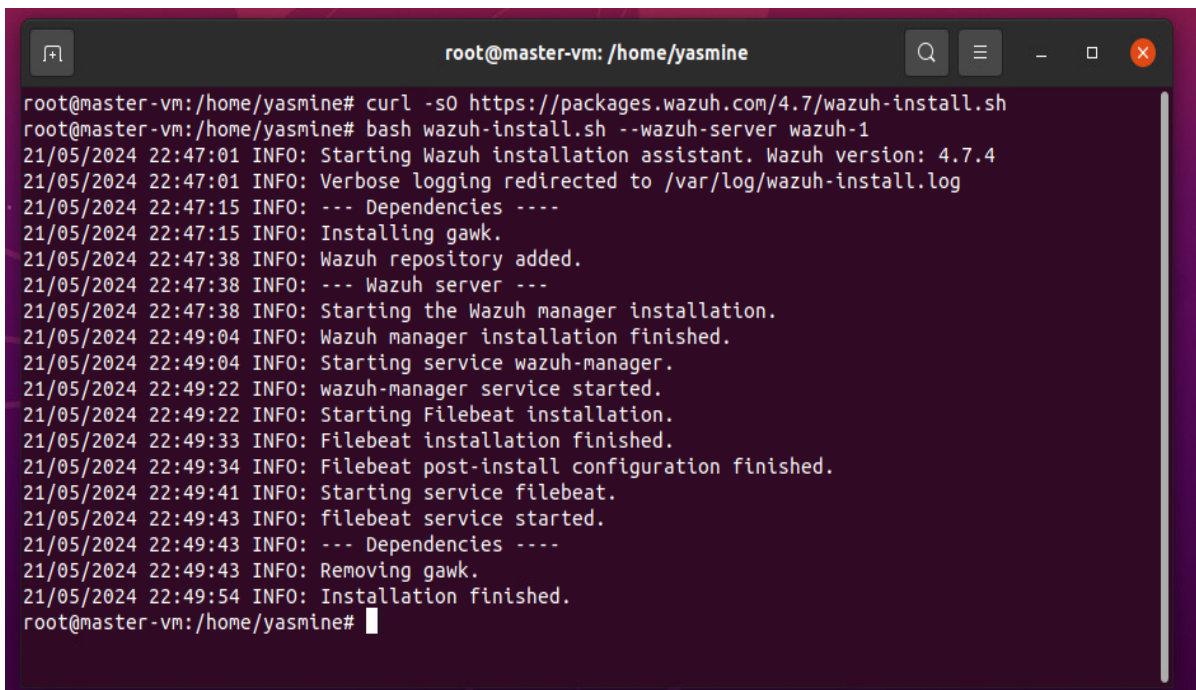
A terminal window titled 'root@master-vm: /home/yasmine' showing the execution of the Wazuh installation script for the indexer. The script is run with the command 'bash wazuh-install.sh --wazuh-indexer node-1'. The output shows various informational messages: starting the installation assistant (version 4.7.4), logging to /var/log/wazuh-install.log, checking dependencies, installing gawk, adding the Wazuh repository, starting the indexer installation, finishing the installation, finishing the post-install configuration, starting the wazuh-indexer service, and initializing the cluster security settings. The installation is completed at 22:37:06.

```
root@master-vm:/home/yasmine# curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
root@master-vm:/home/yasmine# bash wazuh-install.sh --wazuh-indexer node-1
21/05/2024 22:33:54 INFO: Starting Wazuh installation assistant. Wazuh version: 4.7.4
21/05/2024 22:33:54 INFO: Verbose logging redirected to /var/log/wazuh-install.log
21/05/2024 22:34:05 INFO: --- Dependencies ----
21/05/2024 22:34:05 INFO: Installing gawk.
21/05/2024 22:34:28 INFO: Wazuh repository added.
21/05/2024 22:34:28 INFO: --- Wazuh indexer ---
21/05/2024 22:34:28 INFO: Starting Wazuh indexer installation.
21/05/2024 22:35:38 INFO: Wazuh indexer installation finished.
21/05/2024 22:35:38 INFO: Wazuh indexer post-install configuration finished.
21/05/2024 22:35:38 INFO: Starting service wazuh-indexer.
21/05/2024 22:36:26 INFO: wazuh-indexer service started.
21/05/2024 22:36:26 INFO: Initializing Wazuh indexer cluster security settings.
21/05/2024 22:36:34 INFO: Wazuh indexer cluster initialized.
21/05/2024 22:36:34 INFO: --- Dependencies ----
21/05/2024 22:36:34 INFO: Removing gawk.
21/05/2024 22:37:06 INFO: Installation finished.
root@master-vm:/home/yasmine#
```

FIGURE 4.15 : Script de l'installation de Wazuh Indexer

— Installation de Wazuh Server :

Cette capture définit l'installation réussie du serveur Wazuh, y compris l'ajout des dépôts, l'installation des dépendances et le démarrage du service

A terminal window titled 'root@master-vm: /home/yasmine' showing the execution of the Wazuh installation script for the server. The script is run with the command 'bash wazuh-install.sh --wazuh-server wazuh-1'. The output shows various informational messages: starting the installation assistant (version 4.7.4), logging to /var/log/wazuh-install.log, checking dependencies, installing gawk, adding the Wazuh repository, starting the Wazuh manager installation, finishing the manager installation, starting the wazuh-manager service, starting Filebeat installation, finishing Filebeat installation, finishing Filebeat post-install configuration, starting the filebeat service, and removing gawk. The installation is completed at 22:49:54.

```
root@master-vm:/home/yasmine# curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
root@master-vm:/home/yasmine# bash wazuh-install.sh --wazuh-server wazuh-1
21/05/2024 22:47:01 INFO: Starting Wazuh installation assistant. Wazuh version: 4.7.4
21/05/2024 22:47:01 INFO: Verbose logging redirected to /var/log/wazuh-install.log
21/05/2024 22:47:15 INFO: --- Dependencies ----
21/05/2024 22:47:15 INFO: Installing gawk.
21/05/2024 22:47:38 INFO: Wazuh repository added.
21/05/2024 22:47:38 INFO: --- Wazuh server ---
21/05/2024 22:47:38 INFO: Starting the Wazuh manager installation.
21/05/2024 22:49:04 INFO: Wazuh manager installation finished.
21/05/2024 22:49:04 INFO: Starting service wazuh-manager.
21/05/2024 22:49:22 INFO: wazuh-manager service started.
21/05/2024 22:49:22 INFO: Starting Filebeat installation.
21/05/2024 22:49:33 INFO: Filebeat installation finished.
21/05/2024 22:49:34 INFO: Filebeat post-install configuration finished.
21/05/2024 22:49:41 INFO: Starting service filebeat.
21/05/2024 22:49:43 INFO: filebeat service started.
21/05/2024 22:49:43 INFO: --- Dependencies ----
21/05/2024 22:49:43 INFO: Removing gawk.
21/05/2024 22:49:54 INFO: Installation finished.
root@master-vm:/home/yasmine#
```

FIGURE 4.16 : Script de l'installation de Wazuh Server

— Installation de Wazuh Dashboard :

Cette capture illustre l'installation du tableau de bord Wazuh, y compris la configuration

de l'application web et l'initialisation du service, permettant l'accès à l'interface web.

```
root@master-vm: /home/yasmine
root@master-vm:/home/yasmine# bash wazuh-install.sh --wazuh-dashboard dashboard
21/05/2024 22:51:25 INFO: Starting Wazuh installation assistant. Wazuh version: 4.7.4
21/05/2024 22:51:25 INFO: Verbose logging redirected to /var/log/wazuh-install.log
21/05/2024 22:51:36 INFO: --- Dependencies ---
21/05/2024 22:51:36 INFO: Installing gawk.
21/05/2024 22:51:47 INFO: Wazuh web interface port will be 443.
21/05/2024 22:51:55 INFO: Wazuh repository added.
21/05/2024 22:51:55 INFO: --- Wazuh dashboard ---
21/05/2024 22:51:55 INFO: Starting Wazuh dashboard installation.
21/05/2024 22:52:49 INFO: Wazuh dashboard installation finished.
21/05/2024 22:52:49 INFO: Wazuh dashboard post-install configuration finished.
21/05/2024 22:52:49 INFO: Starting service wazuh-dashboard.
21/05/2024 22:52:50 INFO: wazuh-dashboard service started.
21/05/2024 22:53:12 INFO: Initializing Wazuh dashboard web application.
21/05/2024 22:53:13 INFO: Wazuh dashboard web application initialized.
21/05/2024 22:53:13 INFO: --- Summary ---
21/05/2024 22:53:13 INFO: You can access the web interface https://10.2.1.32:443
User: admin
Password: GHr2z?Qvtbs*zyZ*g9ajHyZtT*NHp0*u
21/05/2024 22:53:13 INFO: --- Dependencies ---
21/05/2024 22:53:13 INFO: Removing gawk.
21/05/2024 22:53:27 INFO: Installation finished.
root@master-vm:/home/yasmine#
```

FIGURE 4.17 : Script de l'installation de Wazuh Dashboard

La figure 4.18 montre le tableau de board Wazuh.

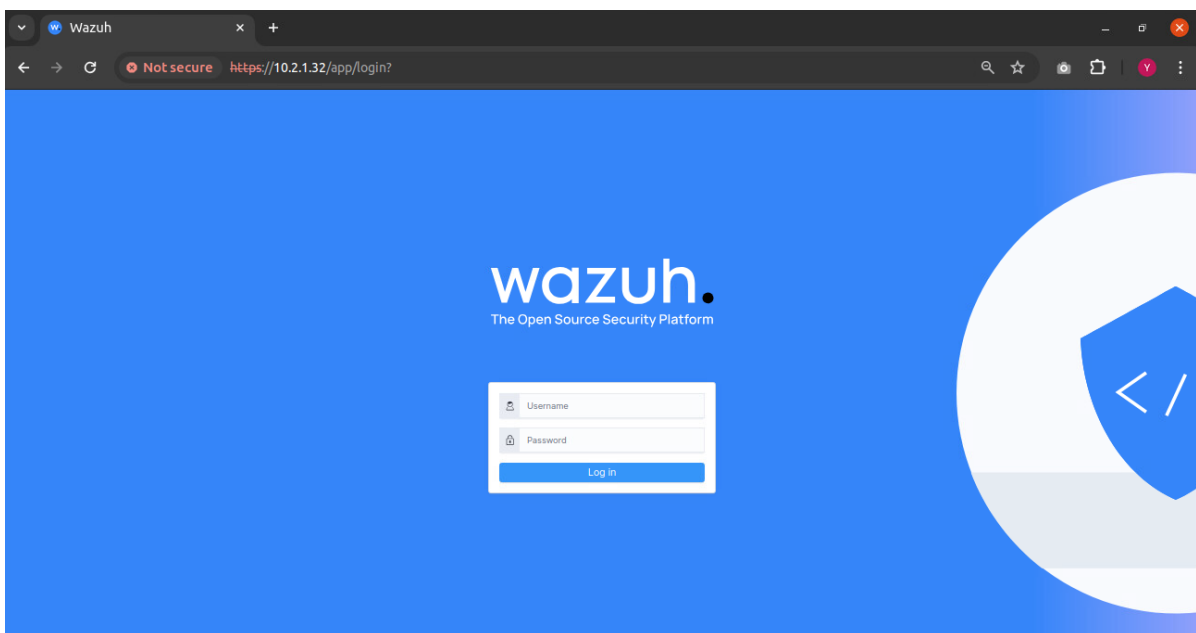


FIGURE 4.18 : Dashboard Wazuh

— **Configuration de la réponse active pour répondre à une tentative Brute Force :**

La figure 4.19 montre la configuration de réponse active dans Wazuh. Cette configuration permet de déclencher la commande "firewall-drop" qui bloque temporairement les connexions réseau en réponse à une règle spécifique.

```
<active-response>
  <command>firewall-drop</command>
  <location>all</location>
  <rules_id>5763</rules_id>
  <timeout>180</timeout>
</active-response>
```

FIGURE 4.19 : Configuration de la réponse active

la figure 4.20 illustre la définition de la commande pour le blocage du pare-feu. Cette configuration associe le nom de la commande "firewall-drop" à l'exécutable "firewall-drop" et permet d'utiliser cette commande avec un délai d'attente. Cela signifie que lorsque cette commande est invoquée, elle peut être configurée pour bloquer les connexions réseau pendant une durée spécifiée.

```
<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

FIGURE 4.20 : Commande de blocage du pare-feu

— Configuration du serveur Wazuh pour le détection et la suppression des logiciels malveillants :

La figure 4.21 montre la définition des règles locales dans Wazuh.

```
GNU nano 4.8                                local_rules.xml
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>

<group name="syscheck,pci_dss_11.5,nist_800_53_S1.7,">
  <!-- Rules for Linux systems -->
  <rule id="100200" level="7">
    <if_sid>550</if_sid>
    <field name="file">/root</field>
    <description>File modified in /root directory.</description>
  </rule>
  <rule id="100201" level="7">
    <if_sid>554</if_sid>
    <field name="file">/root</field>
    <description>File added to /root directory.</description>
  </rule>
</group>
```

FIGURE 4.21 : Définition des règles locales

L'image ci-dessous montre le script utilisé pour la suppression des menaces. Ce script analyse les alertes, vérifie si la commande "add" est reçue, puis supprime le fichier malveillant détecté et enregistre le résultat de l'opération dans un fichier de log

```
GNU nano 4.8 remove-threat.sh
#!/bin/bash

LOCAL=$(dirname $0);
cd $LOCAL
cd ../

PWD=$(pwd)

read INPUT_JSON
FILENAME=$(echo $INPUT_JSON | jq -r .parameters.alert.data.virustotal.source.file)
COMMAND=$(echo $INPUT_JSON | jq -r .command)
LOG_FILE="$PWD/../logs/active-responses.log"

#----- Analyze command -----#
if [ ${COMMAND} = "add" ]
then
# Send control message to execd
printf '{"version":1,"origin":{"name":"remove-threat","module":"active-response"},"command":"check_keys", "parameters":{"keys":[]}}\n'

read RESPONSE
COMMAND2=$(echo $RESPONSE | jq -r .command)
if [ ${COMMAND2} != "continue" ]
then
echo "$(date '+%Y/%m/%d %H:%M:%S') $0: $INPUT_JSON Remove threat active response aborted" >> ${LOG_FILE}
exit 0;
fi
fi

# Removing file
rm -f $FILENAME
if [ $? -eq 0 ]; then
echo "$(date '+%Y/%m/%d %H:%M:%S') $0: $INPUT_JSON Successfully removed threat" >> ${LOG_FILE}
else
echo "$(date '+%Y/%m/%d %H:%M:%S') $0: $INPUT_JSON Error removing threat" >> ${LOG_FILE}
fi

exit 0;
```

FIGURE 4.22 : Script de suppression des menaces

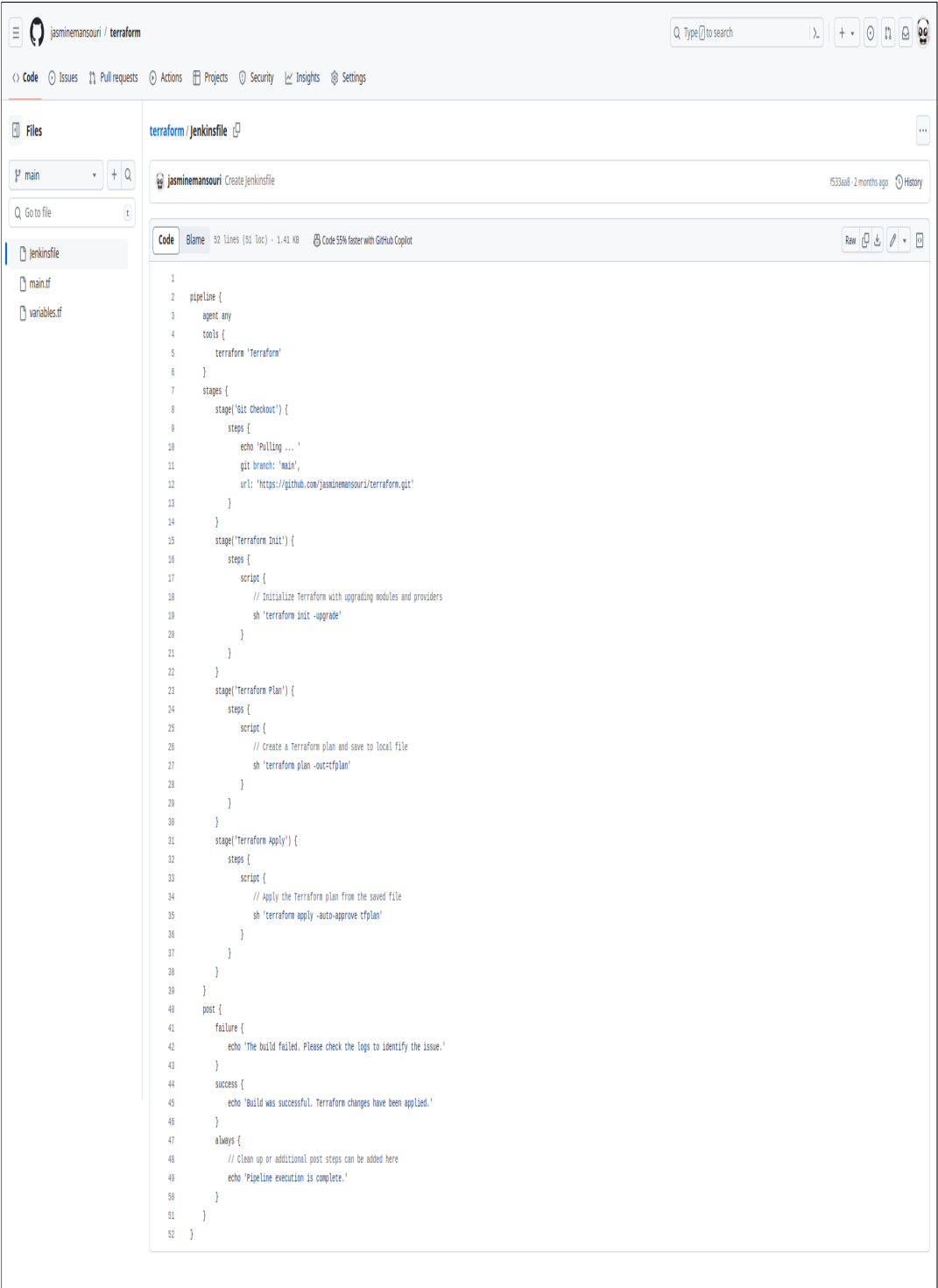
La capture suivante montre la configuration des répertoires à surveiller dans Wazuh

```
<!-- Directories to check (perform all possible verifications) -->
<syscheck>
  <disabled>no</disabled>
  <frequency>43200</frequency>
  <scan_on_start>yes</scan_on_start>
  <!-- Directories to check (perform all possible verifications) -->
  <directories >/etc,/usr/bin,/usr/sbin</directories>
  <directories >/bin,/sbin,/boot</directories>
  <directories realtime="yes">/root</directories>
  <!-- Directories to check (perform all possible verifications) -->
```

FIGURE 4.23 : Configuration des répertoires à surveiller

2.5 La phase de l'automatisation

Nous avons mis en place un fichier Jenkins configurant un pipeline pour exécuter les commandes Terraform, afin d'initialiser, planifier et appliquer des changements d'infrastructure.



The screenshot shows a GitHub repository named 'jasminemansouri / terraform'. The 'jenkinsfile' is selected in the file browser on the left. The main area displays the Jenkinsfile code, which is a pipeline script for Terraform. The code includes stages for 'git Checkout', 'Terraform Init', 'Terraform Plan', and 'Terraform Apply', each with specific steps and shell commands. The pipeline is configured to run on the 'main' branch of the repository.

```
1 pipeline {
2   agent any
3   tools {
4     terraform 'terraform'
5   }
6   stages {
7     stage('git Checkout') {
8       steps {
9         echo 'Pulling ...'
10        git branch: 'main',
11        url: 'https://github.com/jasminemansouri/terraform.git'
12      }
13    }
14    stage('Terraform Init') {
15      steps {
16        script {
17          // Initialize Terraform with upgrading modules and providers
18          sh 'terraform init -upgrade'
19        }
20      }
21    }
22    stage('Terraform Plan') {
23      steps {
24        script {
25          // Create a Terraform plan and save to local file
26          sh 'terraform plan -out=tfplan'
27        }
28      }
29    }
30    stage('Terraform Apply') {
31      steps {
32        script {
33          // Apply the Terraform plan from the saved file
34          sh 'terraform apply -auto-approve tfplan'
35        }
36      }
37    }
38  }
39  post {
40    failure {
41      echo 'The build failed. Please check the logs to identify the issue.'
42    }
43    success {
44      echo 'Build was successful. Terraform changes have been applied.'
45    }
46    always {
47      // Clean up or additional post steps can be added here
48      echo 'Pipeline execution is complete.'
49    }
50  }
51 }
52 }
```

FIGURE 4.24 : Fichier Jenkins pour la provisionnement automatique avec Terraform

Nous avons créé un fichier Jenkins configurant un pipeline pour exécuter un playbook Ansible afin de configurer les adresses IP des machines virtuelles.

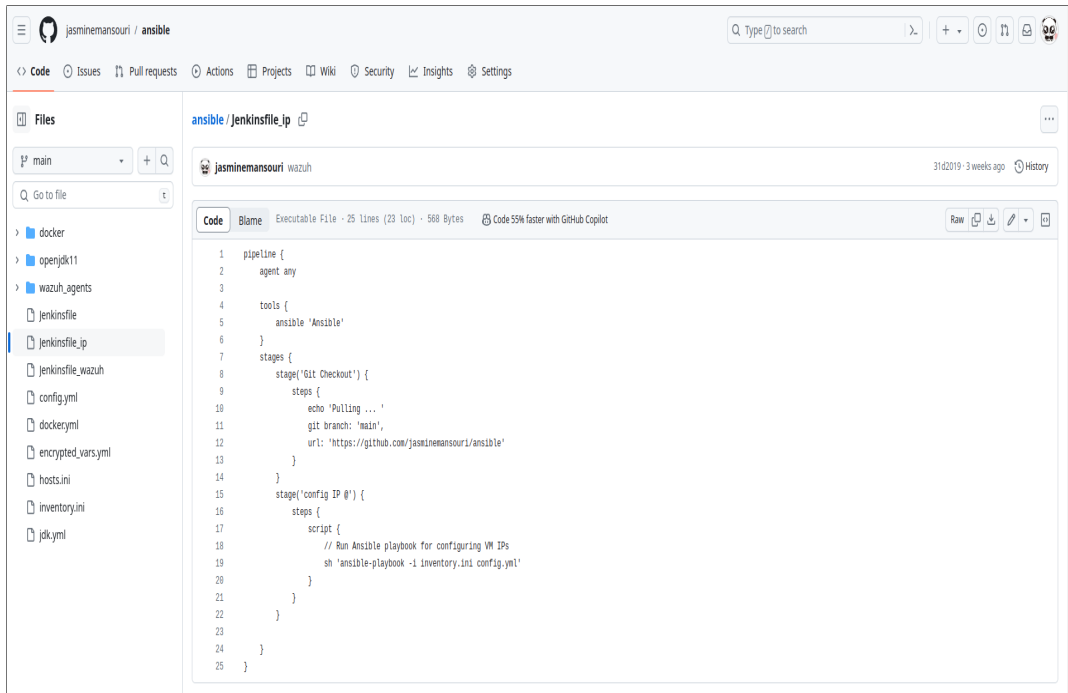


FIGURE 4.25 : Fichier Jenkins pour la configuration automatique des IPs

Nous avons établi un fichier Jenkins configurant un pipeline pour installer Docker et OpenJDK 11 en utilisant des playbooks Ansible.

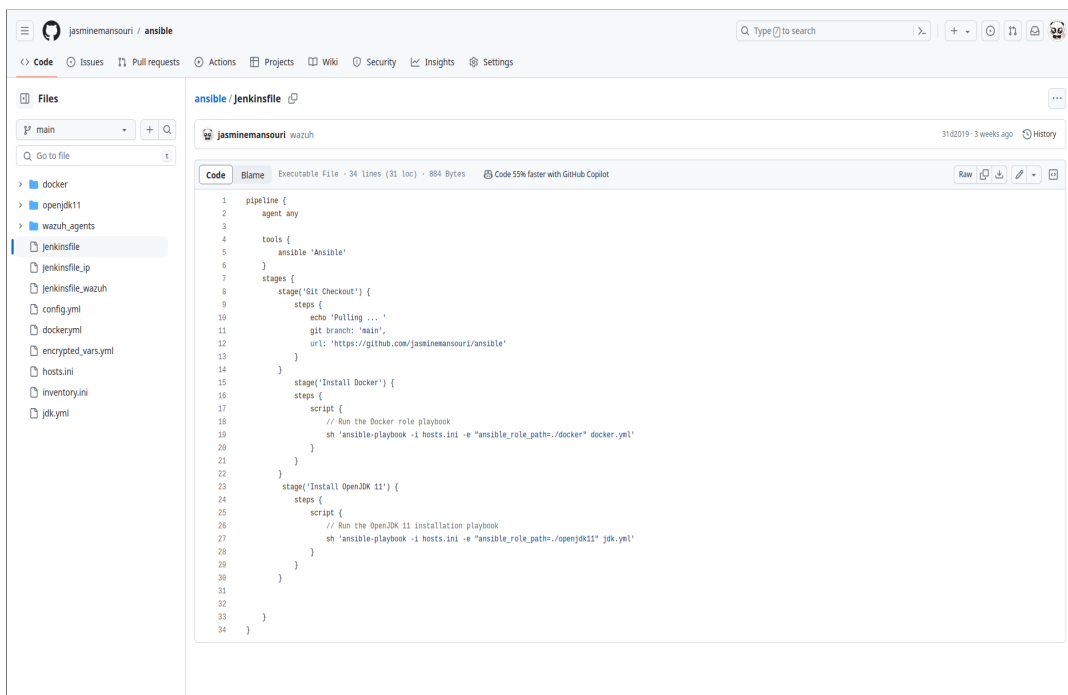


FIGURE 4.26 : Jenkinsfile de l'installation automatique de JDK et Docker

La capture ci-dessous montre notre fichier Jenkins configuré pour automatiser le déploiement de agents Wazuh via Ansible.

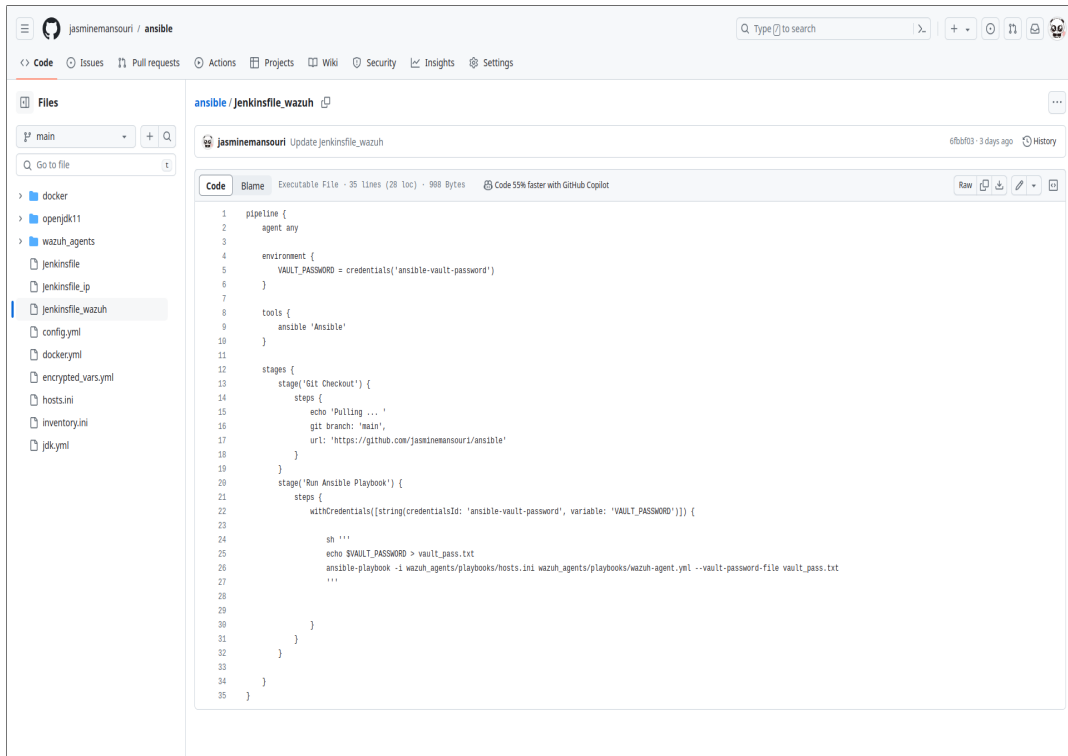


FIGURE 4.27 : Jenkinsfile de l'installation automatique des agnets Wazuh

3 Résultat

La figure 4.28 montre le lancement avec succès de pipeline Jenkins qui sert à exécuter les commandes présentés dans son fichier jenkins pour générer les machines virtuelles.

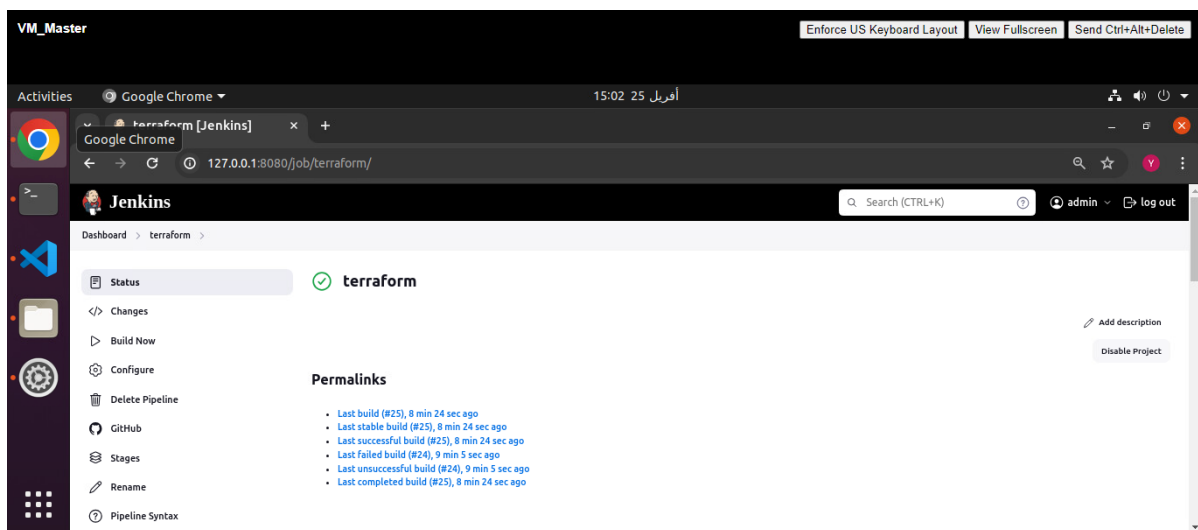
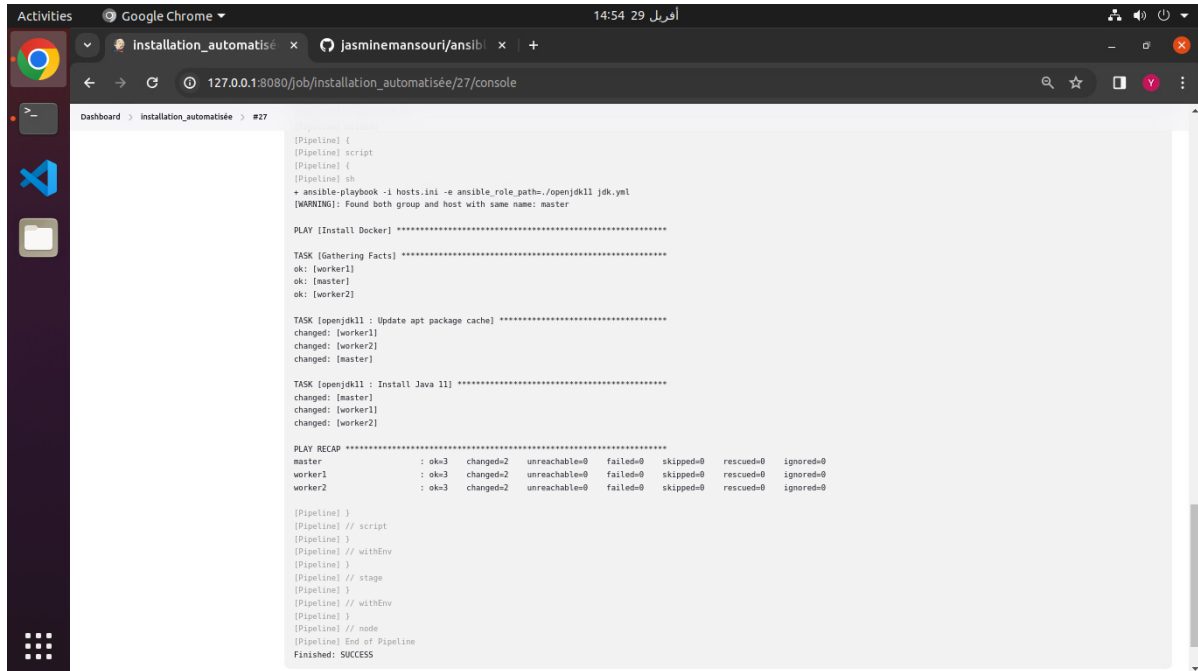


FIGURE 4.28 : Lancement du pipeline Terraform avec succès

3.1 Résultat de la configuration automatisée

La capture ci-dessous montre l'écran de console Jenkins affichant les résultats du playbook Ansible, détaillant l'installation de Docker et de Java 11 sur plusieurs nœuds, en indiquant le succès final de l'exécution du pipeline.



```
[Pipeline] {
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ ansible-playbook -i hosts.ini -e ansible_role_path=/openjdk11 jdk.yml
[WARNING]: Found both group and host with same name: master

PLAY [Install Docker] *****

TASK [Gathering Facts] *****
ok: [worker1]
ok: [master]
ok: [worker2]

TASK [openjdk11 : Update apt package cache] *****
changed: [worker1]
changed: [worker2]
changed: [master]

TASK [openjdk11 : Install Java 11] *****
changed: [master]
changed: [worker1]
changed: [worker2]

PLAY RECAP *****
master      : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
worker1     : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
worker2     : ok=3  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

FIGURE 4.29 : Résultat du playbook Ansible sur l'écran du console Jenkins

la figure 4.30 montre le fichier contenant les variables chiffrées utilisées par Ansible pour des opérations sécurisées dans nos playbooks. Nous avons utilisé Vault pour le cryptage de ces variables afin d'assurer leur sécurité.

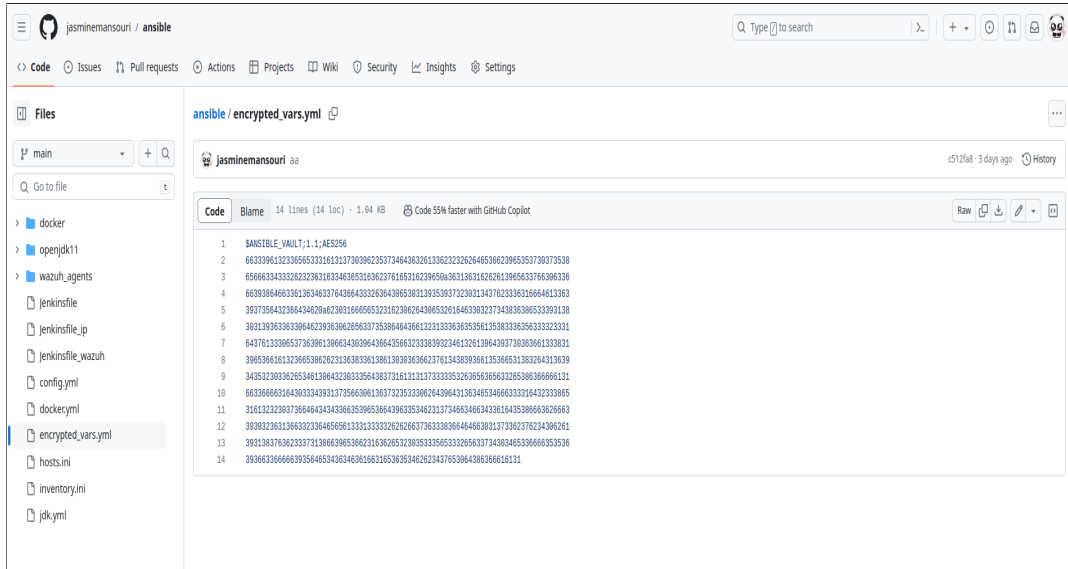


FIGURE 4.30 : Résultat du cryptage par Vault

La figure 4.31 montre l'interface de Jenkins avec le pipeline configuré pour l'installation automatisée des agents Wazuh. Les différentes étapes du pipeline, y compris la vérification du code, l'installation des outils, la récupération du dépôt Git et l'exécution du playbook Ansible, sont visualisées avec des résultats de succès ou d'échec.

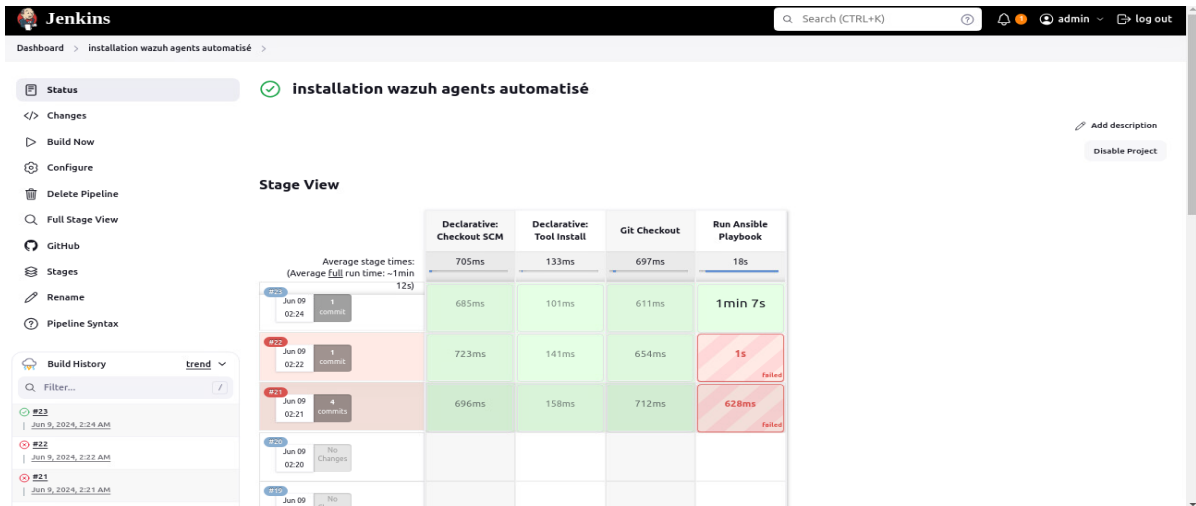


FIGURE 4.31 : Installation automatisée des agents Wazuh avec succès

3.2 Résultat de l'analyse de conformité

La figure 4.32 montre l'interface de Wazuh pour l'évaluation de la configuration de sécurité basée sur le benchmark CIS Ubuntu Linux 20.04 LTS.

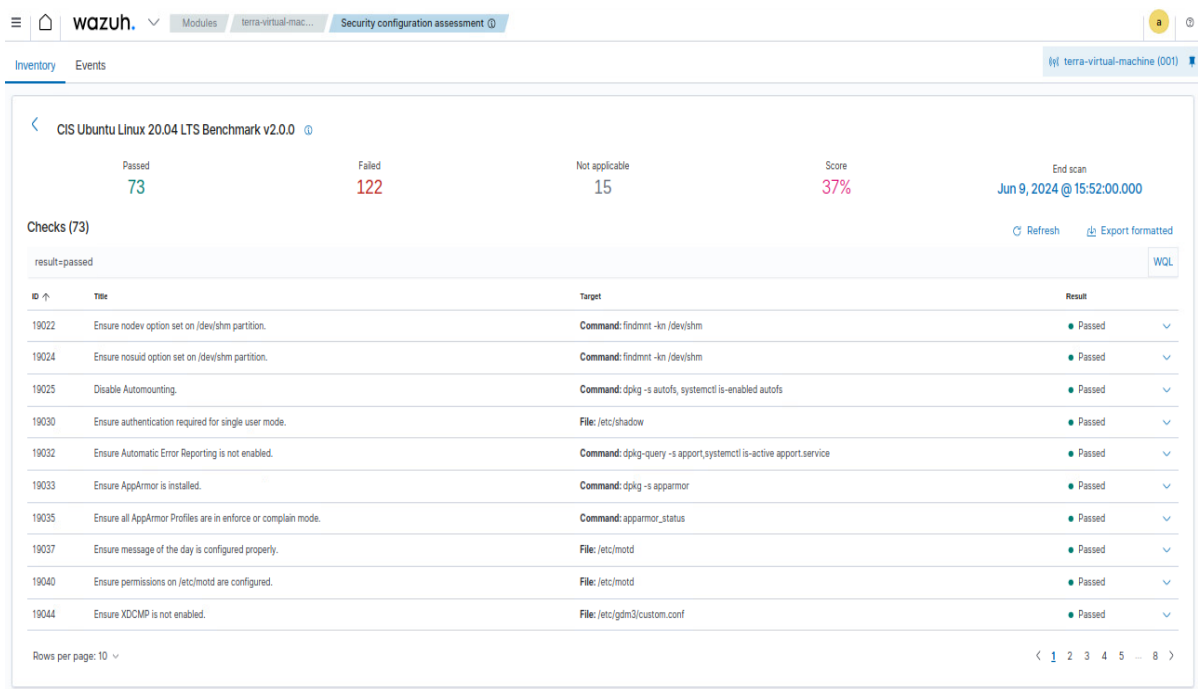


FIGURE 4.32 : Analyse de conformité

La figure 4.33 montre la vérification des permissions de fichiers système, où la commande pour vérifier les permissions sur /etc/gshadow a réussi, tandis que celle pour /etc/shells a échoué. Le fichier /etc/shells nécessite des permissions spécifiques pour assurer la sécurité des chemins d'accès des shells de connexion.

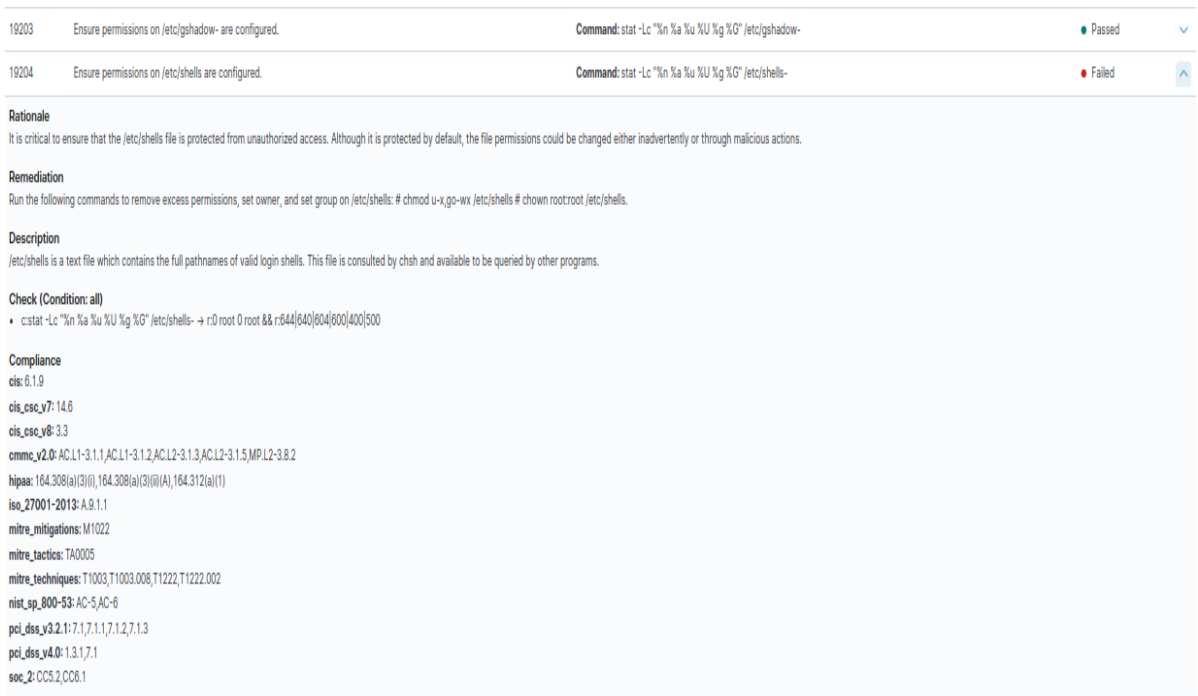
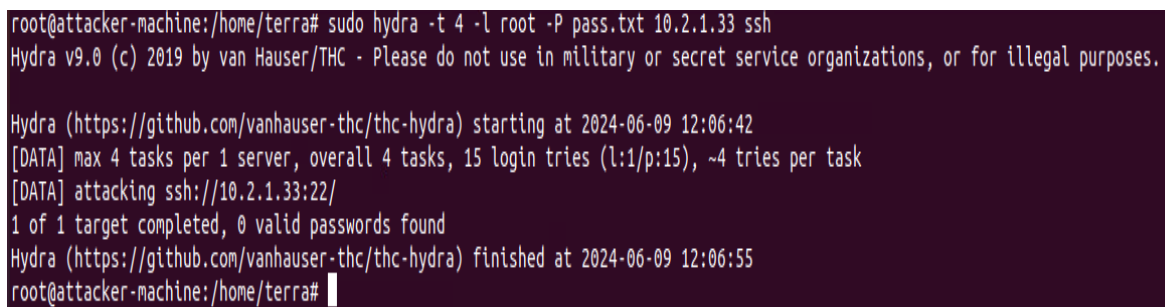


FIGURE 4.33 : SCA détaillé pour la vérification des permissions de fichiers système

3.3 Simulation des attaques

3.3.1 Détection et blocage des attaques par force brute SSH

Comme première étape nous avons lancé une attaque par force brute SSH utilisant l'outil Hydra pour tester les mots de passe à partir du fichier pass.txt contre l'agent Wazuh victime.



```
root@attacker-machine:/home/terra# sudo hydra -t 4 -l root -P pass.txt 10.2.1.33 ssh
Hydra v9.0 (c) 2019 by van Hauser/THC - Please do not use in military or secret service organizations, or for illegal purposes.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-06-09 12:06:42
[DATA] max 4 tasks per 1 server, overall 4 tasks, 15 login tries (l:1/p:15), ~4 tries per task
[DATA] attacking ssh://10.2.1.33:22/
1 of 1 target completed, 0 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-06-09 12:06:55
root@attacker-machine:/home/terra#
```

FIGURE 4.34 : Attaque force brute SSH

Comme résultat, la figure 4.35 montre les alertes affichées par wazuh dashboard suite à la détection de l'attaque brute force ainsi que la réponse active de wazuh en bloquant l'adresse IP malveillante.

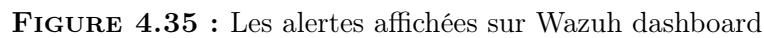


FIGURE 4.36 : Fichier JSON détaillé de la réponse active firewall-drop

3.3.2 Détection et suppression des logiciels malveillants à l'aide de l'intégration avec VirusTotal

Comme première étape nous avons téléchargé le fichier malveillant "eicar" dans le répertoire /root de l'agent à surveiller

```
root@yasmine-VM:~# sudo curl -Lo /root/eicar.com https://secure.eicar.com/eicar.com && sudo ls -lah /root/eicar.com
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100    68    100    68      0      0     322      0  --:--:--  --:--:--  --:--:--  320
-rw-r--r-- 1 root root 68 19:59 10 جول /root/eicar.com
root@yasmine-VM:~# ls
eicar.com  snap  VMwareTools-10.3.24-18733423.tar.gz  vmware-tools-distrib
root@yasmine-VM:~# pwd
/root
root@yasmine-VM:~#
```

FIGURE 4.37 : Téléchargement du fichier malveillant dans le répertoire /root

Comme résultat, la figure 4.38 montre la détection de virusTotal de l'ajout du fichier malveillant ainsi que la réponse active de Wazuh envers cette attaque en supprimant le fichier détecté.

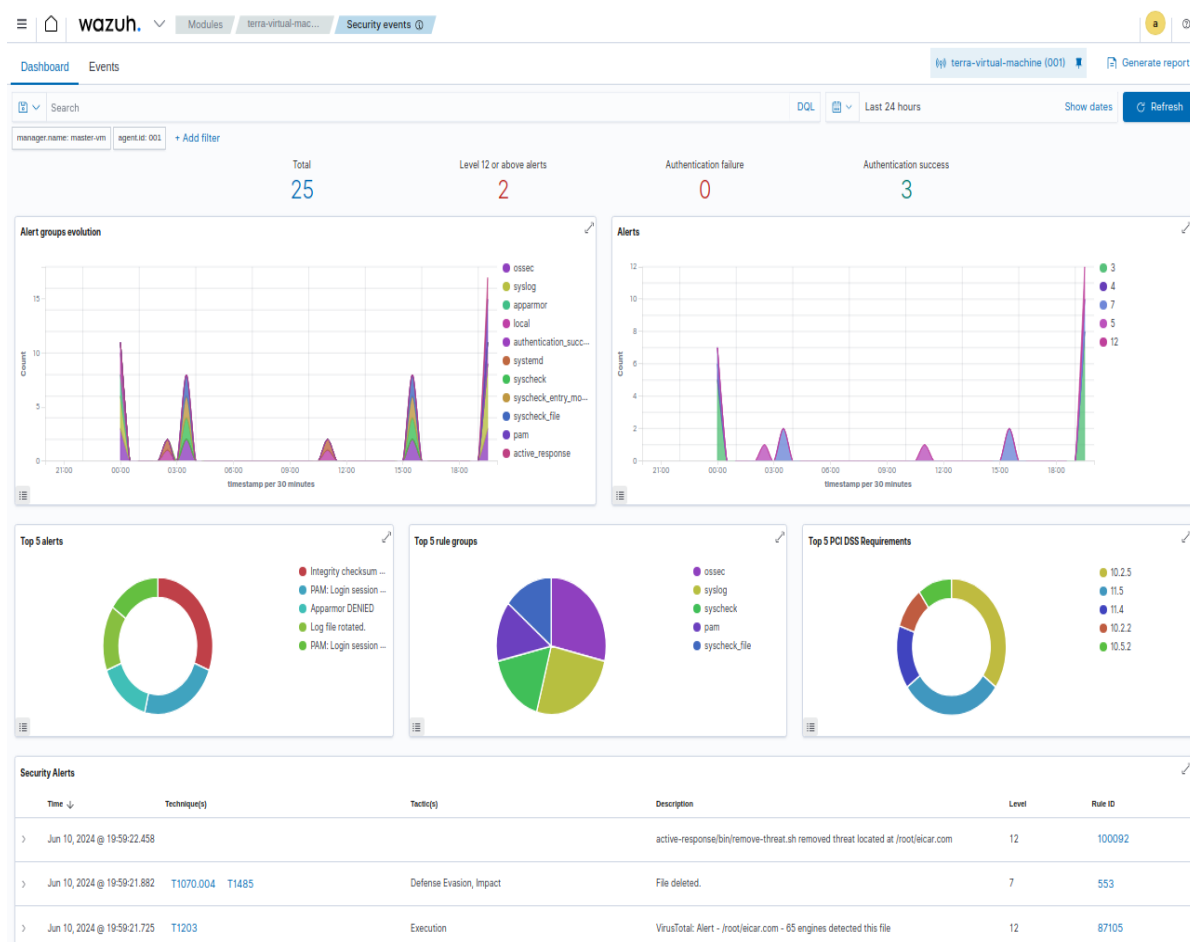


FIGURE 4.38 : Résultat de l'intégration

Conclusion

Ce chapitre représente la réalisation et la mise en place de notre projet. En effet, nous avons décrit les environnements matériels et logiciels sur lesquels nous avons construit notre infrastructure. Nous avons ensuite passé à la présentation de quelques interfaces qui montrent l'implémentation de notre solution et enfin nous avons clôturé ce chapitre par la présentation des résultats .

Conclusion générale

Ce projet nous a permis d'automatiser les meilleures pratiques de sécurité dans l'infrastructure en utilisant Terraform pour le provisionnement et Ansible pour la configuration sécurisée des serveurs. L'intégration de Wazuh a joué un rôle crucial dans la conformité des configurations et la détection et réponse aux menaces, assurant ainsi une surveillance et une protection continues de l'infrastructure.

Diverses technologies ont été mises en œuvre au sein de ce projet afin de garantir la sécurité et la fiabilité de l'infrastructure. La combinaison de ces technologies a permis de mettre en place une solution à forte valeur ajoutée, offrant une externalisation efficace non seulement au niveau de l'architecture mais également sur le plan opérationnel.

Cette solution ne pourrait être efficace sans un déploiement robuste et une surveillance continue. Nous avons ainsi pu répondre aux exigences de sécurité et de conformité du client en proposant une solution adaptée à ses besoins spécifiques. En automatisant les processus, nous avons optimisé les coûts et assuré une haute disponibilité des services.

Nous projetons de continuer à améliorer cette infrastructure en intégrant des fonctionnalités avancées et en explorant des outils supplémentaires pour renforcer encore plus la sécurité et la performance. L'objectif est d'assurer une infrastructure évolutive et durable qui réponde aux besoins croissants de l'organisation.

Enfin, ce projet a été une opportunité d'intégration et d'apprentissage professionnel, qui permet de travailler en collaboration avec les équipes et d'acquérir des compétences précieuses dans l'automatisation de la sécurité de l'infrastructure. Nous avons surmonté divers défis et mis en place une solution pérenne qui apporte une valeur significative à l'organisation.

Webographie

- [1] M. ROUSE, *Virtualisation*, [Accès le 20-Février-2024], 2015. adresse : <https://www.lemagit.fr/definition/Virtualisation>
- [2] ELIT-TECHNOLOGIES, *Qu'est-ce que la virtualisation des serveurs*, [Accès le 23-Février-2024], 2014. adresse : <http://www.elit-technologies.fr/qu-est-ce-que-virtualisation-serveurs/>
- [3] WIKIPEDIA, *Réseau de stockage SAN*, [Accès le 23-Février-2024], 2017. adresse : https://fr.wikipedia.org/wiki/Réseau_de_stockage_SAN
- [4] C. NEVEUX, *méthodes d'accès du stockage*, [Accès le 15-Février-2024], 2015. adresse : <http://adminsystnet.blog.free.fr/index.php?post/2015/09/08/Différence-entre-DAS-NAS-et-SAN>
- [5] M. ROUSE, *Hyperviseur*, [Accès le 26-Février-2024], 2015. adresse : <https://www.lemagit.fr/definition/Hyperviseur>
- [6] F. BURNEL, *Les types d'hyperviseurs*, [Accès le 26-Février-2024], 2012. adresse : <https://www.it-connect.fr/les-types-dhyperviseurs/>
- [7] E. DEBRAY, *comparaison des principaux hyperviseurs*, [Accès le 28-Février-2024], 2012. adresse : <https://cisccodatacenter.wordpress.com/2011/04/19/comparer-les-principaux-hyperviseurs/>
- [8] PUBLITHINGS., *Définition Data Center : qu'est-ce qu'un centre de données ?* [Accès le 7-Mars-2024], 2018. adresse : <https://www.lebigdata.fr/definition-data-center-centre-donnees>
- [9] JEAN., *Classification des centres de données : les "tiers", et leur signification*. [Accès le 7-Mars-2024], 2017. adresse : <http://www.eds.fr/2017/06/05/classification-centres-de-donnees-tiers-signification/>
- [10] T. G. P. MELL, *The NIST Definition of Cloud Computing*. [Accès le 11-Mars-2024], 2011. adresse : <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>

- [11] J.-P. FIGER, *L'informatique en nuage [Cloud Computing]*, [Accès le 13-Mars-2024], 2017. adresse : <https://www.figer.com/Publications/nuage.htm>
- [12] T. CHARDONNENS., *Les enjeux du Cloud Computing*. [Accès le 18-Mars-2024], 2012. adresse : https://diuf.unifr.ch/main/is/sites/diuf.unifr.ch.main.is/files/documents/student-projects/B-2012_Thibaud_Chardonrens.pdf,
- [13] MICROSOFT., *Que sont les clouds hybrides ?* [Accès le 18-Mars-2024], 2018. adresse : <https://azure.microsoft.com/fr-fr/overview/what-are-private-public-hybrid-clouds/>
- [14] H. BOUZIDI., *Le Cloud Communautaire, nouveau modèle en vogue*, [Accès le 18-Mars-2024]., 2016. adresse : <https://blog.outscale.com/fr/le-cloud-communautaire-nouveau-modele-en-vogue>
- [15] T. F. FEED, *Les niveaux de services du Cloud Computing*. [Accès le 18-Mars-2024]., 2023. adresse : <https://thefuturefeed.medium.com/diving-into-the-cloud-understanding-the-differences-between-iaas-saas-and-paas-9a47fc9ba477>
- [16] APPREND.A., *IaaS*. [Accès le 20-Mars-2024]., 2018. adresse : <https://apprenda.com/library/paas/iaas-paas-saas-explained-compared/>
- [17] SALESFORCE., *PaaS*. [Accès le 20-Mars-2024]., 2018. adresse : <https://www.salesforce.com/fr/learning-centre/tech/paas/>
- [18] SALESFORCE., *SaaS*. [Accès le 20-Mars-2024]., 2018. adresse : <https://www.salesforce.com/fr/learning-centre/tech/saas/>
- [19] SURBHI., *DevOps*, [Accès le 25-Mars-2024]., 2023. adresse : <https://www.nwking.com/what-is-devops>
- [20] REDHAT., *CI/CD*, [Accès le 03-Avril-2024]., 2022. adresse : <https://www.redhat.com/fr/topics/devops/what-cicd-pipeline>
- [21] IBM, *DevSecOps*, [Accès le 08-Avril-2024]., 2022. adresse : <https://www.ibm.com/fr-fr/topics/devsecops>
- [22] B. PLAN, *Scrum*, [Accès le 5-Février-2024], 2023. adresse : <https://bubbleplan.net/blog/agile-scrum-gestion-projet/>

يهدف هذا المشروع إلى وضع أفضل ممارسات الأمان بصفة آلية في البنية التحتية باستخدام أدوات لتوفير الموارد وتكوين الخوادم بشكل آمن. يتم دمج أدوات لضمان الامتثال للتكوينات وكأداة للكشف عن التهديدات والاستجابة لها. يركز المشروع على تحسين الأمان، الامتثال، وخفض التكاليف من خلال الأتمتة المستمرة.

كلمات مفاتيح :

الحوسبة السحابية، الافتراضية، إشراف ، البنية التحتية كخدمة ، البنية التحتية كرمز ، الأمان، تقييم الأمان، مراقبة تكامل الملفات .

Résumé

Le projet vise à automatiser les meilleures pratiques de sécurité dans l'infrastructure en utilisant Terraform pour le provisionnement et Ansible pour la configuration sécurisée des serveurs. Wazuh est intégré pour la conformité des configurations et comme outil EDR pour la détection et la réponse aux menaces. Le projet se concentre sur l'amélioration de la sécurité, la conformité et la réduction des coûts grâce à l'automatisation

Mots clés :

Cloud, Virtualisation, Conformité, VMware vSphere, Supervision, IaaS , IaC, Sécurité, Terraform, Ansible, Wazuh , Vault , SCA, FIM

Abstract

The project aims to automate best security practices in the infrastructure using Terraform for provisioning and Ansible for secure server configuration. Wazuh is integrated for configuration compliance and as an EDR tool for threat detection and response. The project focuses on enhancing security, compliance, and cost reduction through automation.

Keywords :

Cloud, Virtualization, VMware vSphere, Supervision, IaaS, IaC, Security, Terraform, Ansible, Wazuh, Vault, SCA, FIM.